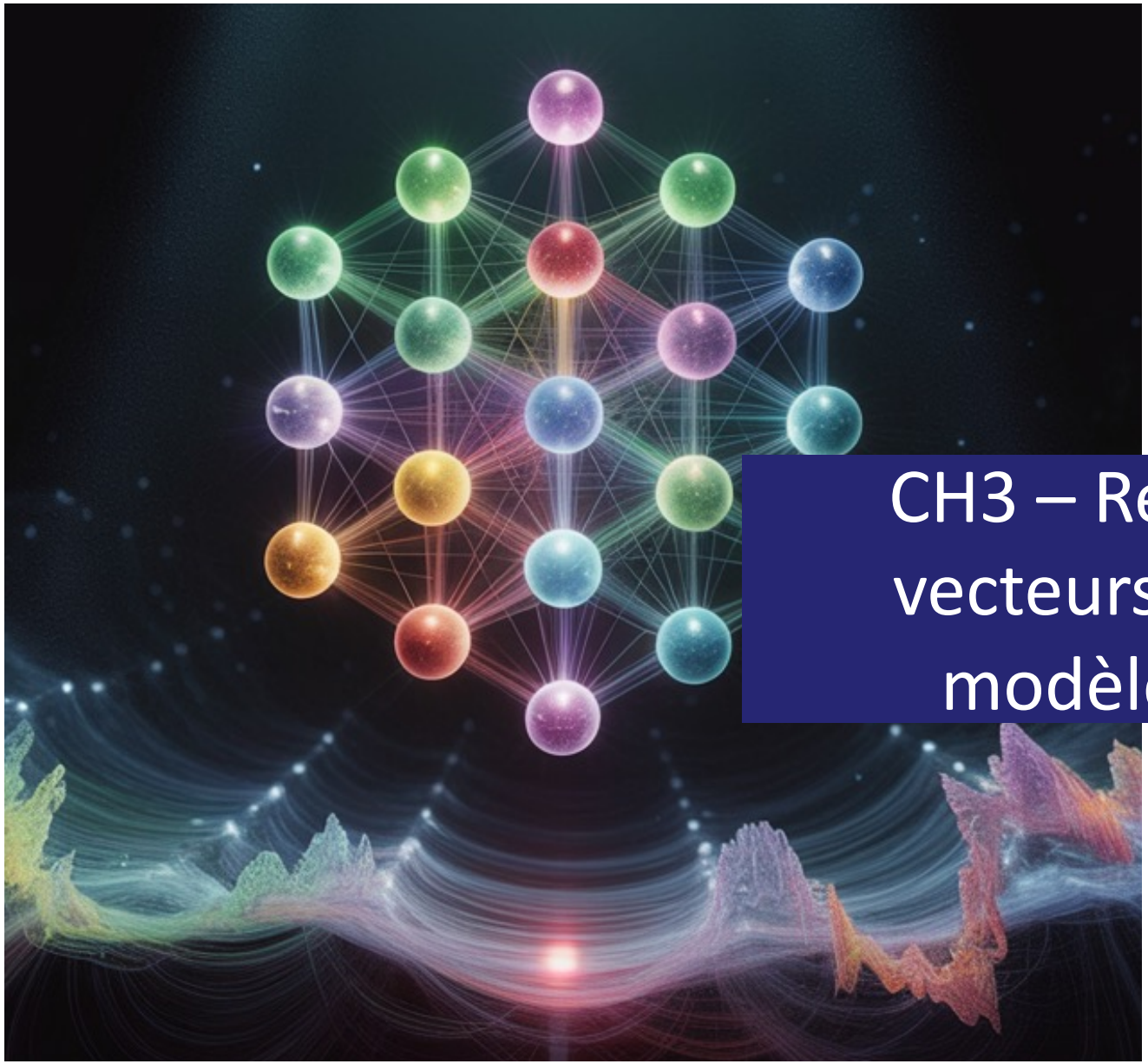


UNIVERSITE DE CORSE
Master Informatique
parcours DFS et DE
2ème année
2026-2027

CH3 – Représenter par des vecteurs : fondements des modèles d'embeddings

Evelyne VITTORI
vittori_e@univ-corse.fr

Partie I - Représenter le monde : des symboles aux vecteurs

CH1 – Représenter le monde en IA : un enjeu central

CH2 – Représentations en IA symbolique : des origines aux ontologies

CH3 – Représenter par des vecteurs : fondements des modèles d'embeddings

CH4 – Indexation et BD vectorielles : principes et premières explorations

Objectifs du chapitre

- Poser les **bases mathématiques** nécessaires
- Identifier les limites des représentations simples (one-hot, TF-IDF)
- Introduire la notion d'**embedding** (vecteurs denses appris)
- Découvrir les modèles **statiques** et leurs limites
- Explorer les modèles **contextuels**

Activités guidées :

- Manipuler des Représentations simples
- Générer et comparer des embeddings avec différents modèles

Word2Vec, FastText, BERT, SentenceBERT

CH3 – Représenter par des vecteurs : Fondements des embeddings

1. Bases mathématiques
2. Premières représentations vectorielles du langage
3. Vers les représentations denses : la notion d’embedding
4. Modèles d'embeddings statiques
5. Modèles d’embeddings contextuels et sentence embedings

La vectorisation n’est pas née avec l’IA

Premiers vecteurs : purements syntaxiques sans aucune ambition sémantique

- Besoins pratiques dans deux domaines :
  - 🔍 Recherche d’information (IR)
  - Statistique textuelle
- Un usage fondamental
  - Rendre les mots et les documents manipulables numériquement

Pourquoi représenter en vecteurs en IA ?

Uniformiser les données 🧩

- Tout devient un vecteur dans un espace $\mathbb{R}^d$:
- un mot , une image , un son → vecteur

Faciliter le calcul 🧮

- mesurer la similarité (ex. cosinus, distance euclidienne)
- faire des opérations algébriques

Rendre possible l'apprentissage automatique 🤖

- Algorithmes de ML/DL = entrées numériques
- Sans vecteurs → pas de descente de gradient

Représentations vectrielles du langage : une évolution historique

One-hot encoding
années 1950–1980

Représentation symbolique de chaque mot par un vecteur binaire de grande dimension (un 1 pour le mot, 0 ailleurs).

Ex. : vocabulaire de 5 mots

chat	[1, 0, 0, 0, 0]
chien	[0, 1, 0, 0, 0]
maison	[0, 0, 1, 0, 0]
voiture	[0, 0, 0, 1, 0]
arbre	[0, 0, 0, 0, 1]

Matrices de cooccurrences
années 1980–1990

Représentation **distributionnelle** : on compte la **fréquence** de cooccurrence des mots dans une fenêtre de contexte.

	chat	chien	maison	...
chat	0	5	2	...
chien	4	0	3	...
maison	1	2	0	...
...	...	...	...	...

TF-IDF
années 1990–2000

Pondération des mots par leur fréquence dans un document (TF) et leur rareté dans le corpus (IDF). Représentation vectorielle creuse et interprétable.

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

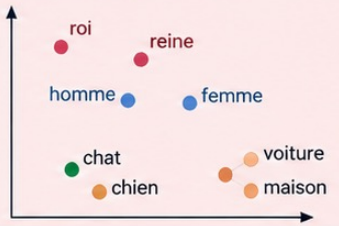
avec

$TF(t,d)$ = fréquence du mot t dans le document d

$$IDF(t) = \log \frac{N}{df(t)}$$

Embeddings de mots
années 2010

Représentations denses et continues apprises automatiquement à partir de grands corpus (Word2Vec, GloVe, FastText, etc.). Les mots proches ont des vecteurs proches.



Embeddings de phrases
années 2018–aujourd'hui

Représentations denses de phrases entières, apprises avec des modèles de langage profonds (BERT, SBERT, etc.). Captent le sens global et le contexte.

1950
Premières représentations symboliques des mots (traitement du langage basé sur des règles)

1980
Approches distributionnelles et matrices de cooccurrences

1990
TF-IDF (recherche d'information, machine learning classique)

2010
Word2Vec, GloVe, FastText (embeddings de mots)

2018
BERT, modèles de phrases (embeddings de phrases)

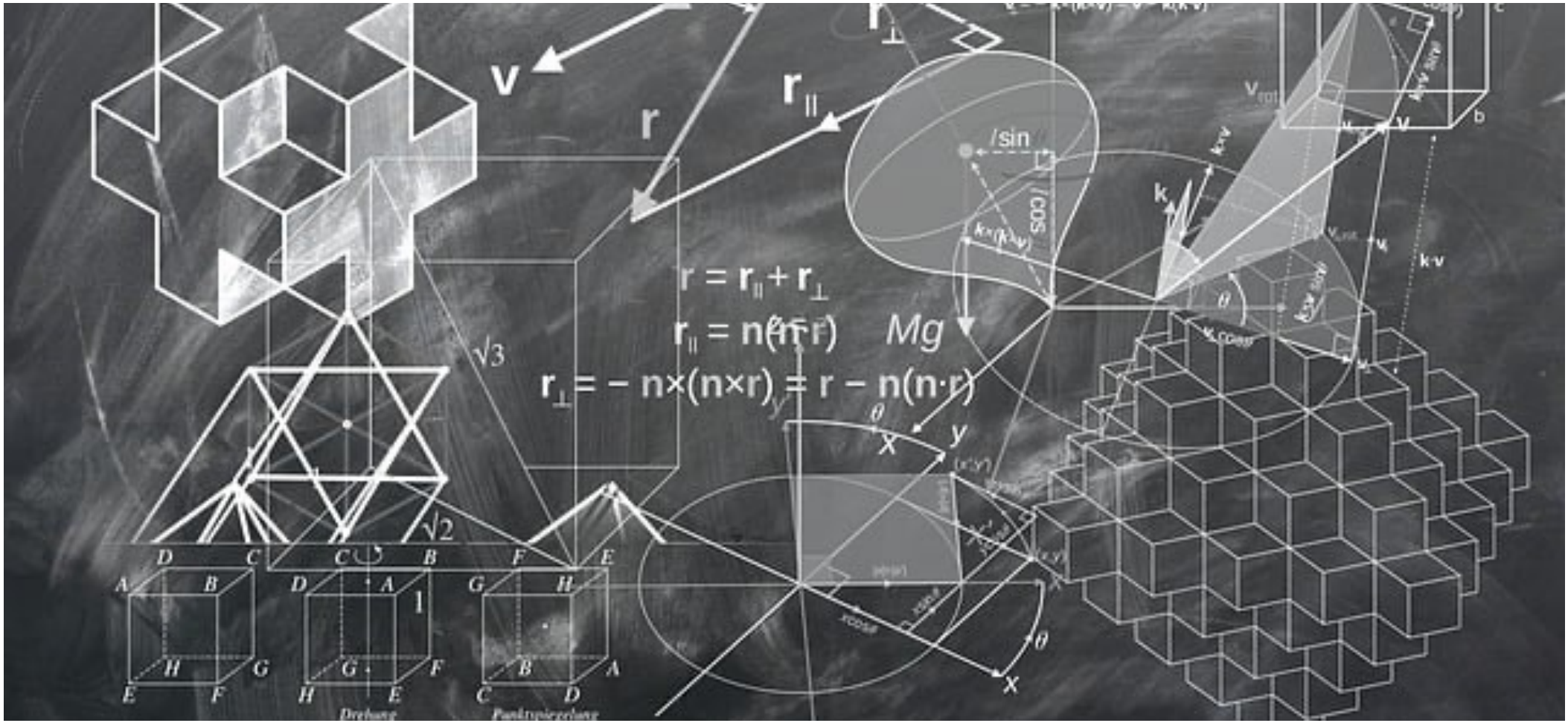
2020+

Approches symboliques
(règles, représentations explicites)
années 1950 – 1980

Machine Learning classique
(représentations statistiques, ingénierie de caractéristiques)
années 1980 – 2010

Deep Learning
(représentations apprises automatiquement)
années 2010 – aujourd'hui

1 – Bases mathématiques



Qu'est ce qu'un vecteur ?

- Liste ordonnée de nombres.
- Sert à représenter des objets dans un espace mathématique.
- Exemples

Position géographique

(48,85, 2,35)

Couleur RGB

(255, 0, 0)

Profil utilisateur

(25, 12, 45, 2000)

Notion de vecteur en algèbre linéaire

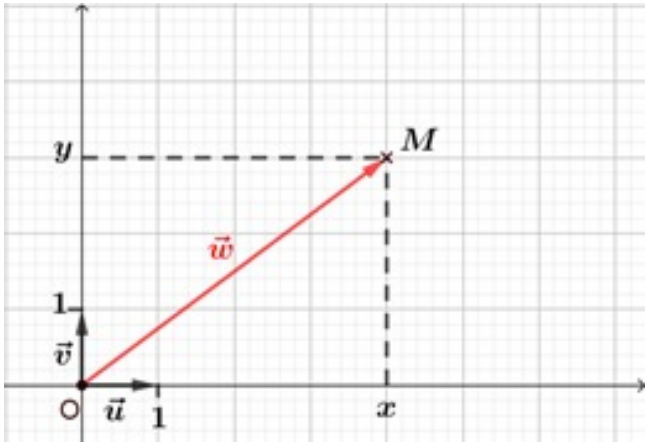
- Suite ordonnée de nombres réels :

$$v = (x_1, x_2, \dots, xn)$$

- En algèbre linéaire, on identifie ce vecteur avec la flèche qui va de l'origine jusqu'au point (x_1, x_2, …, x_n)

- **Norme d'un vecteur** : mesure de la longueur

$$|| v ||= \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$



Coordonnées de w =(x,y)

w(4,3)

$|| w ||= \sqrt{4^2 + 3^2} = \sqrt{25} = 5$

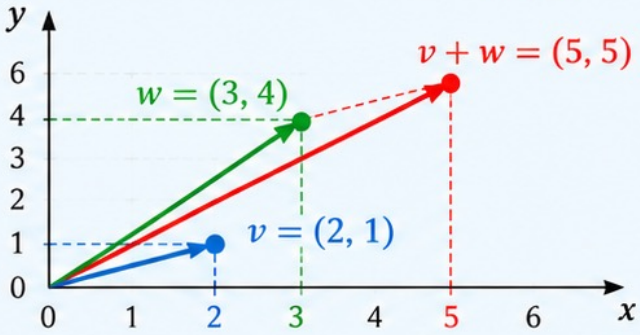
Opérations sur les vecteurs : exemples

Soient $v = (2, 1)$ et $w = (3, 4)$ (vecteurs dans $\mathbb{R}^2$)

1 Addition de vecteurs

$$v + w = (v_1 + w_1,\ v_2 + w_2)$$

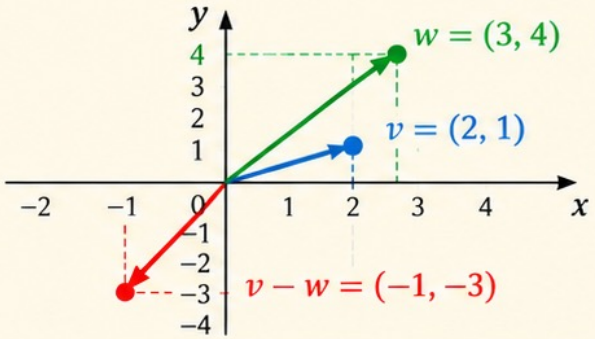
$$v + w = (2 + 3, 1 + 4) = (5, 5)$$



2 Soustraction de vecteurs

$$v - w = (v_1 - w_1,\ v_2 - w_2)$$

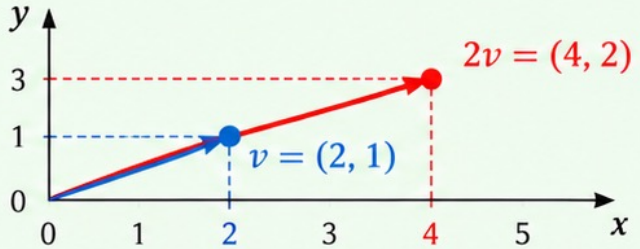
$$v - w = (2 - 3, 1 - 4) = (-1, -3)$$



3 Multiplication par un scalaire

$$\lambda v = (\lambda v_1,\ \lambda v_2)$$

$$2v = 2(2, 1) = (4, 2)$$



4 Produit scalaire de v et w

$$v \cdot w = \sum_i v_i w_i = v_1 w_1 + v_2 w_2$$

$$v \cdot w = 2 \times 3 + 1 \times 4 = 6 + 4 = 10$$

- $v \cdot w > 0$: vecteurs globalement dans la même direction.
- $v \cdot w = 0$: vecteurs orthogonaux.
- $v \cdot w < 0$: vecteurs globalement dans des directions opposées.

Espace vectoriel

- Cadre mathématique dans lequel on peut additionner des vecteurs et les multiplier par des scalaires.
- Les exemples ici utilisent des vecteurs de $\mathbb{R}^2$ pour pouvoir les représenter graphiquement.
- Les mêmes opérations s'appliquent aux vecteurs de dimension n ($\mathbb{R}^n$).

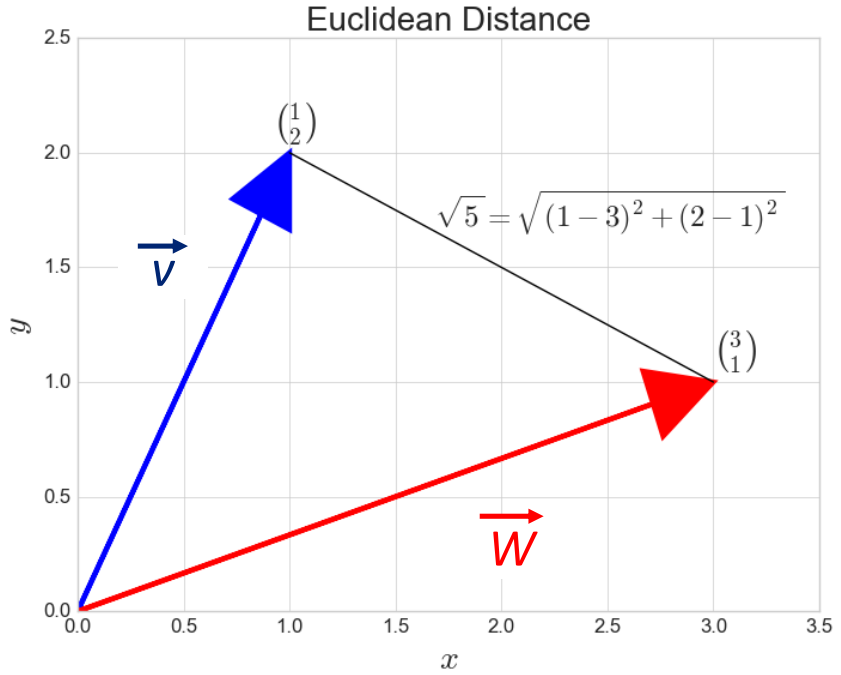
Distance euclidienne entre deux vecteurs

- Mesure la proximité entre deux vecteurs comme une distance géométrique.
- d(v,w =

||v−w||

$= \sqrt{(v_1 - w_1)^2 + (v_2 - w_2)^2 + \cdots + (v_n - w_n)^2}$

👉 Plus la distance est petite → plus les vecteurs sont proches



source : https://www.codearmo.com/media/uploads/2020/10/30/euclidean_distance_between_vectors_image.png

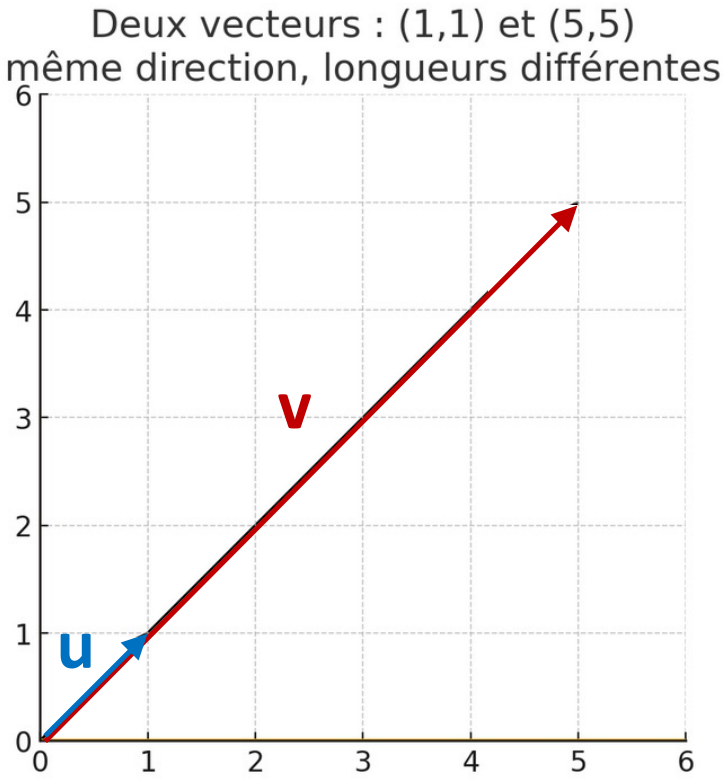
Pourquoi la distance euclidienne n'est pas adaptée à la similarité sémantique ?

- u=(1,1) v=(5,5)

  $||u-v||=\sqrt{(5-1)^2+(5-1)^2}=\sqrt{32}\approx 5.657$

- Deux vecteurs **parfaitement alignés** (même direction) peuvent avoir une distance euclidienne **grande** simplement parce que l'un est plus long !

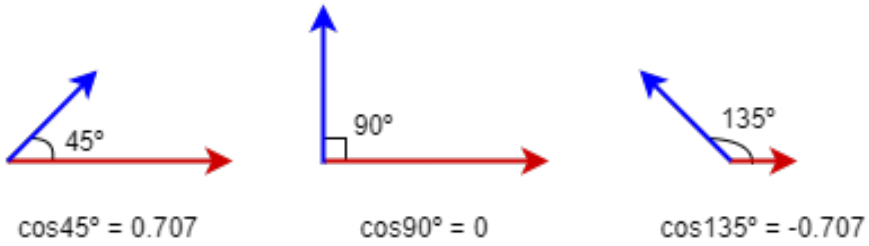
🔎 La distance euclidienne est sensible à la longueur, pas seulement à la direction



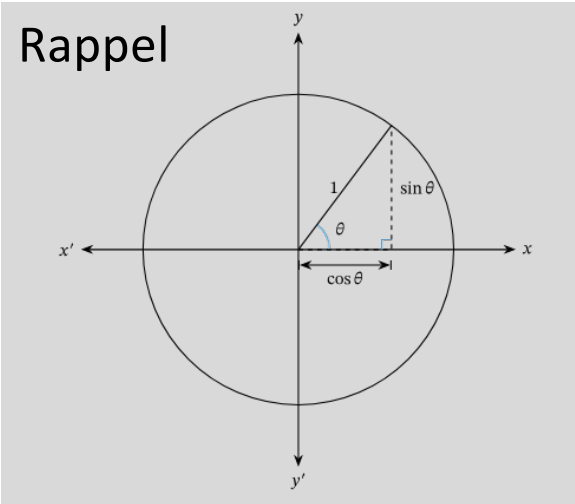
Similarité cosinus de deux vecteurs : plus adaptée à la mesure de la similarité sémantique

- La **similarité cosinus** est le cosinus de l'angle θ entre deux vecteurs
- Elle permet de comparer l'orientation entre deux vecteurs
  - $\text{sim}(v, w) = \frac{v \cdot w}{||v||\ ||w||} = \cos(v,w)$
- Valeurs entre -1 et 1 :
  - proche de 1 → vecteurs très similaires (même direction)
  - proche de 0 → indépendants
  - proche de -1 → opposés

👉 Plus l'angle est petit, plus les vecteurs représentent des objets proches (ou similaires)



source : https://www.ultipa.com/fr/docs/graph-analytics-algorithms/cosine-similarity



source : https://www.nagwa.com/fr/explainers/825143870519/

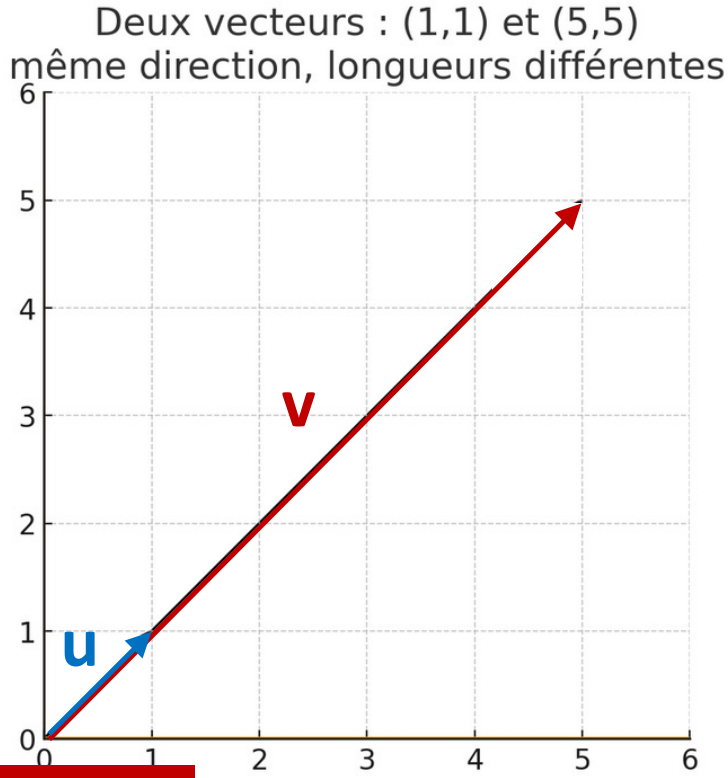
La similarité cosinus est focalisée sur le sens des vecteurs

- u=(1,1) v=(5,5)

sim(*u, v*) = cos(u,v) = 1

$$\cos(u,v) = \frac{u \cdot v}{\|u\|\|v\|} = \frac{1 \cdot 5 + 1 \cdot 5}{\sqrt{1^2 + 1^2}\,\sqrt{5^2 + 5^2}}$$

$$= \frac{10}{\sqrt{2}\,\sqrt{50}} = \frac{10}{\sqrt{2}\,5\sqrt{2}} = \frac{10}{10} = 1$$



- La similarité représente bien le fait que un et v sont parfaitement alignés

🔎 La similarité cosinus est particulièrement adaptée à la recherche de similarités sémantiques

Normalisation des vecteurs

- Vecteur normalisé =

  Vecteur de longueur 1 obtenu en divisant chaque coordonnée par la norme du vecteur initial

$$v=[2,-1,3]$$

$$\| v \| = \sqrt{2^2 + (-1)^2 + 3^2} \approx 3{,}74$$

$$\hat{v} = \frac{v}{\|v\|} = \left[\frac{2}{3{,}74}, \frac{-1}{3{,}74}, \frac{3}{3{,}74}\right] \approx [0.53, -0.27, 0.80]$$

Très utile pour la recherche sémantique !

- Pourquoi normaliser les vecteurs?
  - Rendre la distance euclidienne sensible au sens, pas à la taille
  - Simplifier le calcul de la similarité cosinus

Similarité cosinus = produit scalaire

  - si u et v sont normalisés alors

    sim(v, w) = cos(θ)=u·v

Orientation des vecteurs	Cosinus	Distance
Même direction	1	0
90°	0	√2
Opposés	−1	2

Vecteurs normalisés et similarités

- vecteurs normalisés

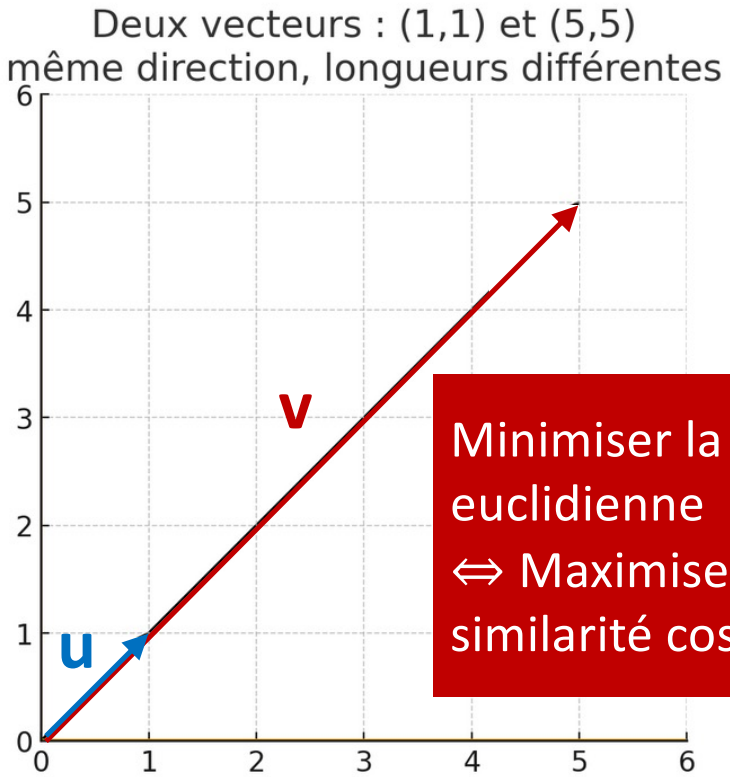
$$\hat{u} = \frac{u}{\|u\|} = \frac{(1,1)}{\sqrt{2}} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

$$\hat{v} = \frac{v}{\|v\|} = \frac{(5,5)}{5\sqrt{2}} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

- Distance euclidienne

$$\|\hat{u} - \hat{v}\| = \|0\| = 0$$

- Similarité cosinus (pas de changement!)
  - cos($\hat{u}$, $\hat{v}$)= $\hat{u}$. $\hat{v}$ = 1



Minimiser la distance euclidienne
⇔ Maximiser la similarité cosinus

Vecteurs normalisés et similarités

- **Sans normalisation**
  - La distance euclidienne dépend
    → **de la direction**
    → **et de la norme**
  - Un vecteur plus long paraît plus éloigné même s'il pointe dans le même sens

👉 Sans normalisation, la distance euclidienne mélange direction et norme

- **Avec normalisation**
  - Tous les vecteurs ont une norme égale à 1
  - **Similarité cosinus = Produit scalaire**

👉 Distance euclidienne et similarité cosinus sont directement reliées

$$\|\hat{u} - \hat{v}\|^2 = 2(1 - \cos(\theta))$$

Minimiser la distance euclidienne
⇔ Maximiser la similarité cosinus

Notion de distance cosinus

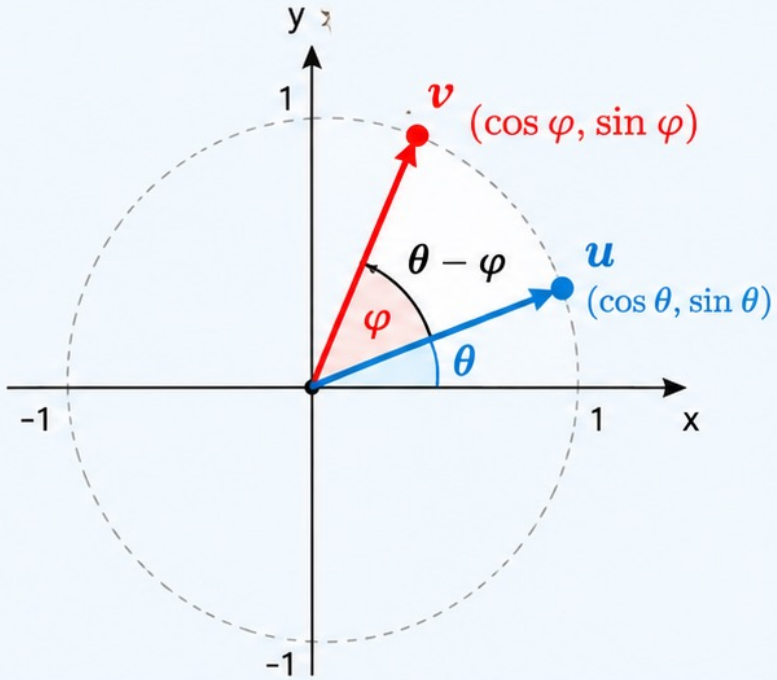
- Transforme la similarité cosinus en distance (mesure d'éloignement) :
  - $d_{cos}(u,v)=1-\cos(u,v)$
- Valeurs :
  - $0 \rightarrow$ vecteurs identiques
  - $1 \rightarrow$ vecteurs orthogonaux
  - $2 \rightarrow$ vecteurs opposés

🧠 **Pourquoi les deux existent ?**

- **Similarité cosinus** (entre -1 et 1) dit *"à quel point c'est proche"*
- **Distance cosinus** (entre 0 et 2) dit *"à quel point c'est loin"*

- **Plus la valeur est proche de 0, plus les vecteurs sont proches**
  - ex: vecteurs identiques : similarité = 1 et distance cosinus=0

Vecteurs normalisés : cosinus ou distance euclidienne ?



1 Vecteurs normalisés

On considère des vecteurs de norme 1 :

$$\|u\| = \|v\| = 1$$

Les vecteurs sont sur le cercle unité : seule la direction compte.

2 Relation entre cosinus et distance euclidienne

Pour deux vecteurs normalisés u et v :

$$\begin{aligned}\|u - v\|^2 &= \|u\|^2 + \|v\|^2 - 2u \cdot v = 2 - 2u \cdot v \\ &= 2 - 2\cos(\theta)\end{aligned}$$

3 Même information

Similarité cosinus ↑ ⟺ Distance euclidienne ↓

→ Même classement des voisins

Plus le cosinus est grand, plus la distance euclidienne est petite (sur des vecteurs normalisés).

4 Pourquoi choisir l'une ou l'autre ?

- Même **information** sur des vecteurs normalisés (elles sont reliées par $\|u - v\|^2 = 2 - 2\cos(\theta)$).
- Le choix dépend surtout de la métrique proposée ou optimisée par les algorithmes et outils (de recherche vectorielle, indexation, clustering, etc.).

Des vecteurs aux tenseurs : généraliser à plusieurs dimensions

Scalaire (0D)

Une seule valeur

5

Forme : ()
(pas de dimension)

Vecteur (1D)

Une liste de valeurs

2	5	8	1

Forme : (4)
4 éléments

Matrice (2D)

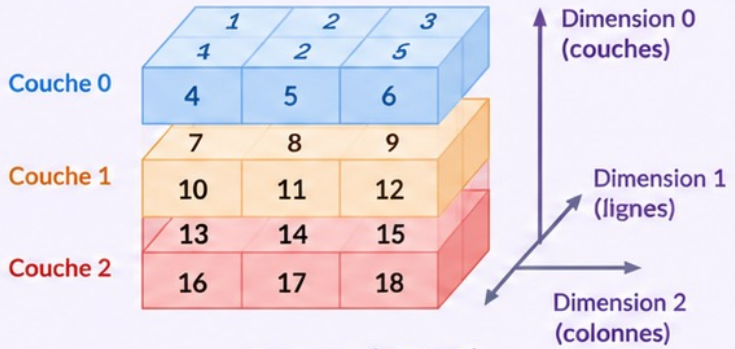
Lignes × colonnes

1	2	3
4	5	6

Forme : (2, 3)
2 lignes, 3 colonnes

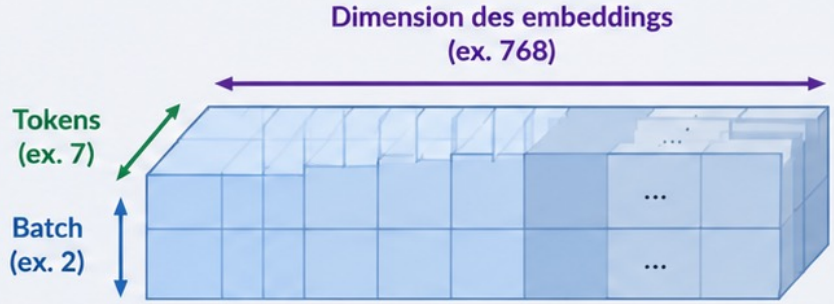
Tenseur (3D)

Un ensemble de matrices



Forme : (3, 2, 3)
3 matrices de taille 2 × 3

Exemple : sorties d'un modèle de type Transformer



Dimensions : [batch, tokens, dimension des embeddings]

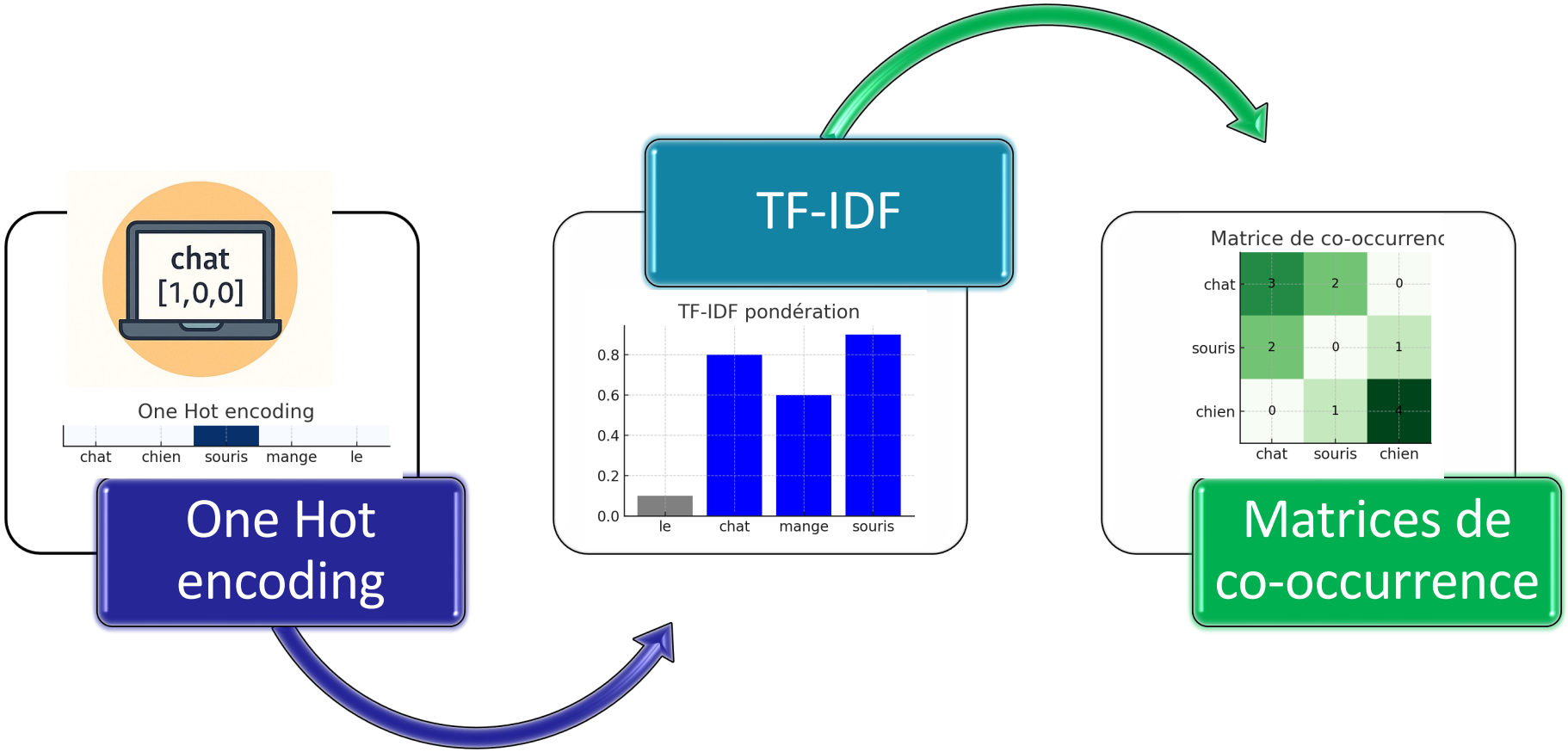
Par exemple : [2, 7, 768]

- 2 phrases dans le batch
- 7 positions de tokens par phrase
- un vecteur de 768 dimensions pour chaque token

En deep learning, un tenseur est un tableau multidimensionnel permettant de représenter et de manipuler des données numériques.

2 – Premières représentations vectorielles du langage : symboliques et statistiques

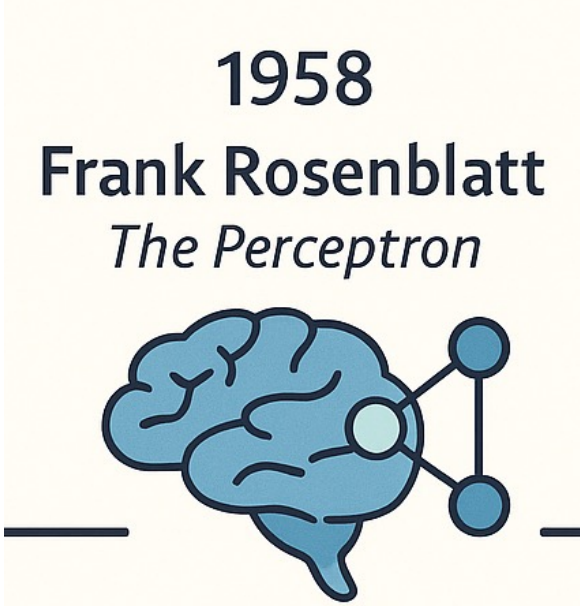
Premières représentations vectorielles du langage : symboliques et statistiques



One Hot encoding

Première approche de représentation vectorielle

- Idée de représentation des entrées dans un réseau de neurone artificiel par des vecteurs binaires
- 1960-1970 - Adoption de cette approche en NLP comme codage standard des mots
  - Un mot= un vecteur unique



Rosenblatt, F. (1958). *The Perceptron: A probabilistic model for information storage and organization in the brain*. *Psychological Review, 65(6)*, 386–408
https://homepages.math.uic.edu/~lreyzin/papers/rosenblatt58.pdf

Vecteurs One-Hot

- Chaque symbole est représenté par un vecteur binaire de la taille du vocabulaire
- Représentation symbolique explicite et interprétable :
  - chaque dimension = un mot
- Vecteur creux (*sparse*), surtout si le vocabulaire est grand

Exemple : vocabulaire = {chat,chien,maison,arbre}

- chat = (1,0,0,0)
- chien = (0,1,0,0)
- maison = (0,0,1,0)
- arbre = (0,0,0,1)

👎 Espace énorme (taille du vocabulaire !!)

Vecteurs One-Hot : comparaisons ? calculs de similarité ?

- Calculs de similarité cosinus

$$\text{sim}(v, w) = \frac{v \cdot w}{\| v \| \quad \| w \|}$$

m1 («chat») = (1, 0, 0, 0)
m2 («chien») = (0, 1, 0, 0)
m3 («maison») = (0, 0, 1, 0)
m4 («arbre») = (0, 0, 0, 1)

- Similarité m1(chat) – m2(chien)

m1 x m2=1x0+0x1+0x0+0x0=0

$\| m1 \| = \| m2 \| = \sqrt{1^2 + 0 \ \ldots + 0} = 1$

sim(m1,m2) =0 *Aucune similarité*

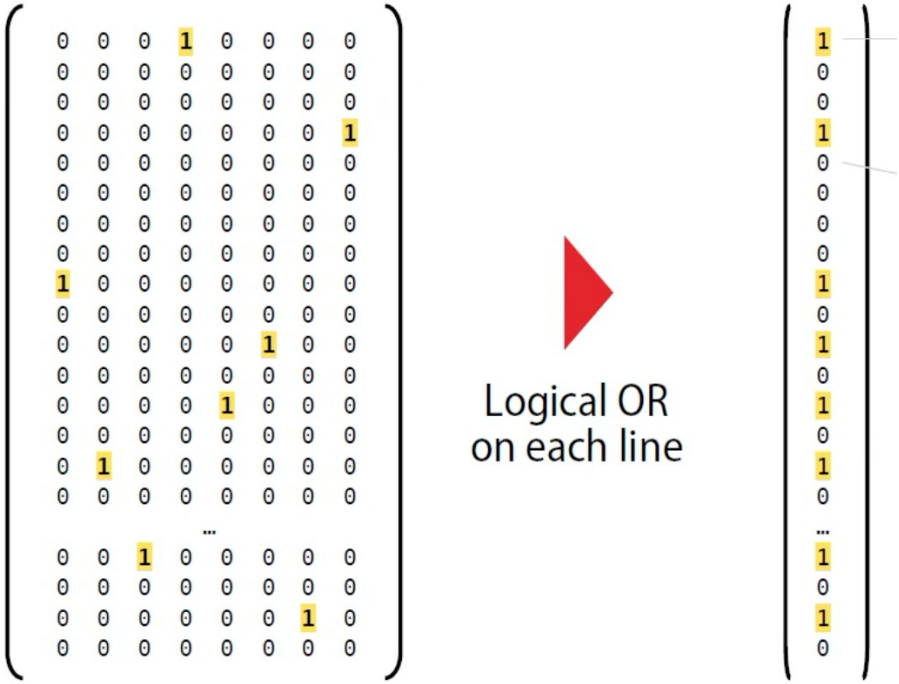
- Autres similarités

sim(m1,m3) …. =0

👎 Aucune notion de sémantique
Aucune similarité identifiée:
"chat" est aussi éloigné de "chien" que de "maison"

Amélioration : One Hot Document Vector

- Un vecteur par document avec 1 pour chaque mot du vocabulaire apparaissant dans le document
- Avantages :
  - Diminution de la taille totale : un vecteur
  - Perte de l'ordre et de la fréquence des mots dans le document



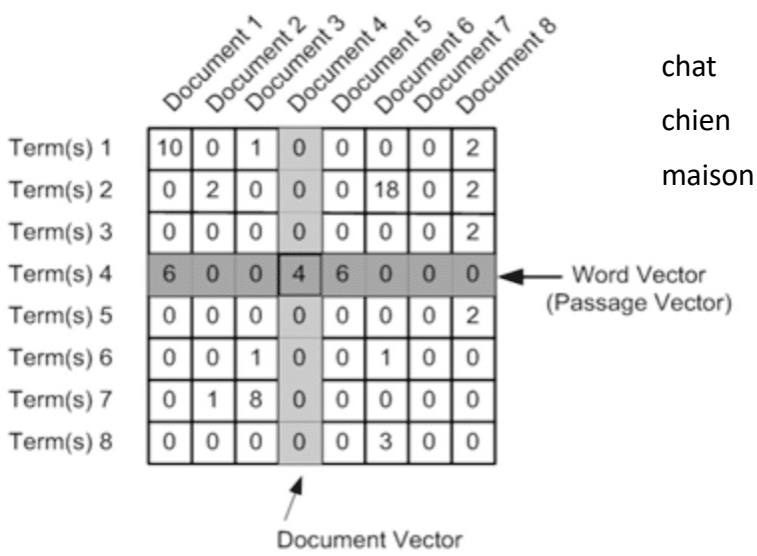
Chaque vecteur a la taille du dictionnaire

source: video CNRS formation FIDLE
https://www.youtube.com/watch?v=iUVBpTeemjU&list=PLll0-qAzf2SZHbTDoU4kFU91wqqe8N-Ug&index=2

Bag of Words (BoW) : Term-Frequency TF

- Représenter un document comme un sac (*bag*) de mots
- Vecteur de dimension V (taille du vocabulaire)
- Chaque case indique la le nombre d'occurrences d'un mot dans le document
- L'ordre des mots est perdu → représentation "non structurée"

- Vocabulaire = {chat, chien, maison}
  - Doc 1 : "le chat dort" → [1, 0, 0]
  - Doc 2 : "un chien chien dans la maison" → [0, 2, 1]



	Doc 1	Doc 2
chat	1	0
chien	0	2
maison	0	1

source : https://datascientest.com/introduction-au-nlp-natural-language-processing

One-hot - Bag of Words (BoW) (Term frequency (TF))

👍Avantages

- simplicité
- *base pour des méthodes statistiques (Naive Bayes, SVM)*

👎 Limites

- Trés grande dimension des vecteurs
- Vecteurs très creux
- Pas de prise en compte de la sémantique
- Pas de prise en compte de l'ordre des mots

Vecteurs pondérés

TF-IDF : Term Frequency – Inverse Document Frequency

Karen Spärck Jones propose l'IDF

1972

Sparck Jones, K. (1972). *A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1).*

Salton, G., & Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5)*

Salton & Buckley popularisent **TF × IDF** comme standard pour la recherche d'information

1988

1975

Gerard Salton formalise le modèle d'espace vectoriel

G. Salton , A. Wong , C. S. Yang, A vector space model for automatic indexing, Communications of the ACM, v.18 n.11, p. 613-620, novembre 1975

TF-IDF dans *Introduction à la recherche d'information*

2008

Manning, C. D., Raghavan, P., & Schütze, H. (2009). *Introduction to Information Retrieval*. Cambridge University Press

2000s

TF-IDF devient l'outil de référence dans la recherche documentaire et le TAL statistique

Vecteurs pondérés (TF-IDF) de documents

N = nb de documents

📦 Étape 1 — Calcul de TF(t, d) (term frequency)

- **TF(t, d)** (*term frequency*)
  = fréquence du terme *t* dans un document *d*

  $= \frac{\text{nombre d'occurrences de t dans d}}{\text{nombre total de mots dans d}}$

🌍 Étape 2 — Calcul de IDF(t) (inverse document frequency)

- **DF(t)** *document frequency* = nombre de documents contenant le terme t (mot comptés une seule fois)
- **IDF(t)** (*inverse document frequency*)

  $= ln\left(\frac{N}{df(t)}\right)$

  ln= logarithme népérien (base e)

⚖️ Étape 3 — Pondération finale :

$$tfidf(\mathrm{t},\mathrm{d}) = tf(\mathrm{t},\mathrm{d}) \times idf(t)$$

🧩 Etape 4 - Vecteur TF-IDF du document

Construction d'un vecteur TF-IDF par document

Pour un vocabulaire $V = \{t_1, t_2, .., .. t_n\}$,
le document *d* est représenté par le vecteur :

d= [tfidf(t1, d1),tfidf(t2,d1), … tfidf(tn,d1]

Ajout d'une pondération selon l'importance des mots dans chaque document

Vecteurs pondérés (TF-IDF) de documents
Exemple

Exemple : Corpus de 3 phrases

d1 : « chat dort »

d2 : « chien court »

d3 : « maison dort »

Vocabulaire : {chat, chien, maison, dort, court}

$N = 3$

🧱 Étape 1 — TF(t, d) - term frequency

$$= \frac{\text{nombre d'occurrences de t dans d}}{\text{nombre total de mots dans d}}$$

Mot	$tf(t,d_1)$	$tf(t,d_2)$	$tf(t,d_3)$
chat	0.5 = 1/2	0	0
chien	0	0.5	0
maison	0	0	0.5
dort	0.5=1/2	0	0.5
court	0	0.5	0

🌍 Étape 2 — Calcul de IDF(t) - inverse document frequency

df(t) = nombre de documents contenant t

IDF(t) $= ln\left(\frac{N}{df(t)}\right)$

Mot	df(t)	IDF(t)
chat	1	ln(3/1) = **1.10**
chien	1	**1.10**
maison	1	**1.10**
dort	2	ln(3/2) = **0.41**
court	1	**1.10**

- *Les mots rares (chat chien, maison, court) ont un idf elévé 1.10 : ils caractérisent fortement leur document*
- *dort est plus fréquent (idf 0,41): il est moins discriminant*

Vecteurs pondérés (TF-IDF) de documents
Exemple

⚖️ Étape 3 — Pondération finale

$$TFIDF(t,d) = TF(t,d) \times IDF(t)$$

Mot	$tfidf(t,d_1)$	$tfidf(t,d_2)$	$tfidf(t,d_3)$
chat	0.5×1.10 = **0.55**	0	0
chien	0	0.5×1.10 = **0.55**	0
maison	0	0	0.5×1.10 = **0.55**
dort	0.5×0.41 = **0.205**	0	0.5×0.41 = **0.205**
court	0	0.5×1.10 = **0.55**	0

🧩 Etape 4 - Vecteur TF-IDF du document

Document	Vecteur TF-IDF = (chat, chien, maison, dort, court)
d_1	(0.55, 0, 0, **0.205**, 0)
d_2	(0, 0.55, 0, 0, 0.55)
d_3	(0, 0, 0.55, **0.205**, 0)

Vecteurs pondérés (TF-IDF)

👍Avantage
- Calculs de similarité plus significatifs
  Ebauche de sémantique

- Calculs de similarité cosinus entre documents

$$\text{sim}(v, w) = \frac{v \cdot w}{\| v \| \quad \| w \|}$$

d1 (« chat dort ») = (0.55, 0, 0, **0.205**, 0)
d2 (« chien court ») = (0, 0.55, 0, 0, 0.55)
d3 (« maison dort ») = (0, 0, 0.55, **0.205**, 0)

1. Normes des vecteurs
   - $\| d1 \| = \sqrt{0.55^2 + 0.205^2} \approx 0.587$
   - $\| d2 \| = \sqrt{0.55^2 + 0.55^2} \approx 0{,}778$
   - $\| d3 \| = \sqrt{0.55^2 + 0.205^2} \approx 0.587$
2. Similarité d1 – d2 — *Aucune similarité*

   d1.d2 =sim(d1,d2) = 0
3. Similarité d1 – d3 — *Partage du mot "dort"*

   d1.d3 =0.205×0.205=0.0420

   sim(d1,d3) = d1.d3 / $(0.587 \text{x} 0.587) \approx 0.344$
4. Similarité d2 – d3 — *Aucune similarité*

   d2.d3 =sim(d2,d3) = 0

👎 Limites
- Taille toujours énorme
- Vecteurs creux
- Pas de synonymie
- *Pas de contexte*

A vous de jouer : Exercice (1/3)

Corpus (Nombre de documents : N = 3)

d1 : « soleil brille fort »

d2 : « lune brille doucement »

d3 : « maison ancienne belle »

Vocabulaire et ordre figé (dimension = 8) :
$soleil, lune, brille, fort, doucement, maison, ancienne, belle$

Q1 — Représentation one-hot

- Donnez les vecteurs one-hot des mots « soleil », « lune », « maison » dans l'ordre du vocabulaire.
- Calculez la similarité cosinus entre les paires (soleil, lune), (soleil, maison) et (lune, maison) (*expliquez vos calculs*).
- Pourquoi cette mesure n'apporte-t-elle aucune information sémantique avec le one-hot ?

A vous de jouer : Exercice (2/3)

Q2 — Représentation TF-IDF (base e)

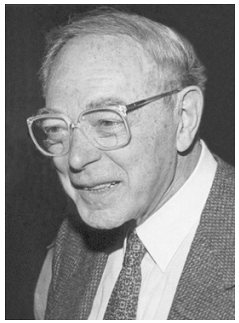
- Etapes 1 et 2 - Calculez TF(t) , DF(t) et IDF(t) pour chaque terme du vocabulaire
- Etapes 3 et 4 - Construisez les vecteurs TF-IDF complets (dimension 8) de d1, d2 et d3 (dans l'ordre du vocabulaire)

Q3 — Similarité cosinus

- Calculez sim(d1,d2), sim(d1,d3) et sim(d2,d3)
- Interprétez : quelle paire est la plus proche ? Est-ce cohérent avec l'intuition ?

Matrices de co-occurrence : La première mise en pratique de l'hypothèse distributionnelle

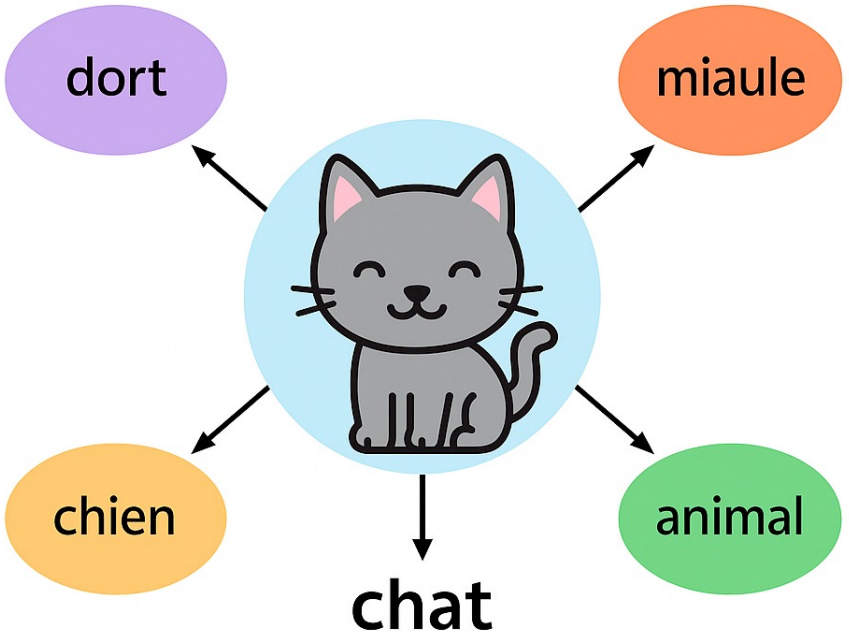
- Le sens d'un mot est défini par les contextes dans lesquels il apparaît



- *"The meaning of entities is related to the distributions of the forms in which they appear."*

Harris, Z. (1954). *Distributional structure*. *Word, 10(2/3)*, 146–162.

https://www.tandfonline.com/doi/pdf/10.1080/00437956.1954.11659520



- Utilisées dès les **années 1960–70** pour analyser la proximité lexicale

Hypothèse fondamentale utilisée par les LLM

Qu'est-ce qu'une matrice de co-occurrence ?

- Chaque mot est représenté par **son vecteur de co-occurrence** (la ligne correspondante)
  - Deux mots apparaissant dans des contextes similaires → vecteurs proches
- Colonnes = mots (ou contextes)
- Cellule $M_{i,j}$= nombre de fois où le mot i apparaît dans le contexte du mot j
- Le contexte peut être :
  - une fenêtre de quelques mots autour,
  - ou bien la phrase/document

mot\ mot	mot 1	mot 2	…	mot n
mot 1	0	2	0	1
mot 2	2	0	0	1
…	0	0	0	0
mot n	1	1	0	0

le mot 1 apparait 2 fois dans le contexte du mot 2 et 1 fois dans le contexte du mot n

👎 Limites
- Taille toujours énorme
- Vecteurs creux

👍 Avantage
- Calculs de similarité plus significatifs

Exemple de matrice de co-occurrence

Corpus

- d1 : « Le chat mange des croquettes »
- d2 : « Le chien dévore des os»
- d3 : « Un chat est un animal domestique»
- d4 : « Un chien est un animal domestique»

- Fenêtre choisie: un mot à gauche et à droite
- Un mot = un vecteur (**une ligne de la matrice)**

mot\ mot	**chat**	**chien**	**mange**	**dévore**	**animal**	**domestique**	**croquettes**	**os**
chat	0	0	1	0	1	0	0	0
chien	0	0	0	1	1	0	0	0
mange	1	0	0	0	0	0	1	0
dévore	0	1	0	0	0	0	0	1
animal	1	1	0	0	0	**2**	0	0
domestique	0	0	0	0	**2**	0	0	0
croquettes	0	0	1	0	0	0	0	0
os	0	0	0	1	0	0	0	0

Exemple de matrice de co-occurrence

Calculs des similarités cosinus

- Similarité chat – chien
  - Produit scalaire *chat.chien*=1
  - Normes ||chat||= $\sqrt{2}$ ||chien||=$\sqrt{2}$

    sim(chat,chien)=$\frac{1}{2}$ =0,5

"chat" et "chien" apparaissent comme sémantiquement proches grace au partage du contexte

TF-IDF n'aurait trouvé aucune similarité: Pourquoi?

mot\ mot	chat	chien	mange	dévore	animal	domestique	croquettes	os
chat	0	0	1	0	1	0	0	0
chien	0	0	0	1	1	0	0	0

A vous de jouer : Exercice (3/3)

Corpus (Nombre de documents : N = 3)

d1 : « soleil brille fort »

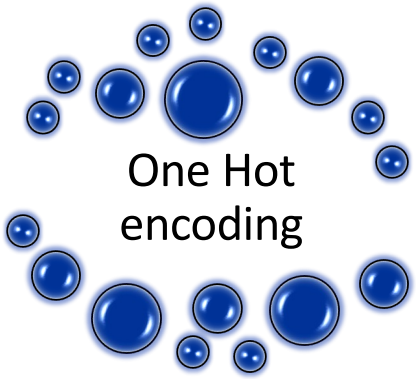
d2 : « lune brille doucement »

d3 : « maison ancienne belle »

Q4 — Construire la matrice de co-occurrence des mots du vocabulaire ‘$soleil, lune, brille, fort, doucement, maison, ancienne, belle$‘ en considérant que deux mots co-apparaissent s’ils sont dans le même document (fenêtre choisie : 2 mots à gauche et à droite)

- Calculer la similarité cosinus entre les vecteurs de soleil et lune. Que montre ce calcul par rapport à TF-IDF ?

Des représentations explicites aux premières représentations distributionnelles



✅ Simple, fidèle au symbolique
❌ Vecteurs très grands et creux
❌ Aucune similarité sémantique
🔍 **Explicite** : chaque dimension = un mot

TF-IDF

✅ Pondération plus fine (mots rares valorisés)
✅ Permet de comparer documents par similarité cosinus
❌ Toujours des vecteurs creux
❌ Pas de synonymie, pas de contexte
🔍 **Explicite** : pondérations interprétables

Matrices de co-occurrence

✅ Capturent les relations entre **mots** (et non seulement entre documents)
✅ Première implémentation de l'**hypothèse distributionnelle**
❌ Toujours creuses et de très grande dimension
🔍 **Explicite** : interprétation possible des co-occurrences

Quand les vecteurs simples ne suffisent plus …

Limites des représentations simples

❌ Vecteurs énormes (taille = vocabulaire) → inefficaces

❌ Pas de vraie notion de proximité sémantique

❌ Difficile de comparer différents types de données

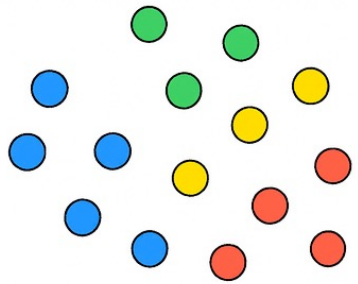
0	0	1	0	0

💡 **Besoins des systèmes modernes (LLM, IA multimodale)**

✅ Représentations plus compactes

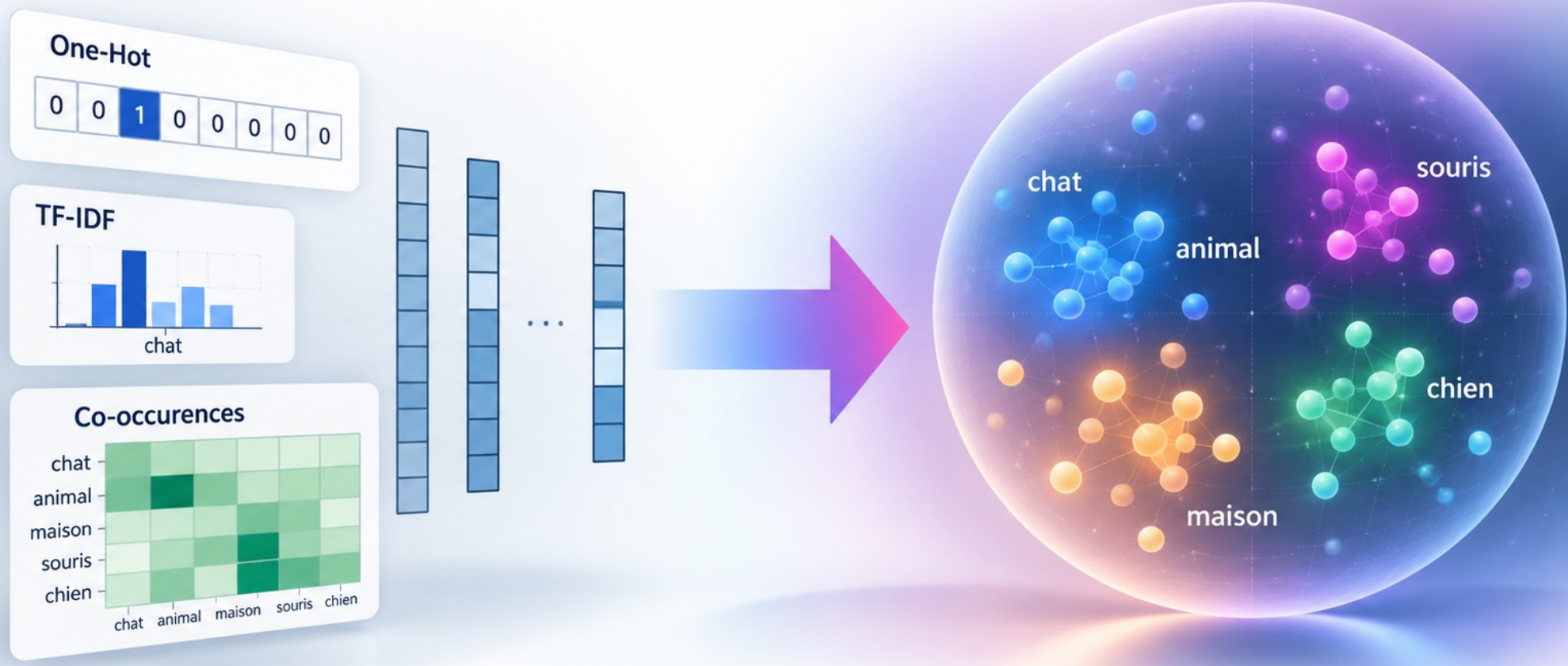
✅ Représentations qui capturent la similarité de sens

✅ Un principe général et universel (texte, image, son)



3 – Vers les représentations vectorielles denses apprises : la notion d'embedding

De la représentation construite à la représentation apprise

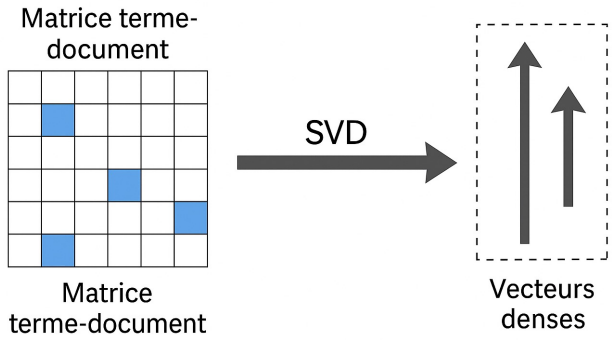


Premières étapes vers la densité : LSA

Latent Semantic Analysis (LSA) (Deerwester et al., 1990)

Deerwester, S. et al. (1990). *Indexing by Latent Semantic Analysis*. *Journal of the American Society for Information Science, 41(6)*.
http://wordvec.colorado.edu/papers/Deerwester_1990.pdf

- Mise en évidence de relations statistiques latentes entre les termes et les documents
- Des mots qui n'apparaissent pas ensemble peuvent tout de même être rapprochés via les dimensions latentes



Mot	Dim1	Dim2
chat	0.9	0.1
chien	0.8	−0.2
noir	0.6	0.5
blanc	−0.5	0.6

Dimensions "latentes" = dimensions découvertes automatiquement par la décomposition SVD

similarite_cosinus(chat,chien)≈0.94

Principe de LSA

1. Construire une matrice terme-document (ou mot-contexte)
   - Chaque cellule contient une pondération (souvent TF-IDF)
   - Cette matrice est immense et creuse

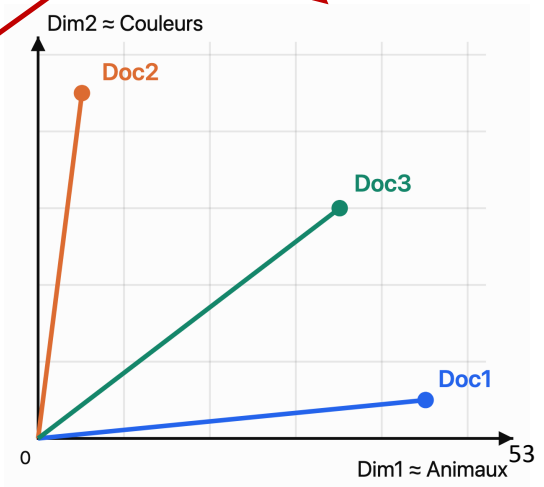
Doc1 = “chat noir”
Doc2 = “chat blanc”
Doc3 = “chien noir”

Terme	Doc1	Doc2	Doc3
chat	1	1	0
chien	0	0	1
noir	1	0	1
blanc	0	1	0

2. Appliquer SVD Décomposition en Valeurs Singulières pour réduire la dimensionnalité et créer des vecteurs denses

dimensions latentes

Mot	Dim1	Dim2
chat	0.9	0.1
chien	0.8	−0.2
noir	0.6	0.5
blanc	−0.5	0.6



Interprétations illustratives Coordonnées fictives à visée pédagogique — non issues du calcul de la SVD

La décomposition en valeurs singulières (SVD)

Principe et application à une matrice terme-document

1 Une matrice terme-document

On construit une matrice A qui représente les documents par leurs termes (exemple simplifié).

	Doc1 chat noir	Doc2 voiture rouge	Doc3 chien moteur
chat	2	0	0
chien	0	0	2
voiture	0	2	0
moteur	0	0	2
noir	1	0	0
rouge	0	1	0

Chaque cellule contient une pondération (ex. fréquence ou TF-IDF) indiquant l'importance d'un terme dans un document.

2 La décomposition SVD

La matrice A peut se décomposer en trois matrices :

$$A = U \Sigma V^{T}$$

U
Axes dans l'espace des lignes (termes)

$$\begin{bmatrix} -0.71 & -0.71 & \cdots \\ 0.71 & -0.71 & \cdots \\ 0.00 & 0.00 & \cdots \\ 0.00 & 0.00 & \cdots \\ \cdots & \cdots & \cdots \end{bmatrix}$$

Σ
Valeurs singulières (importances des axes)

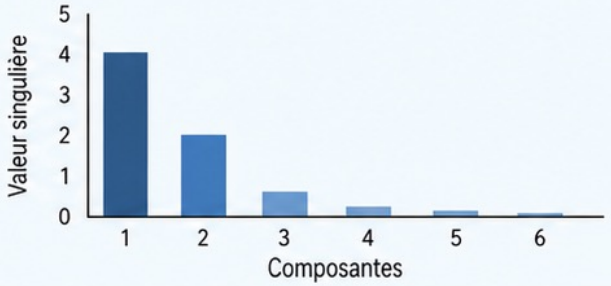
$$\begin{bmatrix} 4.2 & 0 & 0 & \cdots \\ 0 & 2.1 & 0 & \cdots \\ 0 & 0 & 0.5 & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{bmatrix}$$

V^{T}
Axes dans l'espace des colonnes (documents)

$$\begin{bmatrix} -0.68 & -0.73 & \cdots \\ 0.73 & -0.68 & \cdots \\ \vdots & \vdots & \cdots \end{bmatrix}$$

3 Les valeurs singulières

Elles sont classées de la plus grande à la plus petite et indiquent l'importance de chaque dimension latente.



4 Réduction de dimension

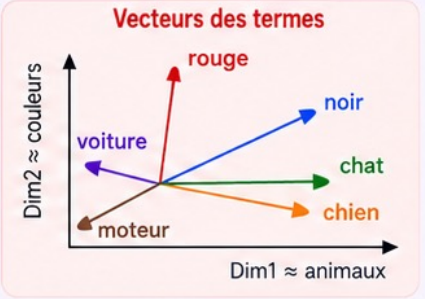
On ne conserve que les k premières composantes (les plus importantes) :

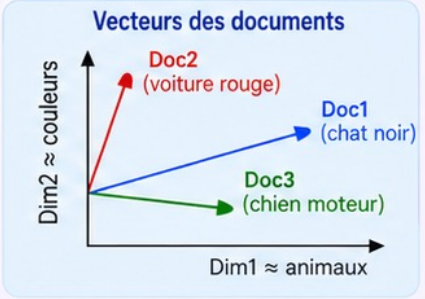
$$A_k = U_k \Sigma_k V_k^{T}$$

On obtient une approximation de A avec moins de dimensions, qui minimise l'erreur de reconstruction (au sens de la norme de Frobenius) parmi toutes les matrices de rang au plus k.

5 Application : représentations latentes

Les lignes de $U_k \Sigma_k$ donnent les coordonnées des termes et les colonnes de $V_k \Sigma_k$ donnent les coordonnées des documents dans l'espace latent (ici $k = 2$).





Limites de LSA

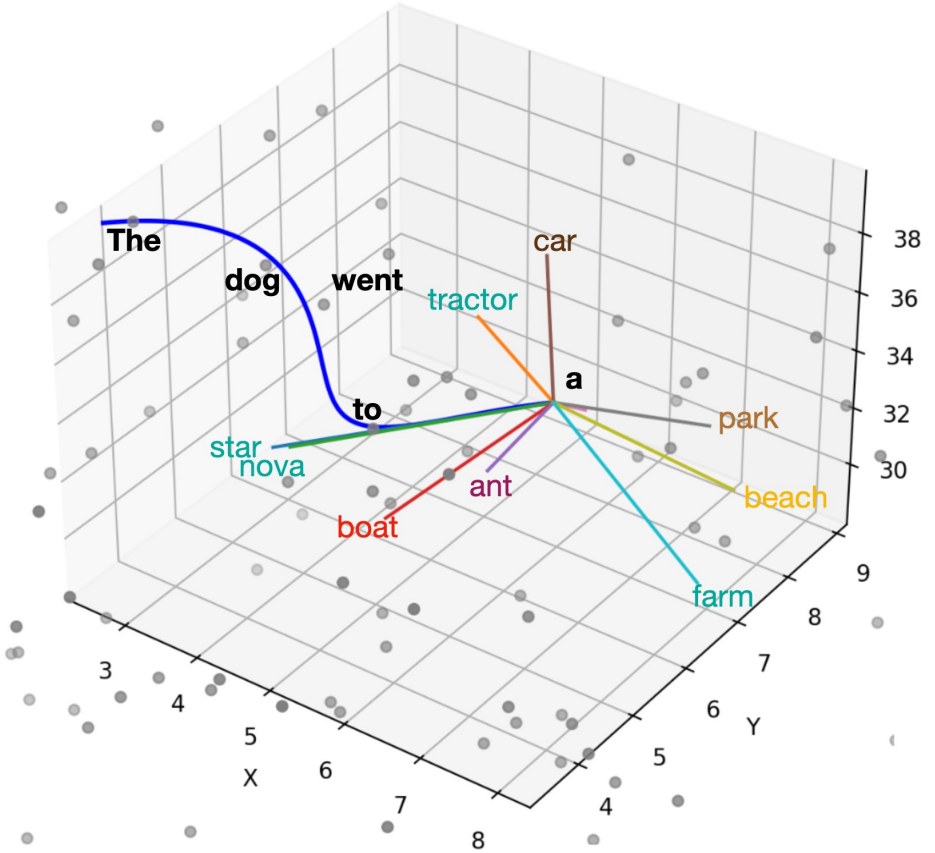
- La réduction de dimension est globale et figée → pas de mise à jour facile avec de nouvelles données
- Pas de prise en compte fine du contexte :
  - un mot = toujours le même vecteur
- Les relations captées sont assez grossières, car purement statistiques
- Lourdeur calculatoire : les matrices sont immenses et coûteuses à décomposer

LSA a ouvert la voie, les approches neuronales vont plus loin

Qu'est-ce qu'un embedding ?

- Un vecteur avec des centaines voire des **milliers de dimensions** :
  - une dimension représente différents aspects sémantiques ou caractéristiques de l'entité représentée (mot, pixel, film, playlist…).
- Les dimensions n'ont **pas une interprétation humaine** directe

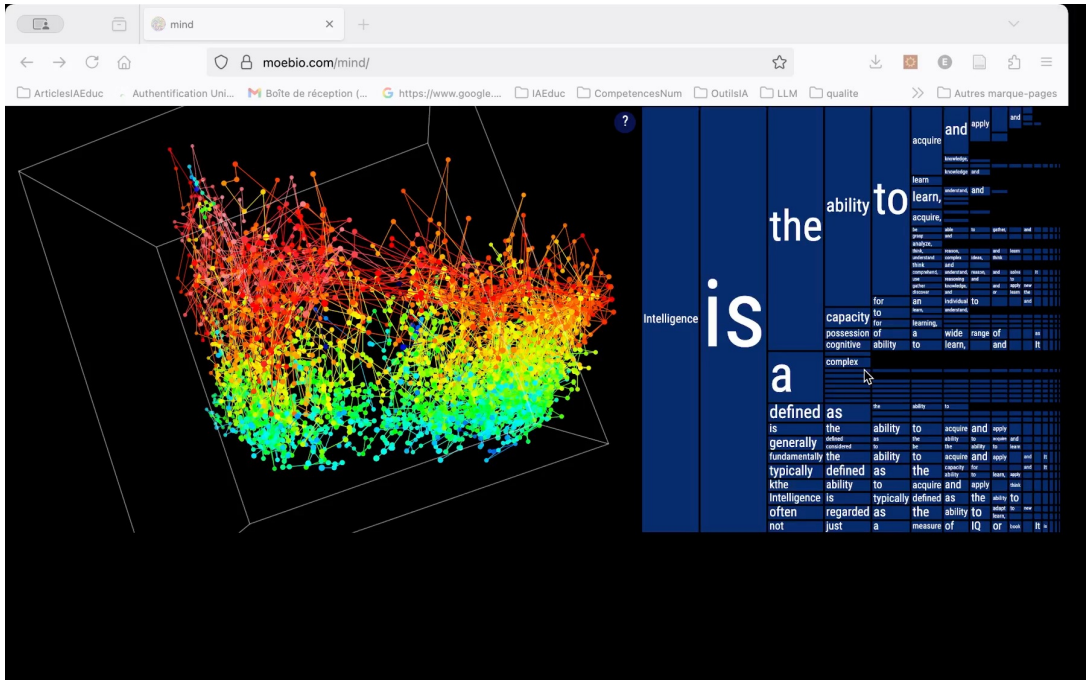
le nombre de dimensions d est un paramètre du modèle



source : https://ironj.github.io/llmviz/

Un vecteur "appris" plongé dans un espace "latent"

"latent" : vient du latin latens, participe présent de latēre → « être caché, dissimulé »

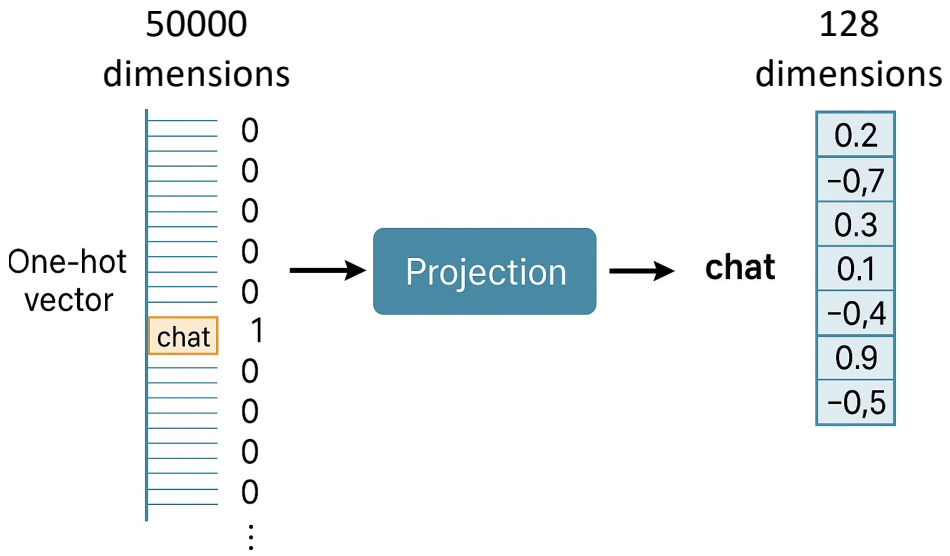


Une visualisation dynamique de cet espace
https://moebio.com/mind/

- L'espace n'est pas défini par des catégories explicites mais par une structure cachée apprise automatiquement
- Les vecteurs sont définis par entrainement à partir de données
  - relations sémantiques ou fonctionnelles dans les données apprises automatiquement

Définition d'un embedding

- Vecteur
  - dense
  - construit par apprentissage :
    encode des proximités sémantiques
    automatiquement quantifiables
  - continu : valeurs réelles



Un embedding est une fonction $f: V \rightarrow \mathbb{R}^d$ apprise à partir des données

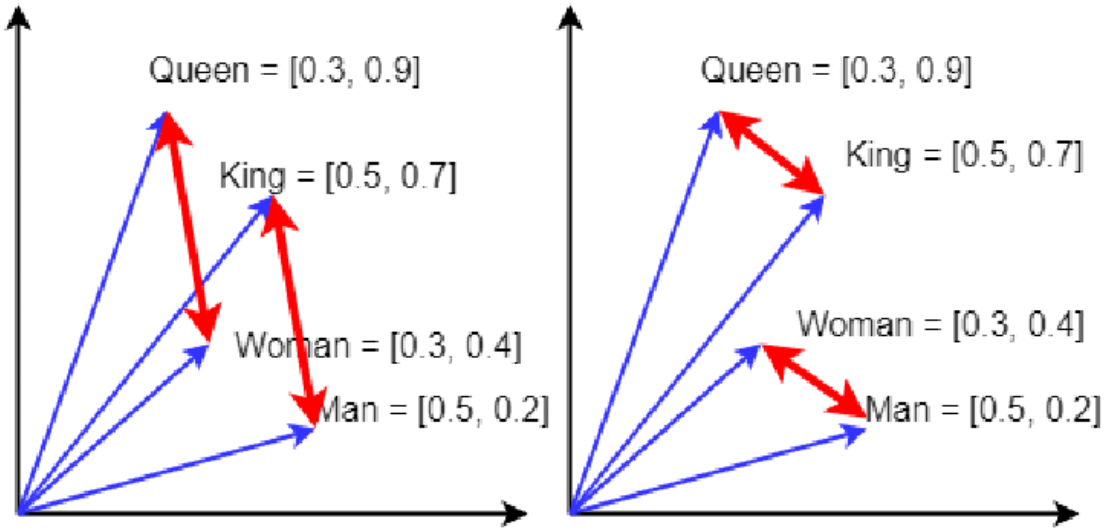
V = **vocabulaire** ou ensemble d'objets à représenter (taille très grande)

d = **dimension de l'espace projeté** : **choisie par le concepteur (hyperparamètre)** (ex. 100–1024)

Embeddings et sémantique

- La proximité géométrique traduit une proximité sémantique.
- Conséquences
  - Les relations de sens deviennent mesurables (distances, angles) :
    - Calculs de similarité significatifs
  - Les opérations vectorielles ont des significations :
    - $King - Man + Woman \approx Queen$

[0.5, 0.7] – [0.5,0.2]+[0.3,0.4] = [0.3, 0,9] = Queen



source: https://www.researchgate.net/figure/The-classical-king-woman-man-queen-example-of-neural-word-embeddings-in-2D-It_fig1_332679657

Sur quoi est basée la construction des embeddings ?

Toujours l'hypothèse fondamentale de la sémantique distributionnelle

- "***You shall know a word by the company it keeps***"
  (*Firth, J. R. 1957. Papers in Linguistics 1934–1951, Oxford University Press.*)

Discutable ? Qu'en pensez vous ?

Critiques de l'hypothèse distributionnelle

- Fréquence = vérité ??
- Représentation de la compréhension humaine ?
  - L'apprentissage humain n'est pas uniquement statistique : perception, action, émotion et capacités innées
  - Importance de l'interaction, de l'attention conjointe, de l'intersubjectivité entre humains

Emily BENDER

– Bender, E. M., & Koller, A. (2020). *Climbing towards NLU: On meaning, form, and understanding in the age of data*. ACL. https://aclanthology.org/2020.acl-main.463/

L'expérience de la pieuvre

Une expérience de pensée pour interroger la compréhension du langage

Inspirée de Bender & Koller (2020), cette expérience imagine une pieuvre très intelligente qui apprend à répondre comme un humain en observant des échanges textuels, sans jamais avoir accès au monde réel.

1 Deux humains discutent du monde réel

Ils échangent par messages écrits à distance sur des objets, des événements, des expériences vécues.

2 Une pieuvre intercepte les échanges

Une pieuvre, placée entre les deux humains, observe tous les messages échangés.
Elle n'a aucun accès au monde réel : ni vue, ni expérience physique, ni interaction directe.

La pieuvre apprend des régularités dans les échanges textuels.

3 La pieuvre apprend à répondre

Après avoir observé de nombreux échanges, la pieuvre est capable de produire des réponses plausibles et cohérentes, en imitant le style et le contenu des humains.

Les réponses de la pieuvre sont souvent pertinentes et indiscernables de celles d'un humain.

4 Mais comprend-elle vraiment ?

La pieuvre produit des réponses adaptées sans avoir d'expérience du monde.
Elle apprend à associer des mots et des contextes, sans savoir ce qu'est réellement une plage, un film ou la crème solaire.

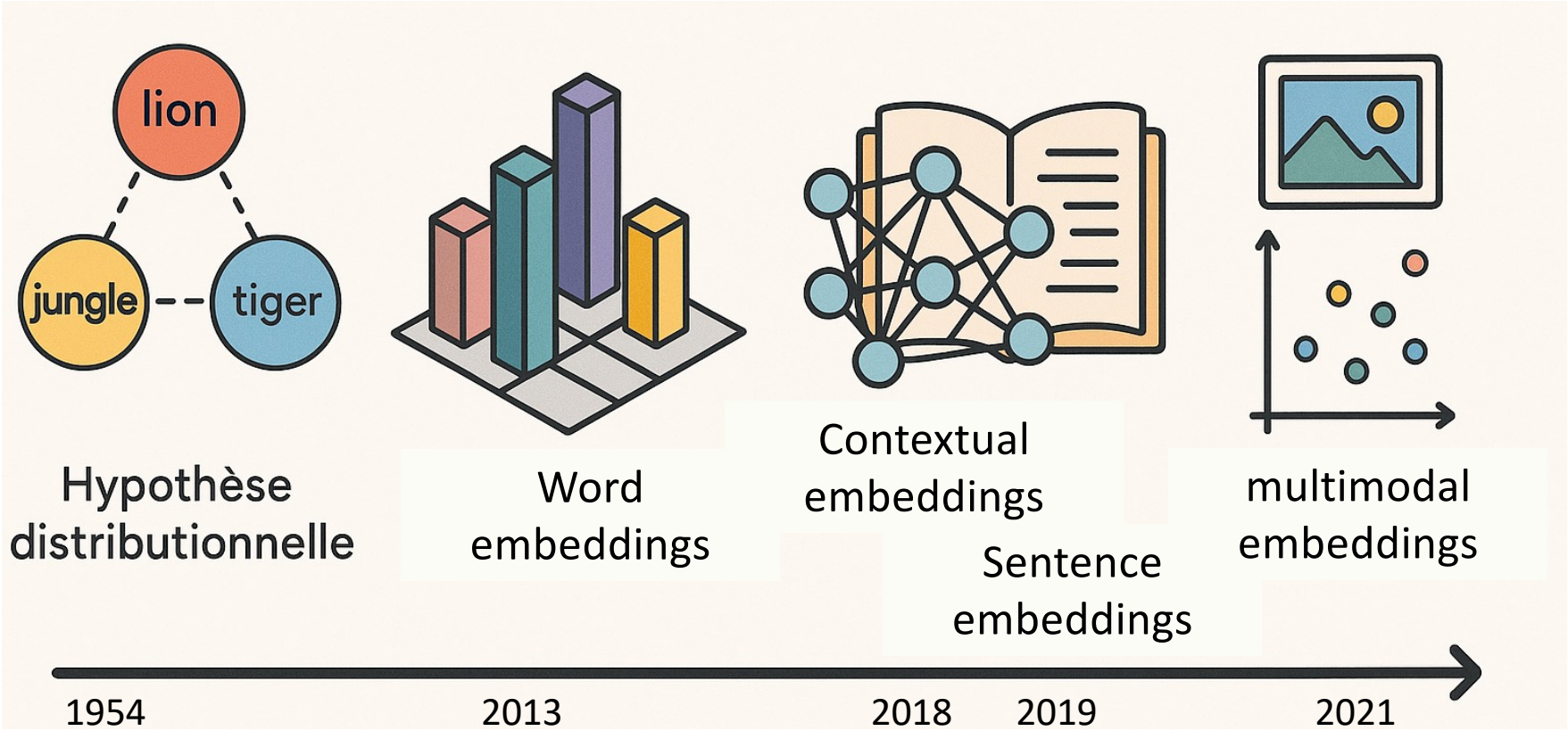
Ce que montre cette expérience

- Un système peut produire des réponses cohérentes sans compréhension réelle du monde.
- Il apprend des régularités statistiques dans les échanges textuels.
- La cohérence linguistique ne garantit pas une compréhension sémantique.
- Cela illustre une limite des modèles de langage actuels : ils manipulent des formes, sans expérience incarnée ni ancrage dans la monde réel.
- La compréhension du langage implique probablement plus qu'une simple prédiction de mots : elle repose sur l'interaction, l'expérience et la situation.

Lien avec les modèles de langage actuels

Comme la pieuvre, les grands modèles de langage (LLM) produisent des réponses pertinentes en analysant des grandes quantités de textes, sans expérience directe du monde. Ils excellent dans la prédiction et la génération de texte, mais leur compréhension reste une question ouverte.

Historique des embeddings



Embeddings : ce qui change selon les modèles

Modèle	Niveau de représentation	Exemple corpus jouet *« La batterie du téléphone est vide. »* *« Le musicien joue de la batterie. »*	Ce que ça change
Word2Vec / GloVe (statiques)	**Un seul vecteur par mot**. Word Embedding	batterie → [0.21, -0.55, 0.89]	❌ Ne distingue pas énergie vs instrument
ELMo (LSTM, semi-contextuel)	Vecteur différent selon le **contexte local** Contextual Embedding	batterie (téléphone) → [0.11, -0.09, 0.65] batterie (musique) → [-0.73, 0.52, -0.12]	✅ Sens différencié grâce au voisinage immédiat
BERT / GPT (transformers)	Vecteur contextualisé par **toute la phrase**	batterie (téléphone) → [0.02, -0.44, 0.93, …] batterie (musique) → [-0.81, 0.18, -0.27, …]	✅ Distinction plus fine et robuste
Sentence-BERT	Un vecteur unique **par phrase entière** Sentence Embedding	« La batterie du téléphone est vide. » → [0.31, -0.22, 0.77, …] « Le musicien joue de la batterie. » → [-0.84, 0.15, -0.41, …]	✅ Comparaison directe de phrases, ❌ perte de nuance intra-phrase

Paramètres des modèles à embeddings

📚 Corpus d'entraînement

- Taille
- Domaine
- Qualité des données

🧠 Modèle choisi

- Statique (Word2Vec, GloVe)
- Contextuel (BERT, GPT)
- Sentence
- Multimodal (CLIP…)

⚙️ Paramètres de conception

- Dimension de l'espace latent (d)
- Taille de la fenêtre de contexte
- Fonction d'objectif (Skip-gram, CBOW, masquage, etc.)

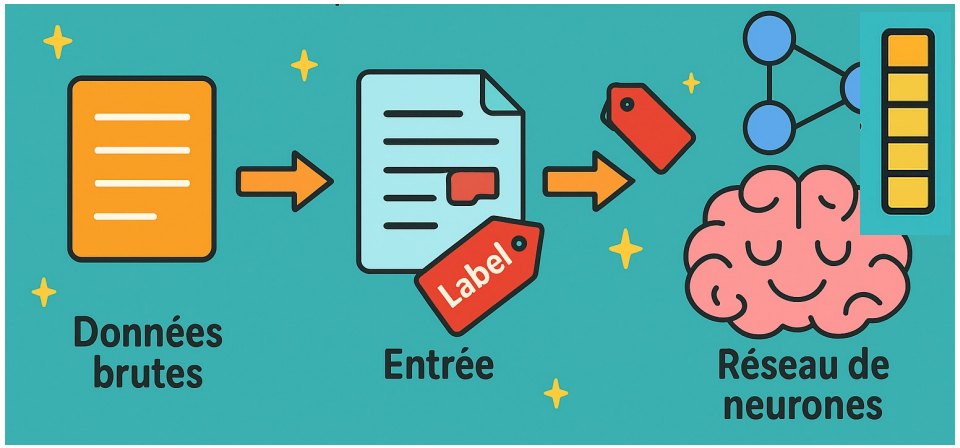
🔧 Paramètres d'entraînement (optimisation)

- Taux d'apprentissage (**learning rate**) → vitesse de mise à jour
- Nombre d'itérations (**epochs**) → combien de fois on passe sur le corpus
- Taille du **batch** → combien d'exemples pour une mise à jour
- Negative sampling → combien d'exemples négatifs générés
- Algorithme d'optimisation → SGD Stochastic Gradient Descent , Adam, …

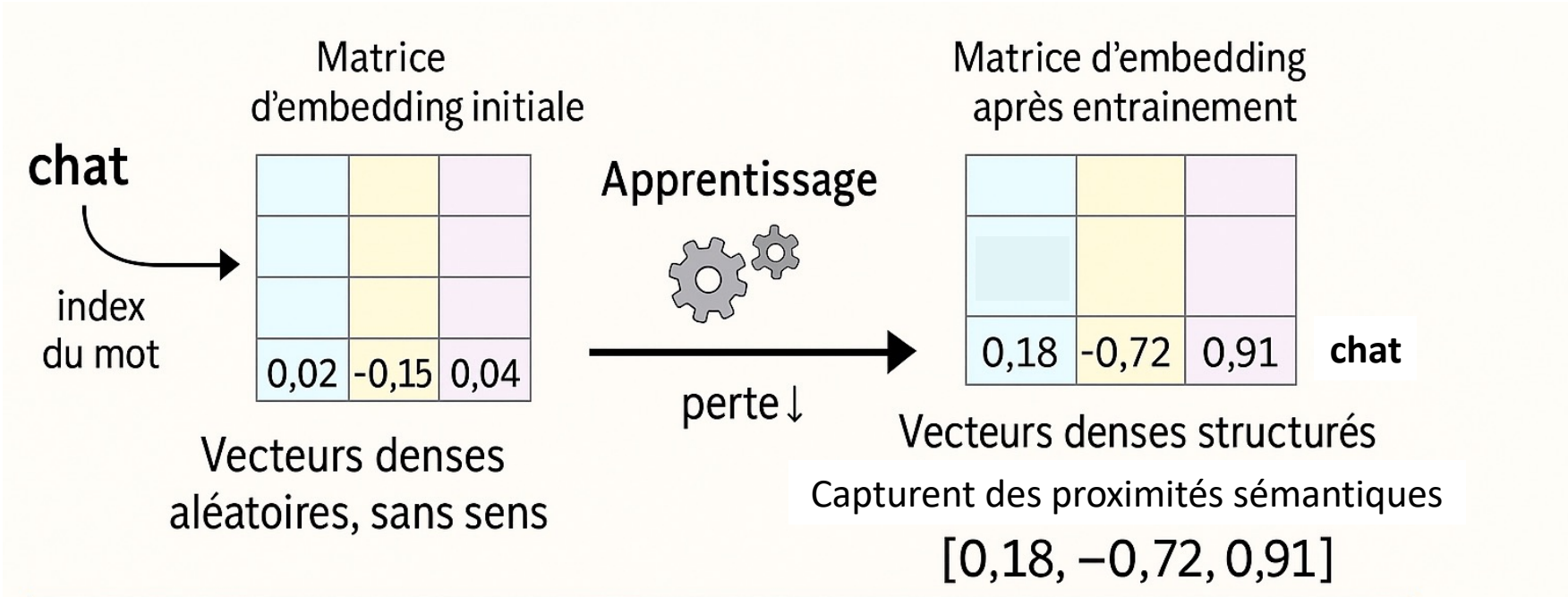
Principe d'apprentissage auto-supervisé

💬 Le modèle fabrique lui-même sa supervision en construisant automatiquement des paires (entrée, label) à partir des données brutes

- Par exemple :
  - prédire un mot manquant,
  - prédire le mot suivant,
  - reconstituer une image dégradée, etc.
- On parle aussi de **prétexte task** (*tâche de pré-entraînement*).



Le principe général d'apprentissage des embeddings : apprendre le sens à partir du hasard



la matrice d'embedding $E \in \mathbb{R}^{|V| \times d}$

= un grand **dictionnaire de vecteurs de mots (ou tokens)** affiné automatiquement par l'apprentissage.

Etapes d'entrainement d'un modèle à embeddings

Chaque mot/objet reçoit un vecteur **aléatoire** (dimension fixée, ex : 100, 300, 768…).

Sélection d'une fenêtre de contexte ou contexte entier

Choix de la tache de pré-entrainement d'apprentissage :

- Prédire le mot cible,
- Prédire le contexte,
- Masquer et prédire

Mise à jour des vecteurs par descente de gradient Ajustement pour réduire l'erreur

Vecteurs finaux capturant la proximité sémantique

Répétition sur le corpus

Entrainement : Phase d'initialisation

Corpus massif

Pas pour les premiers modèles (Word2Vec)

Découpage en tokens

Fragments choisis selon la fréquence statistique

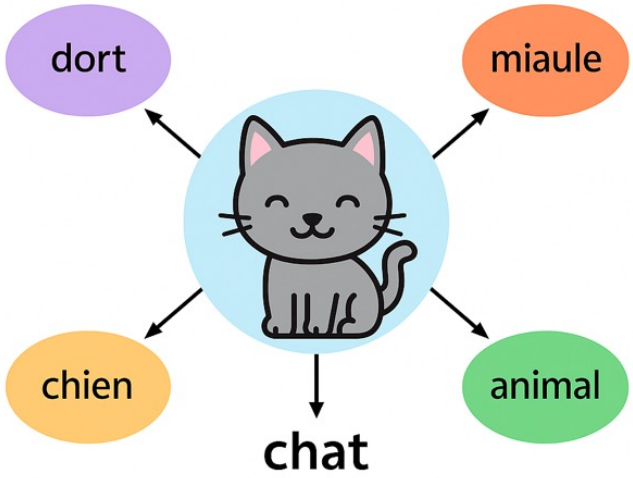
Le chat est incroyable

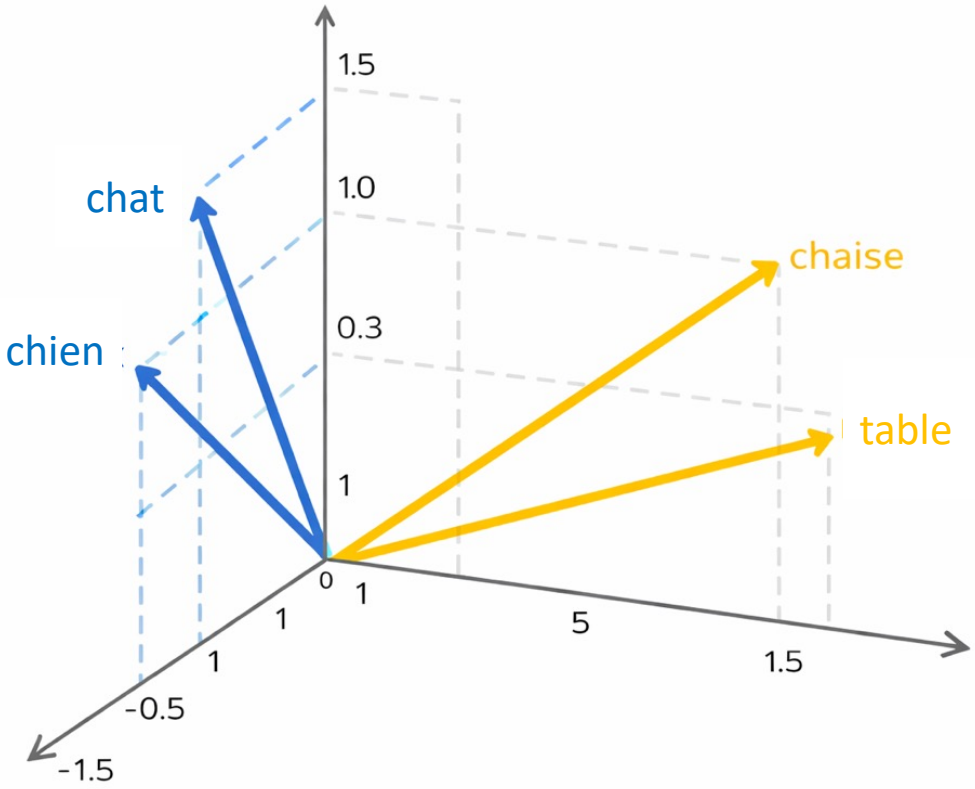
Le	chat	est	inc	roy	able

Vecteurs initialisés aléatoirement

chat	**0.20**	**-0.87**	**1.43**	**2.89**	**…**	**…**	**…**
est	**0.45**	**1.67**	**-0.55**	**1.09**	**…**	**…**	**…**

L’entraînement construit un espace vectoriel latent

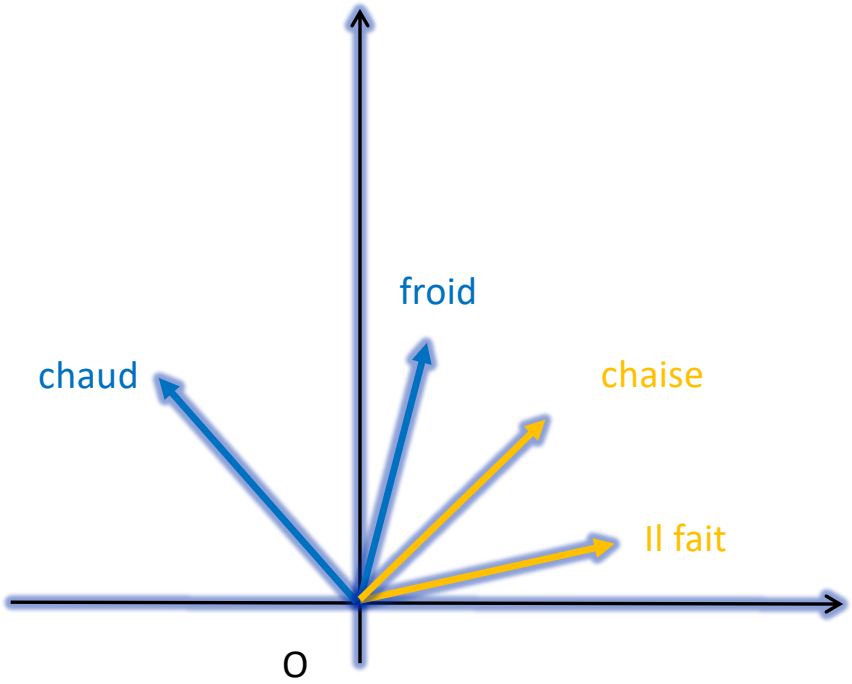




Les mots(tokens) proches en contexte sont proches dans l’espace :
le sens naît des relations

Principe d'apprentissage

Ajustement progressif des vecteurs de mots (ou de tokens)



- Initialisation aléatoire
- Mot masqué → prédiction

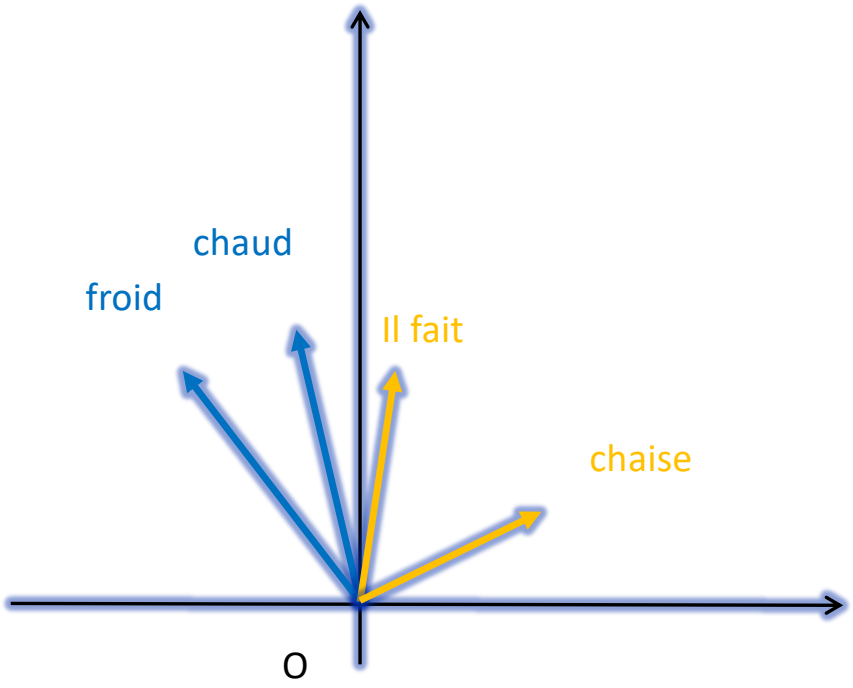
Il fait trés _____

- Comparaison avec le mot attendu

froid → 0.35
chaise → 0.42
chaud → 0.20

Un LLM apprend « tout seul » : apprentissage auto-supervisé

Apprendre = ajuster progressivement l'espace latent



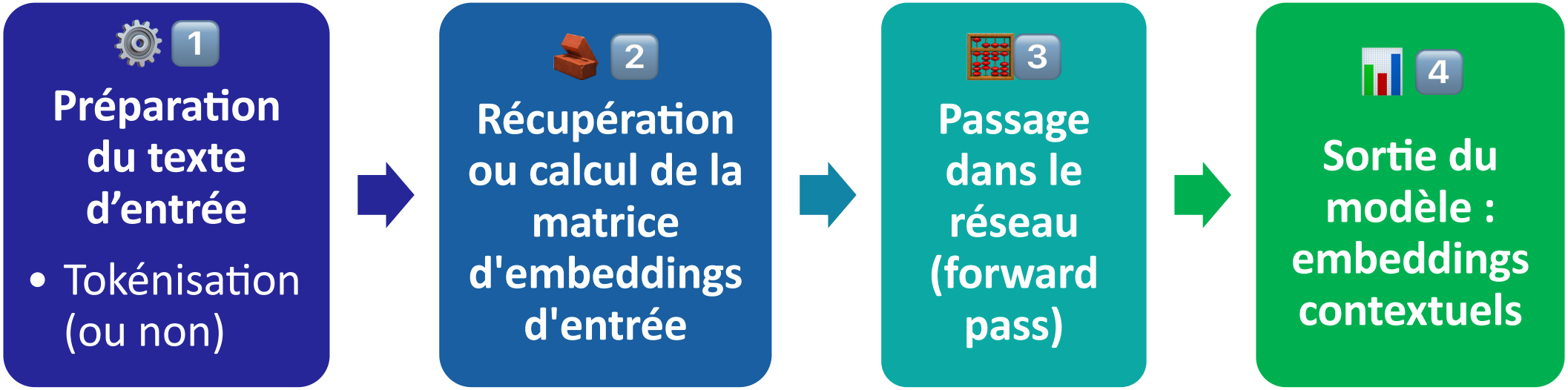
- Ajustement des poids

0.34	1.01	0.65
-1.23	0.26	-0.83
-1.00	-0.83	-0.57
-0.79	-1.60	-1.23

- Les vecteurs se réorganisent
- On recommence …

L'espace latent se construit progressivement

Etapes de l'inférence dans un modèle à embeddings



Deux types de modèles et leurs embeddings

Modèles dédiés aux embeddings 🎯

Vocation principale : apprendre une table de vecteurs

- **Word2Vec (2013)** : prédire le mot ou le contexte (Skip-gram, CBOW)
- **GloVe (2014)** : factorisation de matrices de cooccurrences
- **FastText (2016)** : sous-mots, morphologie

🔑 **But :** produire des représentations vectorielles utilisables directement (similarité, analogies, clustering, etc.).

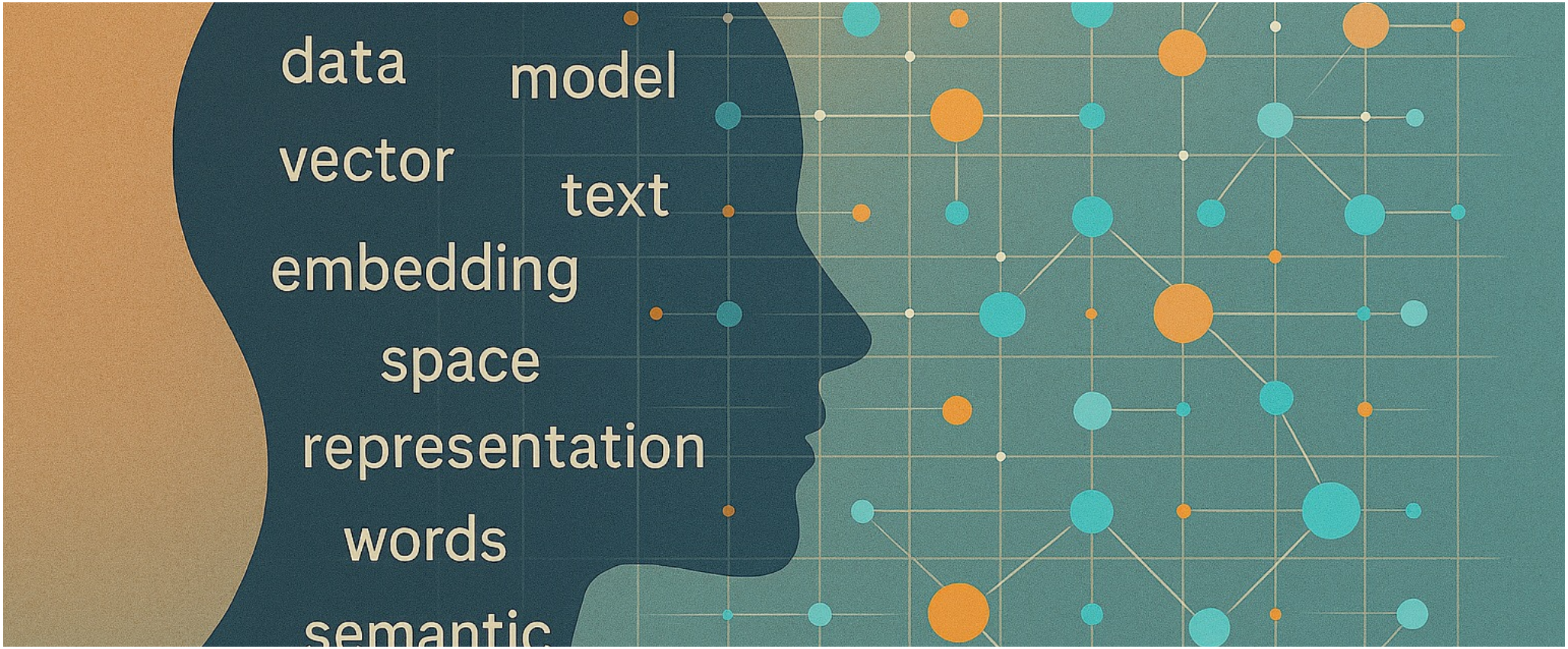
Modèles de langage générateurs d'embeddings 🌍

Vocation principale : modéliser le langage (prédiction, génération)

- **ELMo (2018)** : LSTM contextuels, plusieurs vecteurs par mot
- **BERT (2018)** : contextual embeddings issus des transformers
- **GPT (2018+)** : génération, mais hidden states = embeddings
- **Sentence-BERT (2019)** : adaptation de BERT pour créer des embeddings de phrases

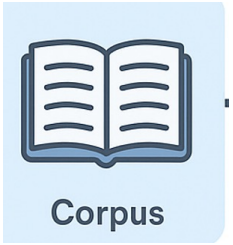
🔑 **But :** comprendre ou générer du texte → **embeddings = sous-produit** qu'on réutilise pour la recherche, la similarité, etc.

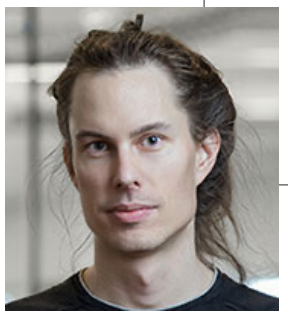
4 – Modèles dédiés aux word embeddings statiques



Word2Vec (Mikolov et al., 2013)

- Objectif :
  - Construire des vecteurs de mots à partir de corpus en fonction des mots voisins
- Paramètre:
  - taille de la fenêtre contextuelle
- Réseau de neurone simple à deux couches :
  - poids ajustés en minimisant les erreurs de prédiction





Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov
Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen
Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado
Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean
Google Inc., Mountain View, CA
jeff@google.com

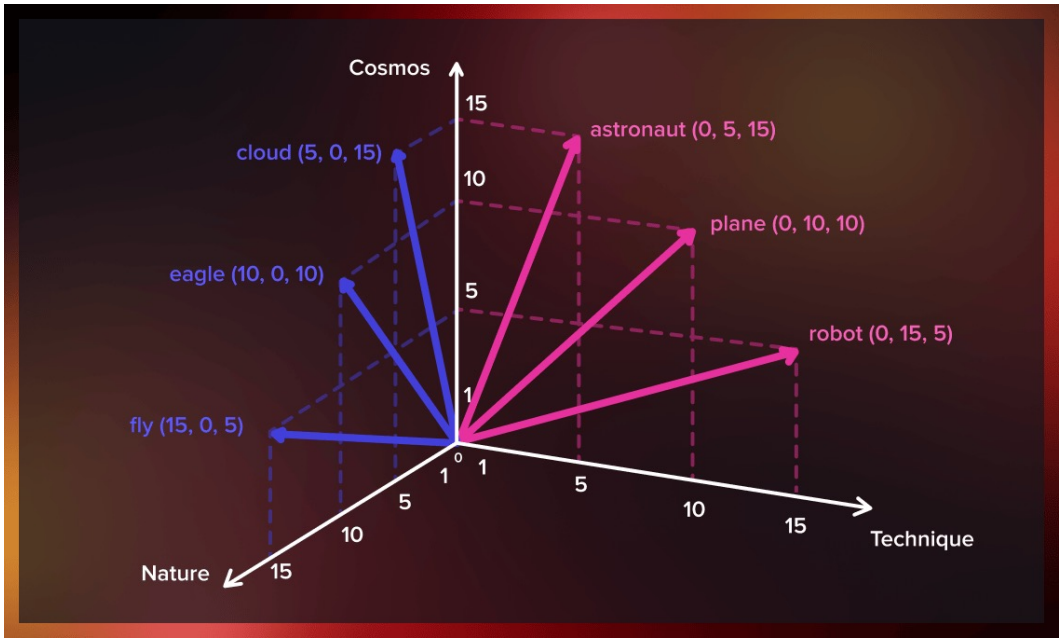
Abstract

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space.* Proceedings of ICLR 2013.https://arxiv.org/pdf/1301.3781

Principe général

- Chaque mot → un seul vecteur fixe, appris à partir de très grands corpus
- Le vecteur ne dépend pas du contexte général, mais encode une sorte de “moyenne” de ses usages
- Le contexte est ici limité aux mots voisins explorés selon la fenêtre définie :
  - ex: 2 mots avant et après

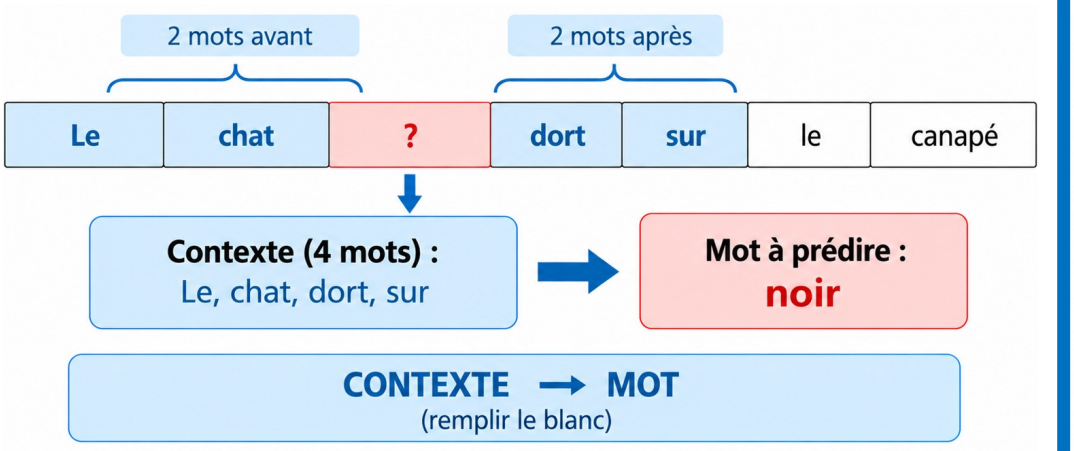


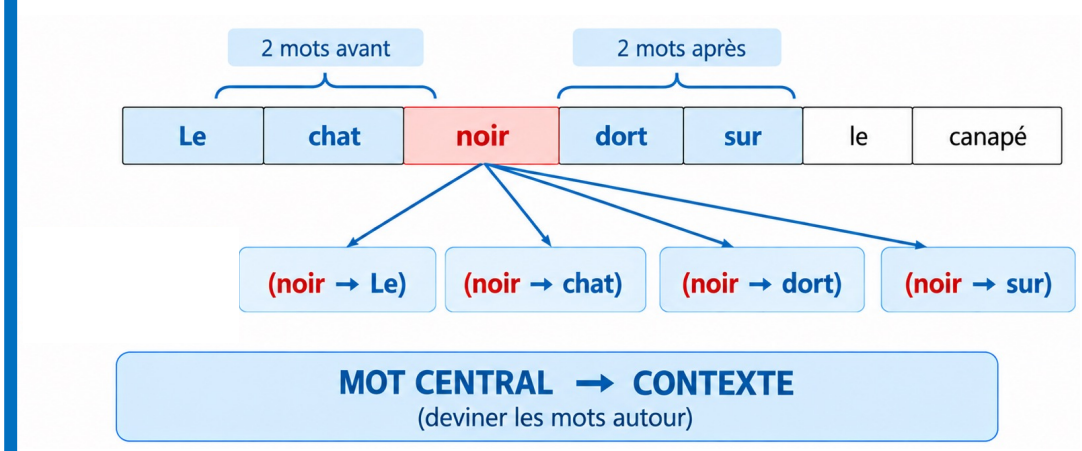
source : https://serokell.io/blog/word2vec

Meme si l'on ne parle pas encore d'embeddings contextuels, le contexte est bien utilisé pendant l'entrainement

Word2Vec: Deux techniques d'entrainement

- **CBOW (Continuous Bag of Words)** : prédire un mot manquant à partir du contexte = "remplir le blanc"
  - ex: 1 mot à partir de 4 autres mots
- **Skip-gram** : prédire le contexte à partir du mot. = "deviner les mots autour"
  - ex: probabilité de plusieurs mots environnants à partir d'un mot donné





ex: fenetre = 2 mots avant et aprés

CBOW : étape par étape (fenêtre = 2)

Phrase : … | Le | chat | **noir** | dort | sur | le | canapé | …

2 mots avant — 2 mots après

Exemple d'entraînement :
(Le, chat, dort, sur) → noir

1 Initialisation des vecteurs

Chaque mot du vocabulaire possède deux vecteurs (entrée et sortie) initialisés aléatoirement (dimension d = 3 ici).

	W_{in} (entrée)	W_{out} (sortie)
Le	[0.1 −0.3 0.2]	[…]
chat	[0.0 0.5 −0.1]	[…]
dort	[−0.2 0.1 0.3]	[…]
sur	[0.4 −0.1 0.2]	[…]
noir	[…]	[…]

2 Récupération des vecteurs du contexte

On **récupère** dans W_{in} les vecteurs des 4 mots de contexte.

Le	[0.1 −0.3 0.2]
chat	[0.0 0.5 −0.1]
dort	[−0.2 0.1 0.3]
sur	[0.4 −0.1 0.2]

3 Construction du vecteur de contexte

On **moyenne** les vecteurs des mots de contexte.

$$\mathbf{h} = \frac{1}{4}(\mathbf{v}_{Le} + \mathbf{v}_{chat} + \mathbf{v}_{dort} + \mathbf{v}_{sur})$$

$$h = [0.075 \quad 0.05 \quad 0.15]$$

(exemple de valeurs)

4 Calcul des scores

On calcule un score pour chaque mot candidat de W_{out} par produit scalaire :

$$s_w = \mathbf{h} \cdot \mathbf{u}_w$$

$s_{noir} = h \cdot u_{noir} = 0.60$

$s_{blanc} = h \cdot u_{blanc} = 0.20$

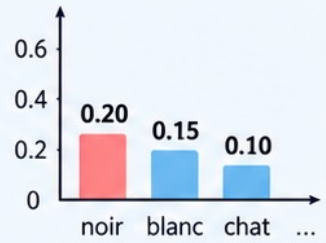
$s_{chat} = h \cdot u_{chat} = -0.10$

…

5 Scores → Probabilités

On transforme les scores en probabilités (softmax).

$$P(w \mid \text{contexte}) = \frac{e^{s_w}}{\sum_j e^{s_j}}$$



6 Calcul de l'erreur

La cible attendue est « noir ». On mesure l'erreur avec la cross-entropie :

$$L = -\log P(\text{noir} \mid \text{contexte})$$

Si $P(\text{noir}) = 0.20$, alors $L = -\log(0.20) = 1.61$ (erreur importante).

7 Rétropropagation

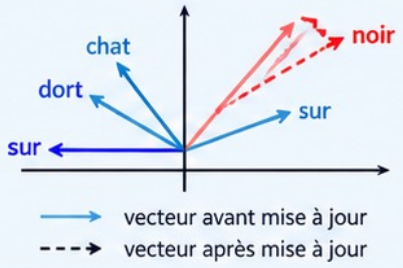
On calcule le gradient de l'erreur et on modifie légèrement :

- les vecteurs d'entrée des mots du contexte (Le, chat, dort, sur)
- le vecteur de sortie du mot cible (noir)

(et éventuellement d'autres mots avec le softmax)

8 Vecteurs mis à jour

Les vecteurs sont ajustés pour augmenter la probabilité de la cible.



Skip-gram : étape par étape (fenêtre = 2)

Phrase : … | Le | chat | **noir** | dort | sur | le | canapé | …

2 mots avant — 2 mots après

Exemples d'entraînement :

noir → Le | **noir** → dort

noir → chat | **noir** → sur

1 Initialisation des vecteurs

Chaque mot du vocabulaire possède deux vecteurs (entrée et sortie) initialisés aléatoirement (dimension $d = 3$ ici).

	W_{in} (entrée)	W_{out} (sortie)
Le	[0.1 −0.3 0.2]	[...]
chat	[0.0 0.5 −0.1]	[...]
dort	[−0.2 0.1 0.3]	[...]
sur	[0.4 −0.1 0.2]	[...]
noir	[0.3 0.2 −0.2]	[...]
…	…	…

2 Choix d'un exemple d'entraînement

On prend un mot central et un mot de son contexte.

Ici : **noir → chat**

On récupère :

vecteur d'entrée du mot central	vecteur de sortie du mot cible
v_{noir}	u_{chat}
[0.3 0.2 −0.2]	[0.0 0.5 −0.1]

3 Calcul d'un score

On calcule un score de compatibilité entre le mot central et le mot cible (par produit scalaire).

$$s = v_{noir} \cdot u_{chat}$$
$$= 0.3 \times 0.0 + 0.2 \times 0.5 + (-0.2) \times (-0.1)$$
$$= 0.12$$

4 Transformation en probabilité

On transforme le score en probabilité que « chat » apparaisse dans le contexte de « noir ».

$$P(\text{chat} \mid \text{noir}) = \sigma(s)$$

(fonction sigmoïde, avec negative sampling)

$$\sigma(0.12) = \frac{1}{1 + e^{-0.12}} \approx 0.53$$

5 Utilisation d'exemples négatifs

On choisit quelques mots qui ne sont pas dans le contexte, par exemple : voiture, ordinateur, arbre.

mot négatif	vecteur de sortie	score	$\sigma(-s)$
voiture	[−0.3 0.1 0.2]	−0.08	0.52
ordinateur	[0.4 −0.2 0.1]	0.06	0.48
arbre	[−0.1 0.3 0.4]	−0.05	0.51

6 Calcul de l'erreur

On veut que le mot cible ait une probabilité élevée et les mots négatifs une probabilité faible.

$$L = -\log \sigma(s_{pos}) - \sum_i \log \sigma(-s_{neg,i})$$

(negative sampling)

Avec $s_{pos} = 0.12$ et s_{neg} autour de 0 :

$L \approx 0.63$ (erreur importante)

7 Rétropropagation : modification des vecteurs

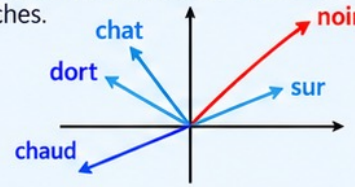
On calcule le gradient de l'erreur et on met à jour légèrement :

- le vecteur d'entrée de « noir » (v_{noir})
- le vecteur de sortie de « chat » (u_{chat})
- les vecteurs de sortie des mots négatifs

$$v \leftarrow v - \eta \frac{\partial L}{\partial v}$$ (descente de gradient)

8 On répète sur tout le corpus

On parcourt tout le texte, pour chaque mot central, et pour chaque mot de son contexte. Après de nombreux ajustements, les mots apparaissant dans des contextes similaires ont des vecteurs proches.



Choix Cbow ou skip-Gram ?

CBOW

- Combine (moyenne) les vecteurs du contexte pour prédire 1 mot
  - Donc 1 prédiction par fenêtre
- Moins coûteux →
  - Plus rapide, car moins de calculs
  - efficace sur gros corpus
- Tendance à lisser les représentations (bon pour mots fréquents)

Skip-Gram

- Pour chaque mot cible, on doit prédire tous les mots du contexte
  - Donc plusieurs prédictions par fenêtre.
- ⇒ Plus coûteux →
  - Plus lent, surtout sur gros corpus
  - mieux sur petits corpus
- Apprend mieux les vecteurs des mots rares, car chaque paire (mot, contexte) est utilisée

Dans les deux cas, on obtient :

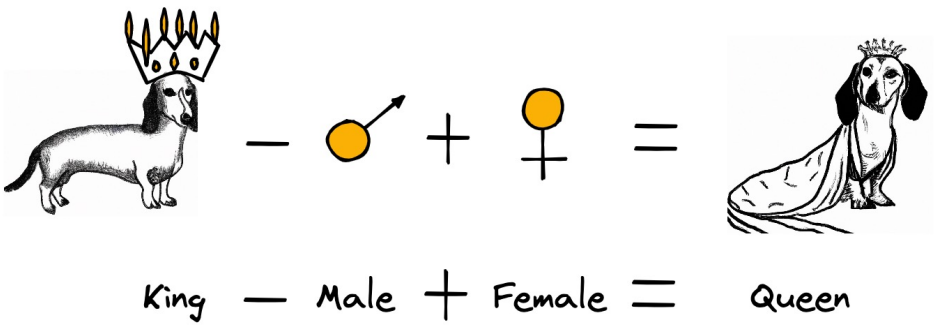
- Une matrice d'embedding (vecteurs denses pour chaque mot)
- Des similarités exploitables

🌟 Apports de Word2Vec

- Passage du discret au continu
  - Chaque mot = vecteur dense de dimension faible (100–300)
  - Représentation compacte et efficace
- Captation des similarités sémantiques
- Analogies et régularités vectorielles
- Efficacité d'entraînement

Impact majeur

- A ouvert la voie aux embeddings modernes et aux LLM
- Utilisé massivement dans NLP



source : https://geroldcsendes.github.io/blog/Word2Vec/

Résultat de l'entrainement d'un modèle Word2Vec

- Le modèle apprend une grande matrice d'embeddings notée W_{in} de taille : $|V| \times d$

où :

  - $|V|$= taille du vocabulaire (min 30000 mots)
  - d= dimension des embeddings (100 à 1024)

- Chaque ligne de cette matrice correspond à un mot du vocabulaire.
  👉 Cette ligne est le vecteur du mot appris

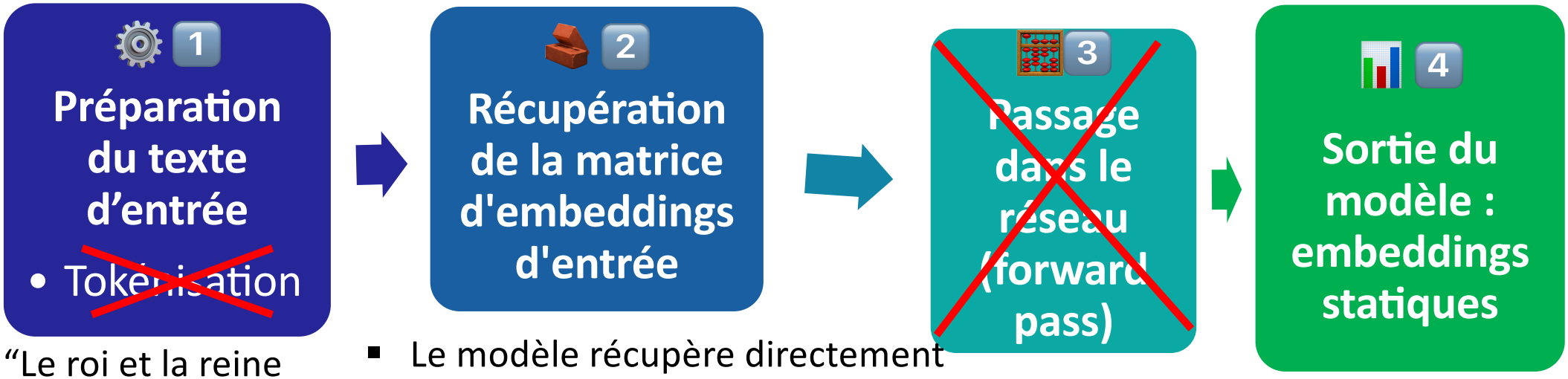
$$W_{in} = \begin{bmatrix} \text{roi} & \rightarrow & [0.21,\ 0.45,\ 0.10] \\ \text{reine} & \rightarrow & [0.25,\ 0.48,\ 0.09] \\ \text{homme} & \rightarrow & [0.33,\ 0.12,\ 0.56] \\ \text{femme} & \rightarrow & [0.34,\ 0.15,\ 0.51] \\ \text{château} & \rightarrow & [0.80,\ 0.10,\ 0.20] \end{bmatrix}$$

La matrice W_{in} est figée après l'entrainement

Chaque mot est représenté par son index = sa ligne dans la matrice

Spécificités de l'inférence dans Word2Vec

Le contexte n'est pas utilisé
il n'est utilisé que lors de l'entrainement



"Le roi et la reine vivent au chateau" → [roi, reine, château]
Les mots absents du vocabulaire sont ignorés

- Le modèle récupère directement les embeddings dans la matrice W_{in} :

v(roi)=[0.21,0.45,0.10]
v(reine)=[0.25, 0.48, 0.09]
v(château)=[0.80, 0.10, 0.20]

Simple recherche dans une table de vecteurs lookup

Word2Vec à l'inférence : obtenir un embedding

Le roi **gouverne** le royaume.
C'est le **roi** du tennis.

roi

1 Recherche dans le vocabulaire
On récupère l'index du mot dans le vocabulaire (ici : 5782).

index : 5782

2 Lookup dans la matrice d'embeddings
On récupère directement la ligne correspondante dans la matrice d'entrée W_{in}.

3 Embedding du mot
Vecteur de dimension d (exemple : $d = 5$ ici).

$v(\text{roi}) = [0.21, 0.45, 0.10, -0.05, 0.33]$

Matrice d'embeddings d'entrée W_{in}

index	mot	v_1	v_2	v_3	v_4	v_5	...
0	le	0.12	−0.21	0.37	...	...	...
1	chat	0.05	0.48	−0.10	...	...	...
...	...	...	...	...	...	...	...
5782	roi	0.21	0.45	0.10	−0.05	0.33	...
5783	reine	0.19	0.41	0.12	−0.08	0.30	...
...	...	...	...	...	...	...	...

Le contexte n'est pas utilisé à l'inférence.
Un même mot a toujours le **même embedding (embeddings statiques).**

Exemple : même vecteur selon le contexte

Le **roi** gouverne le royaume. → $v(\text{roi}) = [0.21, 0.45, 0.10, -0.05, 0.33]$

C'est le **roi** du tennis. → $v(\text{roi}) = [0.21, 0.45, 0.10, -0.05, 0.33]$

- Simple recherche dans une **table de vecteurs (lookup).**
- **Pas de forward pass** dans le réseau d'entraînement.
- **Pas** de calcul de similarité (sauf si l'on recherche des mots proches, voir diapo suivante).
- Le même mot a toujours le même vecteur, quelle que soit la phrase.

Word2Vec : un autre usage — rechercher les mots les plus proches

1 Récupérer l'embedding du mot

On obtient le vecteur du mot dans la matrice d'embeddings.

roi ➡ $v(roi) = [0.21, 0.45, 0.10, -0.05, 0.33]$

2 Calculer la similarité avec tous les mots

On compare $v(roi)$ aux vecteurs de tous les mots du vocabulaire, par similarité cosinus.

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|}$$

Si les vecteurs sont normalisés : cosinus = produit scalaire.

3 Classer les mots par score décroissant

On trie les mots selon leur similarité avec $v(roi)$ et on garde les k plus proches.

4 Résultat : les mots les plus proches

On obtient les mots les plus similaires à « roi », qui partagent des contextes similaires dans le corpus.

Extrait de la matrice d'embeddings W_{in}

index	mot	v_1	v_2	v_3	v_4	v_5	...
0	le	0.12	−0.21	0.37	0.01	−0.18	...
1	chat	0.05	0.48	−0.10	0.03	0.27	...
...	...	...	...	...	...	...	...
5782	**roi**	0.21	0.45	0.10	−0.05	0.33	...
5783	reine	0.19	0.41	0.12	−0.08	0.30	...
...	...	...	...	...	...	...	...

Pour chaque mot w du vocabulaire :

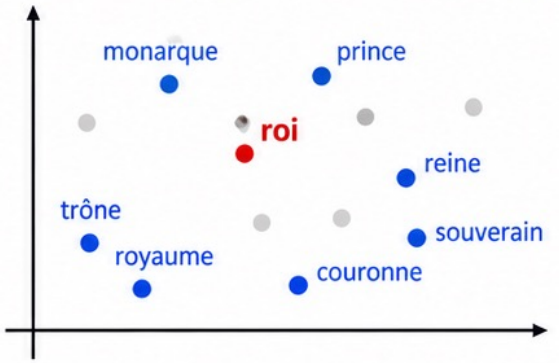
$$\text{score}(w) = \cos(v(roi), v(w))$$

On trie les mots par score décroissant et on sélectionne les k meilleurs.

Exemple de résultats (top 10)

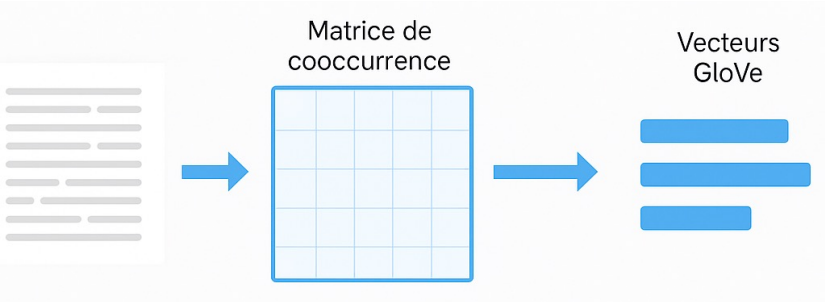
rang	mot	similarité cosinus
1	reine	0.82
2	prince	0.76
3	monarque	0.73
4	royaume	0.65
5	souverain	0.62
6	trône	0.61
7	couronne	0.58
8	rois	0.55
9	impérial	0.52
10	reine-mère	0.49
...	...	...

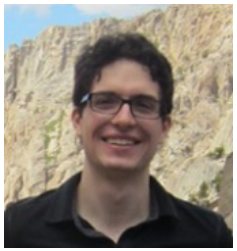
Visualisation (projection 2D des embeddings)



Modèle GloVe (Pennington & al., 2014)

- Global Vectors for Word Representation
- Basé sur une matrice de cooccurrence (globale) :
  - combien de fois un mot i apparaît avec j ?
- Chaque vecteur encode les statistiques globales de coapparition des mots
- Combine approche locale (Word2Vec) + globale (cooccurrence)





Stanford University

Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
https://aclanthology.org/D14-1162.pdf

Modèle FastText (Bojanowski & al. 2017)

Principe : enrichir les embeddings par les n-grams de caractères

- Un n-gram est une séquence continue de n-éléments (mots, caractères, …) : fenêtre "glissante" de longueur n sur une phrase
- Exemple : Le chat dort sur le canapé

Type de n-gram	Exemple	Contient…
1-gram (unigramme)	[Le], [chat], [dort], [sur], [le], [canapé]	1 mot
2-gram (bigramme)	[Le chat], [chat dort], [dort sur], [sur le], [le canapé]	2 mots consécutifs
3-gram (trigramme)	[Le chat dort], [chat dort sur], [dort sur le], [sur le canapé]	3 mots consécutifs
4-gram (quadrigramme)	[Le chat dort sur], [chat dort sur le], [dort sur le canapé]	4 mots consécutifs

Modèle FastText (Bojanowski & al. 2017) fastText

Vecteur d'un mot = somme des vecteurs de ses **n-grams de caractères**

Exemple : « jouer » → <jo, jou, oue, uer, er>

Chaque n-gram a un vecteur

v(jouer)= somme (v(<jo), v(jou), v(oue), v(uer), v(er))

✅ **Avantages** :

- Gère les mots rares ou inconnus (**OOV = Out-Of-Vocabulary**)
- Prend en compte la morphologie (préfixes, suffixes)
- Compatible avec Word2Vec (CBOW, Skip-gram)

facebook AI Research

Meta AI

Bojanowski, P. *et al.* (2017) 'Enriching Word Vectors with Subword Information', *Transactions of the Association for Computational Linguistics*, 5, pp. 135–146. Available at: https://doi.org/10.1162/tacl_a_00051.

Mise en pratique Word2Vec/fastText avec Gensim

GENSIM

topic modelling for humans

Manipuler des embeddings avec Gensim : vue d'ensemble des étapes

Des textes aux vecteurs, puis à leur utilisation

1 Préparer le corpus

Charger et tokeniser un ensemble de textes (phrases ou documents).

```
corpus = [
 ["le", "chat", "chasse", ...],
 ["la", "souris", "se",
 "cache", ...],
 ...
]
```

2 Créer ou charger le modèle

Entraîner un modèle Word2Vec ou FastText avec Gensim, ou charger un modèle pré-entraîné.

```
from gensim.models
import Word2Vec

model = Word2Vec(
    sentences=corpus,
    vector_size=300,
    window=5,
    min_count=2,
    workers=4
)
```

3 Accéder aux vecteurs

Récupérer le vecteur d'un mot (ou vérifier qu'un mot est connu).

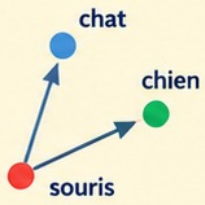
mot	vecteur (d = 300)
chat	[0.12 -0.03 ...]
souris	[-0.21 0.08 ...]
chasse	[0.05 0.31 ...]
...	...

```
vec = model.wv["chat"]
print(vec.shape)
```

```
(300,)
```

4 Comparer des mots

Mesurer la similarité ou la distance entre deux mots.




```
model.wv.similarity(
   "chat", "chien"
)
```

```
0.72
```

5 Explorer l'espace vectoriel

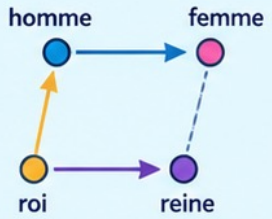
Trouver les mots les plus proches d'un mot (voisins les plus proches).

```
model.wv.most_similar(
  "chat", topn=5)
)
```

chien	0.72
animal	0.68
chats	0.66
félin	0.63
ronronner	0.62

6 Manipuler les relations

Réaliser des analogies vectorielles.




```
model.wv.most_similar(
   positive=["roi", "femme"],
   negative=["homme"],
   topn=3
)
```

reine	0.73
princesse	0.62
royale	0.59

À retenir

- Gensim permet d'entraîner facilement des modèles Word2Vec ou FastText à partir de vos textes ou de charger des modèles pré-entraînés.
- On obtient un vecteur dense pour chaque mot du vocabulaire.
- Ces vecteurs permettent de mesurer la similarité, de trouver des mots proches et de réaliser des analogies.

Objectif des diapos suivantes

- Montrer concrètement ces étapes avec du code :
- entraînement / chargement du modèle,
- accès aux vecteurs,
- mesure de similarité,
- voisins les plus proches,
- analogies vectorielles.

Gensim: bibiothèque python pour les embeddings

- **Entraîner** ses propres modèles (*Word2Vec*, *FastText*)
- **Charger** des modèles pré-entraînés (*KeyedVectors*)
- **Explorer** les relations entre mots Similarité cosinus
  - Recherche de voisins (most_similar)
  - Analogies vectorielles (roi – homme + femme ≈ reine)

- Un seul outil pour plusieurs modèles
  - Word2Vect
  - FastText

```
#Installation
pip install gensim
```

Word2Vec/fastext avec Gensim : étapes

1. Préparer le corpus

   Texte brut → segmentation en phrases → tokenisation en mots

```
corpus= [["le","chat","mange","des","croquettes"], ...]
```

2. Créer et entraîner le modèle

```
model = Word2Vec(sentences=..., vector_size=...,..., ...)
#ou model = FastText(sentences …
```

3. Explorer les vecteurs

```
#Récupérer un vecteur : model.wv["chat"]
#Comparer des mots : model.wv.most_similar("chat")
```

Si on utilise un modèle préentrainé : `model = KeyedVectors.load("fichier.kv")`
Attention: on a directement model (pas besoin de model.wv)

Paramètres d'entrainement Word2Vec/FastText

- sentences: liste de tokens List[List[str]]
- vector_size : dimension des vecteurs (50, 100, 300…)
- window : taille de la fenêtre de contexte
- min_count : filtre les mots rares (ignore les mots qui apparaissent <min_count)
- sg : choix de l'algorithme
  - 0 → CBOW (*plus rapide, corpus grand*)
  - 1 → Skip-Gram (*meilleur sur petits corpus*)
- epochs : nombre d'itérations d'apprentissage (plus pour les petits corpus!)
- workers : nombre de threads CPU pour accélérer (optionnel)

```
from gensim.models import Word2Vec

model = Word2Vec(
  sentences=corpus,  # corpus
  vector_size=100,   # taille vecteur
  window=5,          # fenêtre contexte
  min_count=1,       # filtre
  sg=1,              # 0=CBOW,1 =Skip-Gram
  epochs=5,          # itérations
)
```

Utilisations du modèle

- Obtenir un vecteur dense

```
#model.wv[word: str] -> numpy.ndarray
model.wv["chat"]
# ex: array([ 0.12, -0.03, 0.45, ..., 0.07], dtype=float32)
```

- Mesurer la similarité cosinus entre deux mots

entre 0 et 1

```
# model.wv.similarity(word1: str, word2: str) -> float
print(model.wv.similarity("chat", "chien"))
# ex 0.87
```

- Calculer la distance cosinus entre plusieurs mots

distance cosinus =
1 - similarité_cosinus

```
#model.wv.distances(word: str, other_words: list[str]) -> numpy.ndarray
print(model.wv.distances("chat", ["chien","oiseau"]))
# ex: array([0.13, 0.42])
```

Plus proches voisins

```
# model.wv.most_similar(
    list[str],topn: int = 10) -> list[tuple[str,float]]
```

- Trouver les 3 plus proches voisins d'un mot

```
model.wv.most_similar("chat", topn=3)
#ex : [("chien", 0.87),
  ("souris", 0.81),
  ("animal", 0.78)]
```

Similarité cosinus

Analogies (type roi-homme+femme?)

```
# model.wv.most_similar(
    positive:str | list[str] | dict[str, float],
    negative:str | list[str] | dict[str, float] = None,
    topn: int = 10
) -> list[tuple[str, float]]
```

Quel mot a avec « femme » la même relation que « roi » avec « homme » ?

```
model.wv.most_similar(positive=["roi","femme"],
                      negative=["homme"], topn=3)
# ex : [("reine", 0.89),
    ("princesse", 0.78),
    ("souveraine", 0.75)]
```

Cherche les mots dont le vecteur est le plus proche de cette combinaison linéaire
vect("roi")−vect("homme")+vect("femme")

Interprétation d'une analogie

- vect("roi")−vect("homme")+vect("femme")
- On fait une opération géométrique sur les mots :
  - On retire le sens “homme” du mot “roi”
  - On obtient un vecteur auquel on ajoute le sens “femme”
  - On cherche alors les mots similaires à ce vecteur calculé
    - Le mot qui se trouve le plus proche de ce nouveau point dans l’espace est par exemple “reine”.

Le modèle a appris que roi/homme et reine/femme sont des relations analogues

Exemple

```
from gensim.models import Word2Vec
sentences = [
 ["le", "chat", "mange", "des", "croquettes"],
 ["le", "chien", "dévore", "des", "os"],
 ["un", "chat", "est", "un", "animal", "domestique"],
 ["un", "chien", "est", "un", "animal",
"domestique"]]
# Entraînement du modèle
model = Word2Vec(
  sentences=sentences,
  vector_size=5,    # taille du vecteur dense
  window=2,          # contexte de 2 mots
  min_count=1,       # garder tous les mots
  sg=1,              # Skip-Gram
  epochs=200         # itérations
)
# Affichage des vecteurs appris
for word in model.wv.index_to_key:
    print(f"{word:12s} -> {model.wv[word]}")
```

corpus jouet

```
un           -> [-0.01702737  0.01183738  0.11816302  0.18594542 -0.19333751]
domestique   -> [-0.14548221  0.13289596  0.18694769 -0.09737356 -0.08016656]
animal       -> [ 0.14577308 -0.02811422 -0.08483662  0.1350295  -0.10190821]
est          -> [-0.03806889  0.0602668   0.02701619 -0.16348517 -0.19588302]
chien        -> [0.14524257 0.10613511 0.14463858 0.02077042 0.12558773]
des          -> [-0.0728342  -0.01310823  0.12506396 -0.14769511 -0.07970104]
chat         -> [-0.15401435 -0.01574212  0.19910468 -0.14510307 -0.05102513]
le           -> [-0.0409691   0.16716252 -0.10931668  0.0067923  -0.09430145]
os           -> [-0.19535488  0.1054064  -0.16874677 -0.0847601  -0.0021147 ]
dévore       -> [-0.00795532 -0.14904255  0.19920957  0.10328539  0.1844665 ]
croquettes   -> [-0.16471216  0.09283617 -0.07887904  0.0185333   0.16983849]
mange        -> [-0.09133989  0.09258478 -0.13255827 -0.0693955   0.18734096]
```

A vous de jouer : TP1-WordEmb.pynb

- Récupérez le noteBook TP1-WordEmb.pynb
- Récupérez le modèle pré-entrainé réduit :

  Deux fichiers à télécharger et à placer dans le même répertoire que le notebook:

  - cc.fr.100k.kv (structure du modèle : le vocabulaire (les mots connus), les paramètres (dimension, etc.), et les métadonnées.)
  - cc.fr.100k.kv.vectors.npy (matrice numérique des vecteurs)

Partie A — Entraînement de modèles Word2Vec et FastText from scratch (corpus jouet)

- Visualisation des vecteurs
- Exploration des similarités
- Variation des paramètres
- Comparaison Word2Vec-FastText

GENSIM
topic modelling for humans

Partie B — Modèle pré-entrainé FastText (version réduite)

- Analogies
- Problème de polysémie

Certaines cellules de code sont juste à tester, parfois en faisant varier certains paramètres, d'autres sont à compléter.
Répondez aussi à plusieurs questions 💭 dans les cellules de texte.

Limites des embeddings statiques

- Vecteurs appris une fois pour toutes
  - Entrainement sur un corpus fixe
  - Pas d'adaptation, pas d'évolution, pas d'incrémentalité
- Polysémie non gérée : un seul vecteur pour plusieurs sens

Besoin : des vecteurs qui varient **selon le contexte**.

5 – Modèles d’embeddings contextuels

Principe des embeddings contextuels
Ce qui change à partir de 2018

- Le vecteur d'un mot dépend de son contexte :
  - plusieurs vecteurs pour chaque mot
- Construction basée sur des réseaux de neurones profonds séquentiels
  - LSTM (**Long Short Term Memory)** puis Transformers et le mécanisme d'attention
- Vecteurs évolutifs : Ne sont pas fixés une fois pour toutes
- Sont calculés dynamiquement à partir d'une phrase entière

Captation des nuances de sens et de syntaxe

Notion d'embedding dynamique

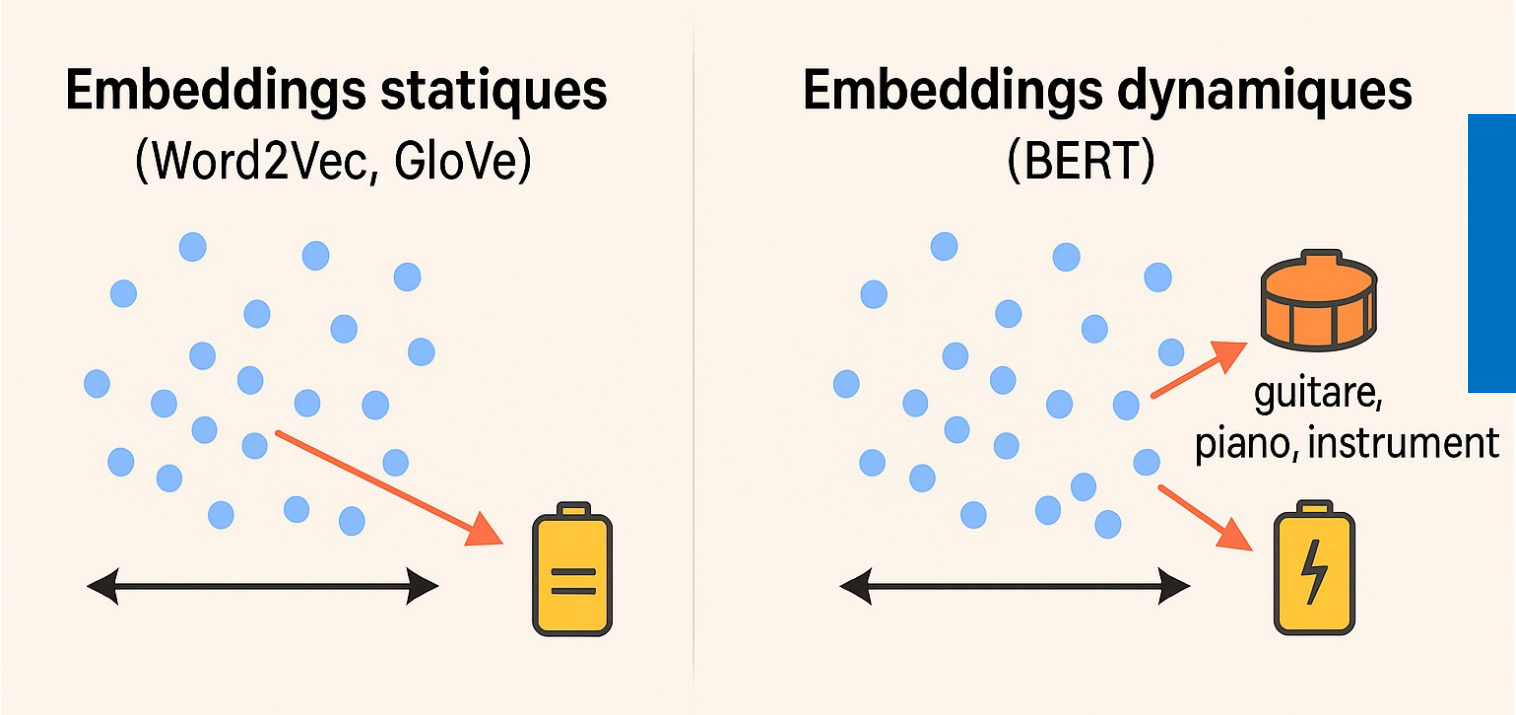
- Plusieurs vecteurs par mot :

  Chaque occurrence d'un mot

  a son propre vecteur

Il joue de la *batterie*.

La batterie de mon téléphone est faible.



On parle de contextual embeddings

Au-delà des embeddings : la vocation des modèles de langage

- **Comprendre le langage** 🧠
  - Analyse du contexte
  - Désambiguïsation du sens
- **Prédire et générer** ✍️
  - Complétion de texte
  - Traduction, résumé, dialogue

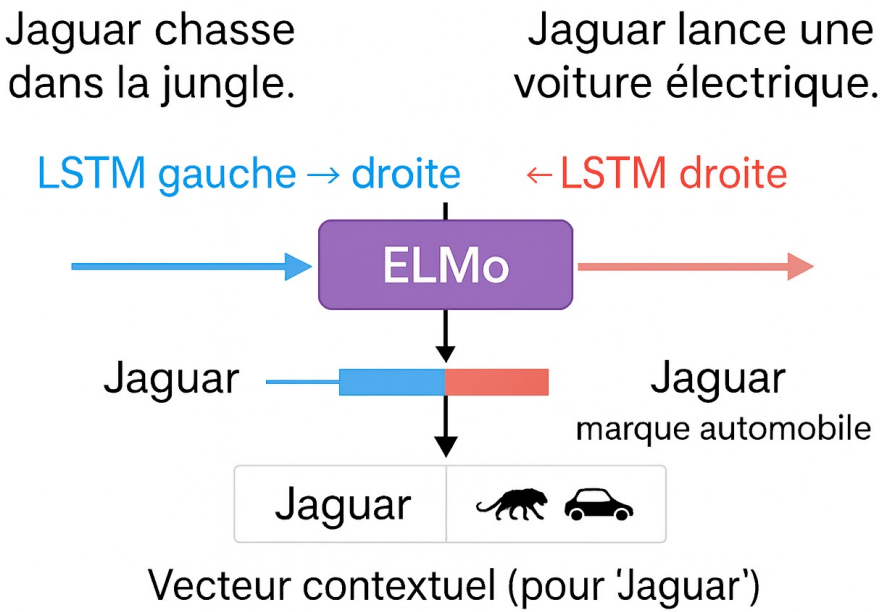
- **Produire des représentations utiles** 🔗
  - Les *embeddings* sont un **sous-produit**
  - Réutilisés pour la recherche sémantique, la classification, etc.

📦 Word2Vec, GloVe = modèles d'embeddings statiques
🤖 ELMo, BERT = modèles de langage qui génèrent aussi des embeddings

ELMo (Mattew & al., 2018)
Embeddings from Language Models

Premier changement majeur : des représentations contextuelles

- Basé sur des LSTM (Long Short-Term Memory) bidirectionnels
  - réseau séquentiel qui construit une mémoire au fur et à mesure qu'il parcour les mots.
- La phrase est parcourue dans les deux directions
- Le contexte intervient dans la représentation de chaque occurrence

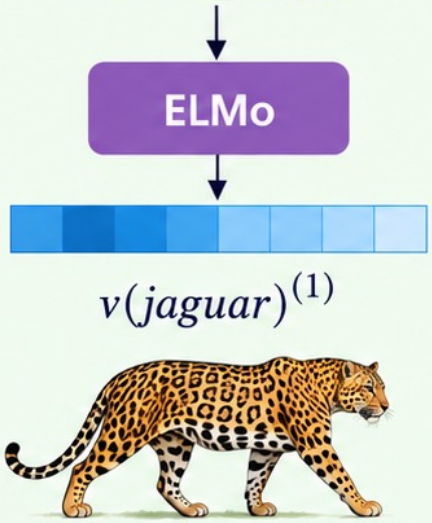


Peters, Matthew E., et al. (2018). *Deep contextualized word representations.* In: *Proceedings of the 2018 Conference of the North American Human Language Technologies (NAACL-HLT 2018)*, Volume 1 (Long Papers), pp. 2227–2237.
https://aclanthology.org/N18-1202

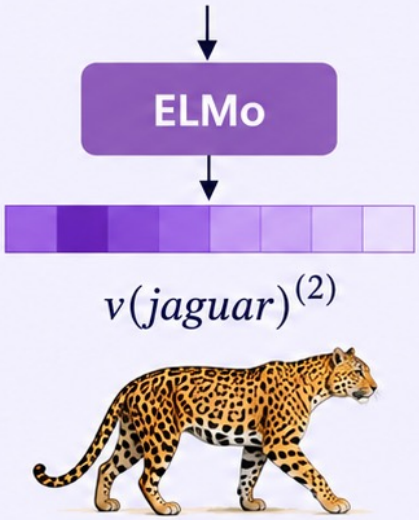
ELMo : des embeddings contextualisés

- Contrairement à Word2Vec, un mot a un vecteur fixe, indépendamment du contexte.
- ELMo calcule **une représentation pour chaque occurrence** d'un mot en fonction de la phrase entière.

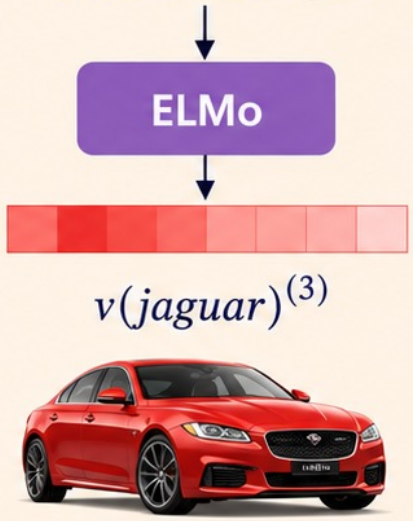
Le **jaguar** chasse dans la jungle.



Le **jaguar** se promène.



Jaguar lance une voiture électrique.



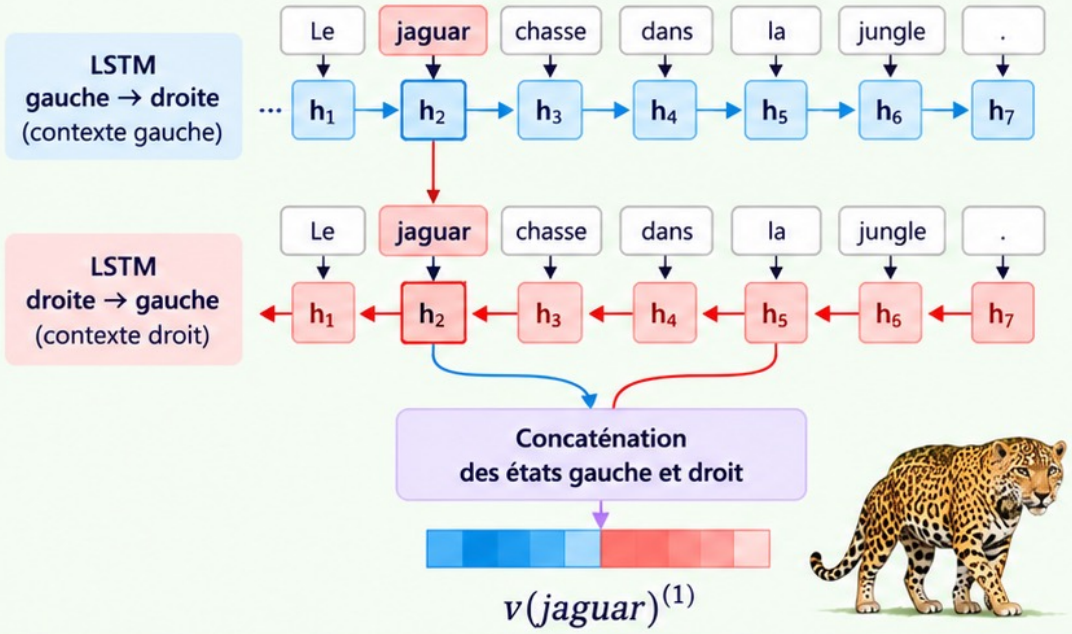
$$v(jaguar)^{(1)} \neq v(jaguar)^{(2)} \neq v(jaguar)^{(3)}$$

Même mot, contextes différents → vecteurs différents.

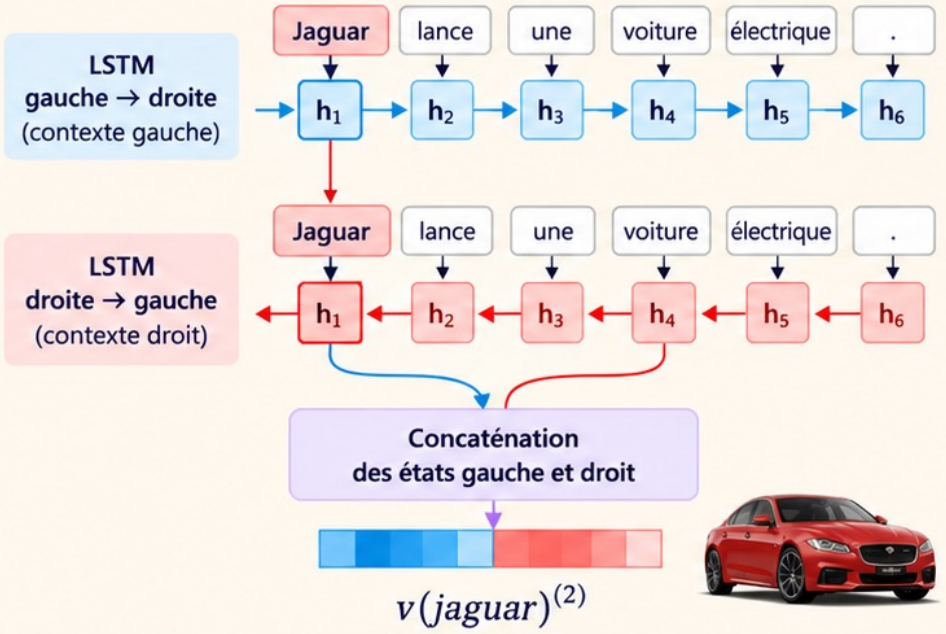
ELMo : comment le contexte produit des vecteurs différents ?

1. Chaque mot est d'abord représenté par un vecteur d'entrée (embedding de base), identique quel que soit le contexte.
2. Deux LSTM parcourent la phrase : un de gauche à droite (contexte gauche), un de droite à gauche (contexte droit).
3. À chaque position, ils calculent un état caché à partir du mot courant et de l'état précédent, qui résume les mots déjà vus.
4. La représentation du mot est obtenue en combinant les états gauche et droit. Comme ces états dépendent du contexte, le vecteur final est différent.

Phrase 1 : « Le jaguar chasse dans la jungle. »

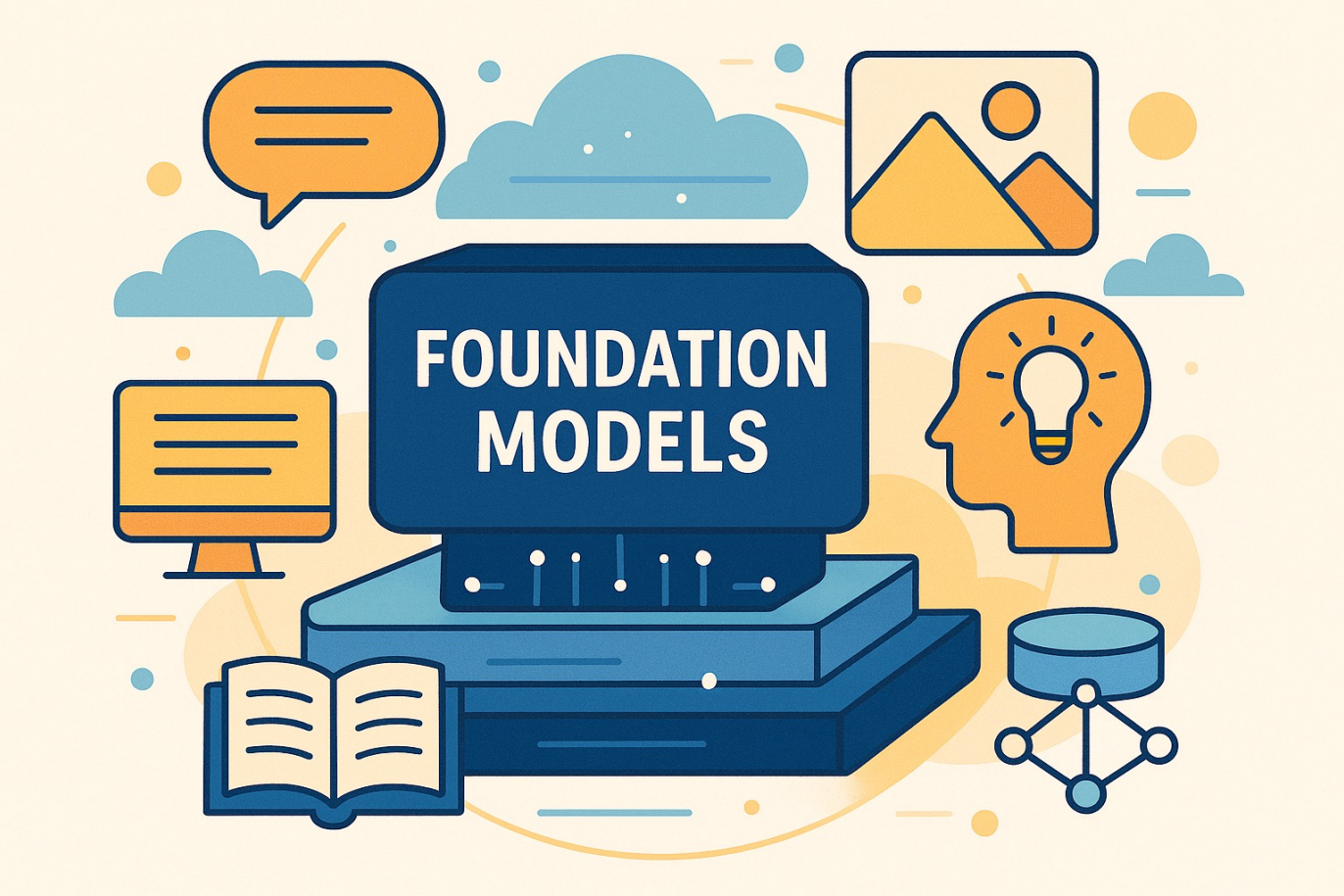


Phrase 2 : « Jaguar lance une voiture électrique. »



Même mot, contextes différents → $v(jaguar)^{(1)} \neq v(jaguar)^{(2)}$

BERT : le choc des Transformers

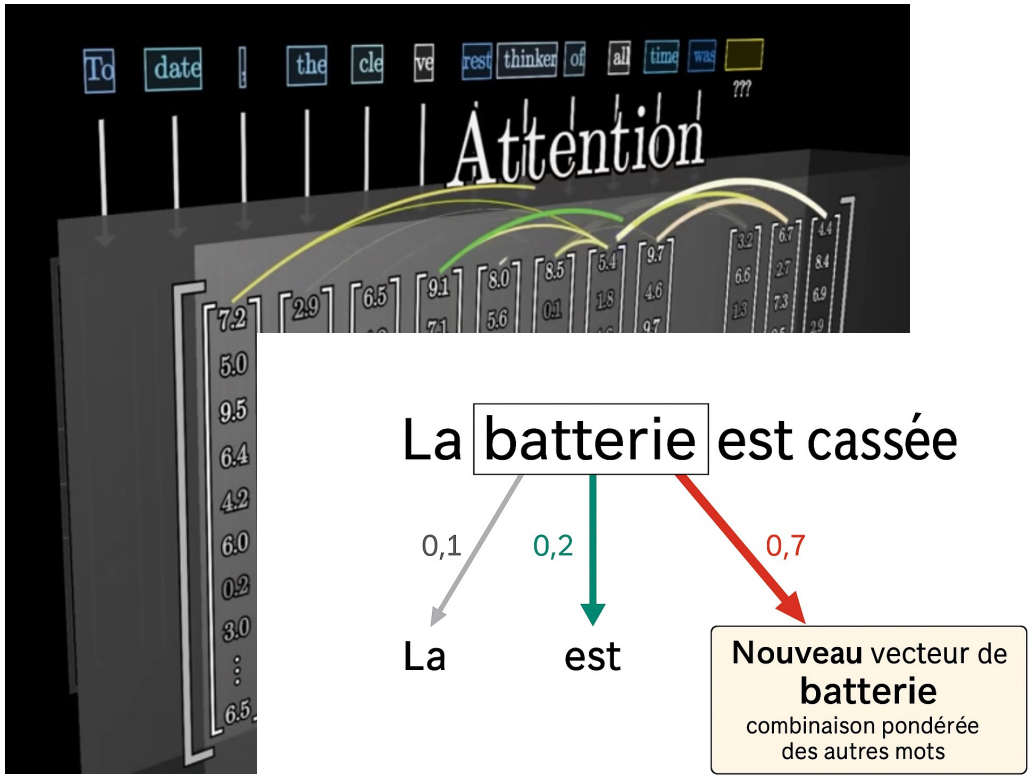


"Attention is all we need" (Vaswani & al., 2017)

- Mécanisme central : l'attention

  → chaque mot regarde tous les autres et pondère leur importance pour construire son embedding

  → calcul de **poids d'attention** (*attention scores*)

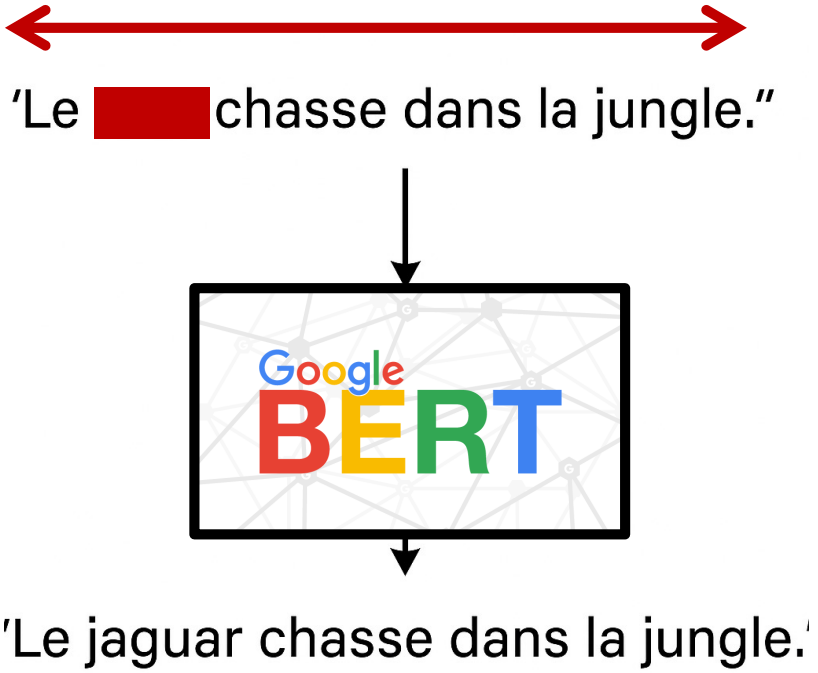


Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems, 30 (NIPS 2017). Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Une video explicative : https://www.youtube.com/watch?v=eMlx5fFNoYc&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=7

BERT
Bidirectional Encoder Representations from Transformers (Devlin & al. 2019)

- Remplace les LSTM par les **Transformers** (self-attention)
- Apprentissage auto-supervisé : **Masked Language Modeling** (Objectif= prédire des mots masqués)
- Produit des embeddings puissants et contextuels



Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019).
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
Proceedings of NAACL-HLT 2019, pp. 4171-4186.
arXiv:1810.04805

BERT: variantes et améliorations

Plusieurs modèles améliorent les performances de BERT initial :

- RoBERTa : plus de données, meilleur entraînement
- DistilBERT : version légère et rapide
- ALBERT : paramètres partagés → efficacité
- **CamemBERT** : adaptation française de BERT, entraînée sur 138 Go de texte OSCAR (Inria + Facebook AI)
- Tous gardent le principe d'embeddings contextuels

Deux types d'Embeddings dans les transformers encodeurs (ex: BERT)

Embeddings de tokens

Les embeddings ne portent plus sur des mots (Word2Vec) mais sur des tokens =
unités lexicales élémentaires
(caractères, sous-mots ou parfois mots entiers)

- Résolution du problème de mots inconnus (OOV Out Of Vocabulary)
- Meilleure généralisation morphologique (prefixes, …)

Embeddings de position

Définissent la position des tokens dans la phrase

ce vecteur déplace légèrement le token selon où il se trouve dans la phrase, pour que deux occurrences du même mot aient des vecteurs proches mais distincts

👉 sans ces embeddings le modèle produit le même embedding pour

"*le chat mange la souris*"

et "*la souris mange le chat*"

Spécificités de l'inférence dans un Transformer encodeur

1 Préparation du texte d'entrée

- Tokénisation

2 Calcul de la matrice d'embeddings d'entrée

"Le roi et la reine vivent au chateau" →
La phrase est convertie en séquence de tokens avec ajout de marqueurs spéciaux
[CLS] , Le , roi , et, la , reine, vivent , au , château , [SEP]

token de classification

token de séparation

Chaque token est remplacé par son identifiant numérique dans le vocabulaire **SentencePiece (pour Camembert)** (32 000 à 50 000 tokens selon le modèle)

vecteur `input_ids` de longueur 10

Token	ID
[CLS]	0
Le	1278
roi	5304
…	…
château	8125
[SEP]	2

Spécificités de l'inférence dans un Transformer encodeur

🧰 2️⃣ Calcul de la matrice d'embeddings d'entrée

1. Création de la matrice d'embeddings de tokens **E_token**
   Taille = [Vocab_size × Hidden_dim] → par ex. [32 000 × 768]

récupérés dans la Table de lookup des embeddingsà partir des ID de tokens

2. Création de la matrice d'embeddings de position **E_pos**
   Taille = [max_seq_len × Hidden_dim] → [514 × 768] pour CamemBERT

   Chaque ligne correspond à une position dans la séquence (ligne 0=position du premier token, …)

   Ces vecteurs indiquent la position du token dans la phrase (appris)

3. Les deux matrices sont sommées
   X= E_token + E_pos

Token	Embedding (E_token)
Le	[0.10, 0.25, -0.30, 0.05]
roi	[0.20, 0.40, -0.10, 0.15]
reine	[0.18, 0.42, -0.12, 0.13]
château	[0.50, -0.10, 0.25, 0.05]

Position	E_pos
0	[0.01, 0.02, 0.03, 0.04]
1	[0.05, 0.10, 0.15, 0.20]
2	[0.09, 0.18, 0.27, 0.36]
3	[0.13, 0.26, 0.39, 0.52]

les vecteurs de position n'ont pas de signification explicite

Exemples de matrices d'embeddings de tokens dans un Transformer encodeur

```
[d1] Le chat mange des croquettes
Taille séquence = 7 ; dimension = 768
00 <s>          → [ 0.0087  0.0363  0.0243 -0.0603  0.0025 -0.004  -0.0248  0.0653]
01 _Le          → [-0.0803 -0.5432  0.0693 -0.0847 -0.0808 -0.4355  0.153   0.028 ]
02 _chat        → [ 0.1432 -0.3108  0.3444 -0.0643  0.0886  0.2922 -0.1484  0.0214]
03 _mange       → [-0.0287 -0.3534  0.0603 -0.1681 -0.1689 -0.5769 -0.054   0.0368]
04 _des         → [ 0.0219  0.0805  0.0063 -0.2048 -0.0684 -0.2624 -0.069   0.044 ]
05 _croquettes  → [ 0.0378  0.0953  0.3981 -0.0812  0.039   0.073  -0.0946  0.0848]
06 </s>         → [-0.06    0.1087  0.048  -0.2047 -0.1651 -0.0672 -0.0953  0.0367]
```

```
[d3] Un chat est un animal domestique
Taille séquence = 8 ; dimension = 768
00 <s>          → [ 0.0165  0.0239  0.0124 -0.0503  0.0074  0.0018 -0.0176  0.0584]
01 _Un          → [ 0.055  -0.3241 -0.2755 -0.1839  0.1302 -0.5228 -0.1555 -0.095 ]
02 _chat        → [ 0.1577 -0.0803  0.3833 -0.1008  0.1115  0.2819 -0.1008  0.0919]
03 _est         → [ 0.1064 -0.3084 -0.3182 -0.1878  0.0274 -0.6565  0.0491 -0.0629]
04 _un          → [ 0.1376 -0.225  -0.3191  0.0019 -0.0301 -0.139  -0.0411  0.0201]
05 _animal      → [ 0.0678 -0.2288 -0.1005 -0.0376  0.1428 -0.0086 -0.2718 -0.0126]
06 _domestique  → [ 0.0657 -0.2859  0.6732 -0.0518 -0.0976  0.417  -0.3391 -0.0148]
07 </s>         → [-0.0289  0.1522 -0.0005 -0.2202 -0.1277  0.0104 -0.0769  0.0627]
```

Spécificités de l'inférence dans un Transformer encodeur

2 Calcul de la matrice d'embeddings d'entrée

Token	Position	Token embedding	Position embedding	Somme (entrée du modèle)
Le	0	[0.10, 0.25, -0.30, 0.05]	[0.01, 0.02, 0.03, 0.04]	[0.11, 0.27, -0.27, 0.09]
roi	1	[0.20, 0.40, -0.10, 0.15]	[0.05, 0.10, 0.15, 0.20]	[0.25, 0.50, 0.05, 0.35]
reine	2	[0.18, 0.42, -0.12, 0.13]	[0.09, 0.18, 0.27, 0.36]	[0.27, 0.60, 0.15, 0.49]
château	3	[0.50, -0.10, 0.25, 0.05]	[0.13, 0.26, 0.39, 0.52]	[0.63, 0.16, 0.64, 0.57]

La matrice d'entrée est définie par la somme des deux matrices d'embeddings de token et de position

$$X = E_{token} + E_{pos}$$

Spécificités de l'inférence dans un Transformer encodeur

3 Passage dans le réseau (forward pass)

La matrice d'entrée X passe à travers :

- 12 blocs d'auto-attention
- suivis de couches feed-forward (Perceptron Multi Couches)
- avec normalisation et résidus à chaque étape

Ce processus raffine les vecteurs, qui deviennent contextualisés :

chaque token est ainsi représenté *en tenant compte des autres*

4 Sortie du modèle

Sortie de la dernière couche →

last_*hidden_state [seq_len x 768]*

- Chaque ligne correspond à un token de la phrase
- Chaque vecteur (768 dimensions) est contextualisé

Classification (phrase entière)
→ on prend le vecteur du token [CLS] : vecteur contextuel de la phrase

•Pour une recherche sémantique
→ on peut moyenner tous les vecteurs (mean pooling)

BERT : les matrices d'embeddings de tokens

Matrices des embeddings de tokens (token x dimensions)

- Une matrice par entrée (phrase)
  - T lignes: tokens
  - H colonnes : dimensions cachées (768 pour CamemBERT-base)
- La dernière version (dernière couche) : **last_hidden state**

Hidden-states

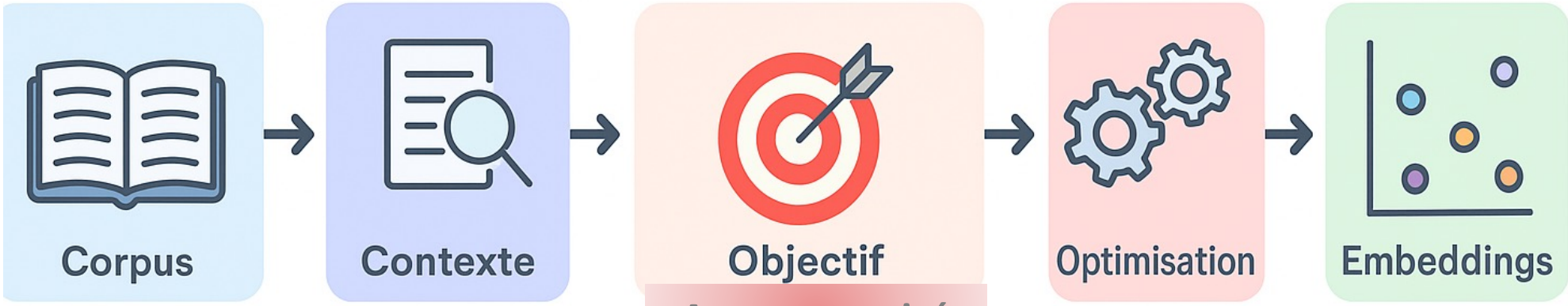
```
La batterie est cassée dans la salle de répétition.
Taille séquence : 12 Dimension vecteurs : 768
```

	token	dim1	dim2	dim3	dim4	dim5	dim6	dim7	dim8	dim9	dim10
0	<s>	0.03	0.12	0.10	-0.09	0.03	-0.02	0.06	0.19	0.13	0.07
1	__La	-0.07	0.30	0.10	-0.03	0.15	-0.24	0.14	0.01	0.14	0.08
2	__batterie	0.04	-0.07	-0.05	0.03	0.07	-0.14	0.11	-0.05	0.09	0.13
3	__est	-0.03	0.02	-0.31	-0.11	-0.06	-0.16	0.18	-0.10	0.11	0.23
4	__cassée	0.05	0.15	0.15	0.08	-0.12	-0.05	0.03	0.09	0.03	0.17
5	__dans	0.08	0.10	-0.09	0.11	0.00	-0.09	0.03	0.06	-0.01	0.20
6	__la	-0.06	0.32	0.08	0.11	0.10	-0.09	0.12	0.03	0.06	0.10
7	__salle	0.01	0.00	-0.22	0.05	0.13	0.01	0.06	-0.11	0.06	0.07
8	__de	-0.10	0.35	-0.10	-0.01	0.05	-0.03	0.10	-0.03	0.15	0.17
9	__répétition	-0.13	-0.13	0.17	0.02	0.11	-0.07	-0.13	0.09	0.14	0.07
10	.	-0.05	0.11	0.12	-0.14	-0.02	-0.00	0.01	0.13	0.09	0.04
11	</s>	-0.08	0.20	0.10	-0.12	-0.03	-0.06	0.08	0.12	0.12	0.07

Tokens ajoutés
<s> = *start of sentence* → [CLS] classification token
</s> = *end of sentence* → [DEP]

avec CamemberBERT-base

Spécificités de l'entrainement d'un transformer encodeur (ex: CamemBERT)



Massif 138Go de texte en francais = 32 milliards de tokens

Contexte global: toute la phrase

Tache MLM *Masked Language Modeling* : prédire les tokens masqués

- Positions apprises

12 couches Transformer + multi-head attention

Un vecteur contextuel par token

📍 Les embeddings de position sont appris en même temps que les embeddings de token : ils permettent au modèle d'apprendre l'ordre des mots et la structure syntaxique en guidant l'attention

Embedding de phrase (sentence embedding)

- Créés uniquement si nécessaire - 3 méthodes possibles :

Méthode	Principe	Avantages ✅	Limites ⚠️	Cas d'usage
CLS	On prend le vecteur du token spécial <s> (équivalent [CLS]) à la sortie de la dernière couche	Simple, natif dans BERT, vecteur unique par phrase	Pas optimisé pour la similarité sémantique	Classification, NSP (Next Sentence Prediction)
Mean pooling	Moyenne des vecteurs de tous les tokens de la phrase (hors padding)	Plus robuste, capture mieux le « centre » du sens global	Peut diluer les informations fines	Similarité entre phrases, clustering
Pooler output	Vecteur CLS projeté par une couche linéaire + tanh (prévu pour NSP)	Vecteur normalisé, utilisé dans l'entraînement original	Entraîné pour NSP, peu pertinent pour la recherche sémantique	Tâches supervisées de classification

Embedding de phrase (sentence embedding)

- Exemple
  - vecteur pour "La batterie est cassée dans la salle de répétition."
  - Taille séquence (nombre de tokens) =12
  - Dimension des vecteurs = 768

	0	1	2	3	4	5	6	7	8	9
dim	dim1	dim2	dim3	dim4	dim5	dim6	dim7	dim8	dim9	dim10
CLS	0.029	0.123	0.1	-0.094	0.031	-0.016	0.062	0.19	0.128	0.073
Mean	-0.025	0.124	0.004	-0.009	0.035	-0.077	0.066	0.036	0.091	0.118
Pooler	-0.094	0.112	0.023	0.036	-0.063	0.161	0.043	0.109	-0.065	-0.025

Aucune solution n’est optimisée pour mesurer directement la similarité sémantique entre phrases.

Sentence-BERT (SBERT)

Modèle spécialisé pour embeddings de phrase

- C’est une adaptation de BERT (fine-tuning avec un objectif de similarité de phrases)
- Conçu spécifiquement pour produire des sentence embeddings de qualité
- Résultat :
  - un seul vecteur par phrase/document, optimisé pour la recherche sémantique, le clustering, etc

Mise en pratique CamemBERT/SentenceBERT

La librairie python 🤗 Transformers

- Librairie Python incontournable pour manipuler des modèles pré-entraînés en NLP.
- Fournit :
  - **Tokenizers** (préparation des textes → tokens).
  - **Models** (chargement des poids et architecture).
  - **Pipelines** (API haut-niveau pour tâches directes : classification, NER, QA…).
- Compatible avec PyTorch & TensorFlow.
- Des milliers de modèles disponibles sur huggingface.co/models

Du texte aux embeddings : vue d'ensemble des étapes

1 Texte

Une phrase ou un ensemble de phrases

« La batterie est cassée. »

2 Tokenisation

Découper le texte en tokens et les convertir en identifiants numériques

[CLS] la batterie est cassée . [SEP]

input_ids

101	345	987	23	876	12	102

attention_mask

1 1 1 1 1 1 1

3 Modèle Transformer

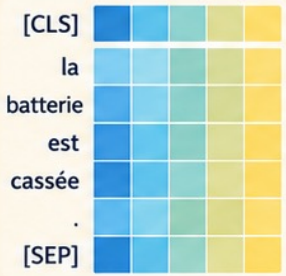
CamemBERT

Les tokens traversent les couches du modèle et sont représentés dans un espace vectoriel.

Couche 12
Couche 2
...
Couche 12

4 Embeddings contextualisés des tokens

Le modèle produit un vecteur pour chaque token de la phrase.



last_hidden_state
[seq_len × 768]

5 Embedding de phrase

On regroupe les vecteurs des tokens pour obtenir un seul vecteur représentant la phrase.

Vecteur [CLS]
utiliser le vecteur du token [CLS]

ou

Mean pooling
moyenne des vecteurs des tokens

1 vecteur de taille 768

6 Modèle spécialisé pour les phrases

Sentence-BERT

Un modèle entraîné pour produire directement un embedding de phrase (de meilleure qualité pour la similarité sémantique).

« La batterie est cassée. »

1 vecteur de taille 768

Deux chemins que nous allons coder dans les diapos suivantes

CamemBERT

- Comprendre comment sont obtenus les embeddings des tokens.
- Observer la sortie last_hidden_state.
- Obtenir un embedding de phrase avec [CLS] ou mean pooling.

Sentence-BERT

- Utiliser un modèle entraîné pour les phrases.
- Obtenir simplement un embedding de phrase avec `model.encode(sentences)`.
- Adapter ces embeddings à des tâches comme la similarité sémantique (recherche, clustering, etc.).

Rappels — De NumPy à PyTorch : manipuler les tableaux / tenseurs

NumPy : tableaux (array)

- Tableaux multidimensionnels de nombres.
- Utilisés pour la manipulation et l'analyse de données.

```python
import numpy as np

a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])

print(a.shape)  # (3, 4)
```

Tableau `a`
Forme : (3, 4)

1	2	3	4
5	6	7	8
9	10	11	12

PyTorch : tenseurs (tensor)

- Tenseurs multidimensionnels de nombres.
- Compatibles avec le **calcul sur GPU et la différentiation automatique** (utiles pour l'entraînement des modèles de deep learning).

```python
import torch

t = torch.tensor([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])

print(t.shape)  # torch.Size([3, 4])
```

Tenseur `t`
Forme : [3, 4]

1	2	3	4
5	6	7	8
9	10	11	12

Principales opérations équivalentes

Opération	NumPy	PyTorch	Exemple / Résultat
Créer un tableau / tenseur	`np.array(...)`	`torch.tensor(...)`	a.shape = (3, 4) ↔ t.shape = [3, 4]
Accéder à la forme	`a.shape`	`t.shape`	(3, 4) ↔ [3, 4]
Ajouter une dimension	`np.expand_dims(a, 0)`	`t.unsqueuse(0)`	(3, 4) → (1, 3, 4)
Supprimer une dimension	`np.squeeze(a, axis=0)`	`t.squeeze(0)`	(1, 3, 4) → (3, 4)
Transposition	`a.T`	`t.T`	(3, 4) → (4, 3)

Conversion NumPy ↔ PyTorch

- **NumPy → PyTorch**

```python
import torch
t = torch.from_numpy(a)
# ou torch.tensor(a)
```

- **PyTorch → NumPy**

```python
a = t.detach().cpu().numpy()
# .detach() : sans gradient
```

À retenir

- Les tableaux NumPy et les tenseurs PyTorch se manipulent de manière très similaire.
- PyTorch permet en plus le calcul sur GPU et la différentiation automatique, indispensables pour les modèles de deep learning.
- Les opérations comme `unsqueeze()` et `squeeze()` sont utilisées pour adapter la forme des tenseurs (ex. dimension batch).

Batch (lot) et opérations squeeze / unsqueeze

- **Batch (lot)** : ensemble d'exemples (ex. phrases) traités simultanément par le modèle.
- La première dimension d'un tenseur correspond au nombre d'exemples (taille du batch).

Données	Forme du tenseur
Un vecteur (une phrase)	[768]
Un batch de 1 phrase	[1, 768]
Un batch de 3 phrases	[3, 768]

1 Exemple simple avec un vecteur

Batch de 1 vecteur

```
x = torch.tensor([[10, 20, 30]])
```

Forme : [1, 3]

batch = 1 | 10 | 20 | 30

Matrice d'une ligne
(1 vecteur dans le lot)

squeeze(0)

2 Après squeeze(0)

```
y = x.squeeze(0)
```

Forme : [3]

10	20	30

Vecteur

3 Exemple avec une matrice

Batch de 1 matrice (3 vecteurs de 4 dimensions)

```
m_batch = m.unsqueees(0)
# ajoute une dimension batch
```

Forme : [1, 3, 4]

batch = 1

1	2	3	4
5	6	7	8
9	10	11	12

Une matrice de 3 × 4 dans le lot
(3 vecteurs, 4 dimensions chacun)

4 Résumé

- `squeeze(dim)` : supprime une dimension de taille 1.
- `unsqueeze(dim)` : ajoute une dimension de taille 1.

À retenir :

- Les valeurs du tenseur ne changent pas, seule sa forme change.
- Ces opérations sont utilisées pour gérer la dimension batch et adapter les tenseurs aux calculs des modèles.

CamemBERT
Importer le modèle et tokeniser

```
# Importer le modèle et le tokenizer
from transformers import CamembertTokenizer, CamembertModel
tokenizer = CamembertTokenizer.from_pretrained("camembert-base")
model = CamembertModel.from_pretrained("camembert-base")

# Encoder (tokenisation) une phrase
sentence = "La batterie est cassée."
inputs = tokenizer(sentence, return_tensors="pt")
```

objet de type BatchEncoding

id des tokens
tableau 2D [batch_size, nb tokens] = [1, 7]

```
# objet BatchEncoding contenant des tenseurs PyTorch
{'input_ids': tensor([[ 5, 61, 2629, 30, 17082, 9, 6]]),
'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1]])}
```

tableau 2D [batch, nb_tokens] avec :
- 1 pour les tokens réels
- 0 pour les tokens padding (token ajoutés pour uniformiser la taille

CamemBERT
Passage dans le modèle

```
# utilisation du modèle en inférence
camem.eval()
with torch.no_grad(): # pas de back propagation

    #matrice des embeddings de position
    pos_embed = camem.embeddings.position_embeddings.weight

    #matrice des embeddings initiale des tokens
    first_embed = camem.get_input_embeddings().weight
```

taille =[max_len, 768]

Vocabulaire complet
taille =[vocab_size, 768]

CamemBERT
Passage dans le modèle

```
# Passage dans le modèle (inférence)
outputs = model(**inputs, return_dict=True)
```

Le résultat est un dictionnaire

Objet spécial HuggingFace :
BaseModelOutputWithPoolingAndCrossAttentions

🔎 **Principaux attributs**

last_hidden_state : Matrice des embeddings de tokens.
Forme : [batch_size, seq_len, hidden_dim]
Exemple : [1, 8, 768] → 1 phrase, 8 tokens, vecteurs de 768 dimensions.

pooler_output (optionnel selon le modèle) : Embedding du token **[CLS]**, passé dans une petite couche supplémentaire (dense + tanh).
Forme : [batch_size, hidden_dim] → ex. [1, 768].

hidden_states *(si output_hidden_states=True):* Liste des matrices pour **chaque couche** du modèle (y compris l'embedding initial).
Longueur : nb_layers + 1
Chaque élément : [batch_size, seq_len, hidden_dim].

attentions *(si output_attentions=True)*
Les poids d'attention pour chaque couche et chaque tête.

CamemBERT : de la phrase aux embeddings des tokens

```python
# Importer le modèle et le tokenizer
tokenizer = CamembertTokenizer.from_pretrained("camembert-base")
model = CamembertModel.from_pretrained("camembert-base")
model.eval() #place le modèle en mode évaluation.

# Encoder une phrase
sentence = "La batterie est cassée."
inputs = tokenizer(sentence, return_tensors="pt")

print("\n=== ENTRÉES DU MODÈLE ===")
print("Phrase :", sentence)
print("Tokens :", tokenizer.convert_ids_to_tokens(inputs["input_ids"][0]))
print("Identifiants :", inputs["input_ids"][0].tolist())
print("Masque d'attention :", inputs["attention_mask"][0].tolist())
print("Dimensions :", tuple(inputs["input_ids"].shape))

# Passage dans CamemBERT
with torch.no_grad():
    #Indique qu'il n'est pas nécessaire de calculer les gradients,
    # puisqu'on utilise le modèle sans l'entraîner
    outputs = model(**inputs, return_dict=True)

# Embeddings contextualisés des tokens
last_hidden = outputs.last_hidden_state

print("\n=== SORTIE DU MODÈLE ===")
print("Dimensions :", tuple(last_hidden.shape))
print("           (batch, tokens, dimension des embeddings)")

print("\nVecteur du premier token (5 premières composantes) :")
print(last_hidden[0, 0, :5].numpy().round(3))
```

```
=== ENTRÉES DU MODÈLE ===
Phrase : La batterie est cassée.
Tokens : ['<s>', '_La', '_batterie', '_est', '_cassée', '.', '</s>']
Identifiants : [5, 61, 2629, 30, 17082, 9, 6]
Masque d'attention : [1, 1, 1, 1, 1, 1, 1]
Dimensions : (1, 7)

=== SORTIE DU MODÈLE ===
Dimensions : (1, 7, 768)
           (batch, tokens, dimension des embeddings)

Vecteur du premier token (5 premières composantes) :
[-0.006  0.097  0.124 -0.087 -0.026]
```

Exemple : 7 tokens, une seule phrase

0	1	2	3	4	5	6
<s>	La	bat	terie	est	cassée	</s>

Découpage schématique : les tokens exacts dépendent du tokenizer.

Forme de `input_ids` : `[1, 7]` = 1 phrase × 7 tokens.

Le rôle de attention_mask

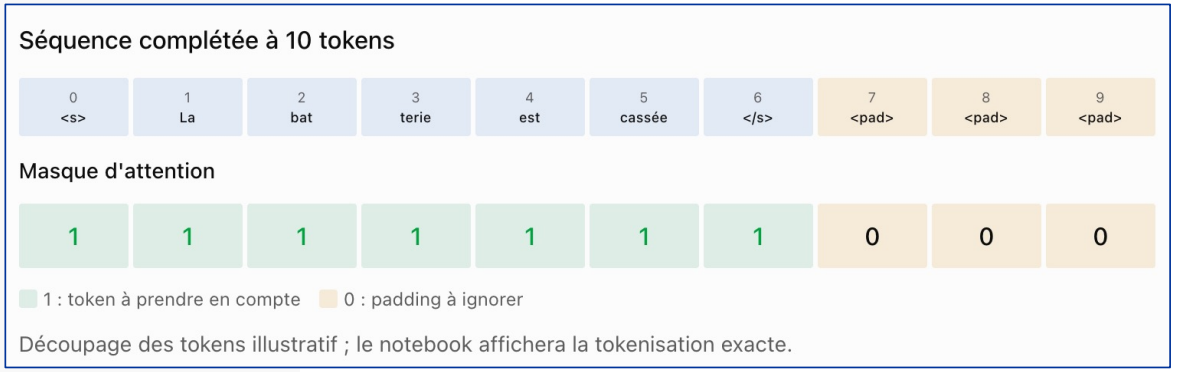
- Lors de la tokenisation, le tokenizer ajoute parfois du **padding** (tokens [PAD]) pour uniformiser les séquences d'un batch (même taille)
- inputs["attention_mask"] est un vecteur [seq_len] avec :
  - 1 pour les tokens réels
  - 0 pour les tokens padding
- Exemple pour une phrase de 5 tokens, padded à 8 :

```
attention_mask = [1, 1, 1, 1, 1, 0, 0, 0]
mask = inputs["attention_mask"].squeeze(0)
```

CamemBERT : Tokenisation avec padding et masque d'attention

```python
# Même phrase, mais avec une longueur fixée à 10 tokens
sentence = "La batterie est cassée."

inputs = tokenizer(
    sentence,
    return_tensors="pt",
    padding="max_length",
    max_length=10
)

print("\n=== ENTRÉES DU MODÈLE AVEC PADDING ===")
print("Phrase :", sentence)
print("Tokens :", tokenizer.convert_ids_to_tokens(inputs["input_ids"][0]))
print("Identifiants :", inputs["input_ids"][0].tolist())
print("Masque d'attention :", inputs["attention_mask"][0].tolist())
print("Dimensions :", tuple(inputs["input_ids"].shape))

# Passage dans CamemBERT
with torch.no_grad():
    outputs = model(**inputs, return_dict=True)

last_hidden = outputs.last_hidden_state

print("\n=== SORTIE DU MODÈLE ===")
print("Dimensions :", tuple(last_hidden.shape))
print("Tokens réels :", int(inputs["attention_mask"].sum()))
```

```
=== ENTRÉES DU MODÈLE AVEC PADDING ===
Phrase : La batterie est cassée.
Tokens : ['<s>', '_La', '_batterie', '_est', '_cassée', '.', '</s>', '<pad>', '<pad>', '<pad>']
Identifiants : [5, 61, 2629, 30, 17082, 9, 6, 1, 1, 1]
Masque d'attention : [1, 1, 1, 1, 1, 1, 1, 0, 0, 0]
Dimensions : (1, 10)

=== SORTIE DU MODÈLE ===
Dimensions : (1, 10, 768)
Tokens réels : 7
```



CamemBERT
Trois façons de générer un embedding de phrase

Une phrase en entrée

« Le chat dort sur le canapé. »

Tokenisation

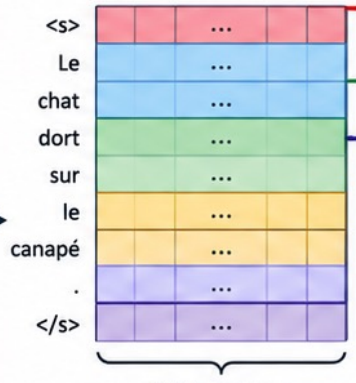
[<s>, Le, chat, dort, sur, le, canapé, ., </s>]

CamemBERT

Sortie du modèle dernière couche

`last_hidden_state`

[batch, tokens, 768]



1 Vecteur du token <s> (CLS)

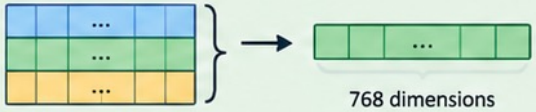
On utilise le vecteur contextualisé du premier token spécial <s>.

```
cls_vec = last_hidden[:, 0, :]
```

[batch, 768]

2 Mean pooling

On calcule la moyenne des vecteurs des tokens utiles (hors padding) à l'aide du masque d'attention.

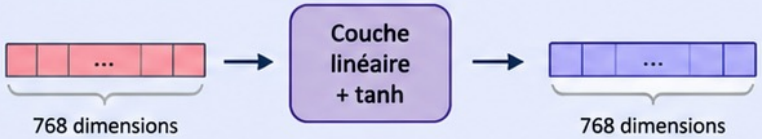



```
mask = inputs["attention_mask"].unsqueeze(-1)
mean_vec = (last_hidden * mask).sum(1) / \
           mask.sum(1)
```

[batch, 768]

3 Pooler output

On utilise le vecteur produist par une couche supplémentaire (transformation apprise du vecteur <s> suivi d'une activation tanh).




```
pooler_vec = outputs.pooler_output
```

[batch, 768]

À retenir

- Les trois méthodes produisent un vecteur de 768 dimensions par phrase.
- Elles ne représentent pas la phrase de la même manière.
- Le *pooler output* n'est pas toujours disponible selon le modèle.

Trois méthodes pour obtenir des embeddings de phrases

```python
# 1. CLS : vecteur du premier token <s>
cls_vec = last_hidden[:, 0, :]

print("\n=== 1. CLS : PREMIER TOKEN <s> ===")
print("Dimensions :", tuple(cls_vec.shape))
print("5 premières valeurs :", cls_vec[0, :5].numpy().round(3))


# 2. MEAN POOLING : moyenne des vecteurs hors padding
mask = inputs["attention_mask"].unsqueeze(-1)

mean_vec = (last_hidden * mask).sum(dim=1) / mask.sum(dim=1)

print("\n=== 2. MEAN POOLING ===")
print("Dimensions :", tuple(mean_vec.shape))
print("Tokens pris en compte :", int(mask.sum()))
print("5 premières valeurs :", mean_vec[0, :5].numpy().round(3))


# 3. POOLER OUTPUT : transformation du vecteur <s>
print("\n=== 3. POOLER OUTPUT ===")

if outputs.pooler_output is not None:
    pooler_vec = outputs.pooler_output
    print("Dimensions :", tuple(pooler_vec.shape))
    print("5 premières valeurs :", pooler_vec[0, :5].numpy().round(3))
else:
    print("Pooler output non disponible pour ce modèle.")
```

```
=== 1. CLS : PREMIER TOKEN <s> ===
Dimensions : (1, 768)
5 premières valeurs : [-0.006  0.097  0.124 -0.087 -0.026]

=== 2. MEAN POOLING ===
Dimensions : (1, 768)
Tokens pris en compte : 7
5 premières valeurs : [-0.053  0.045  0.059 -0.044 -0.005]

=== 3. POOLER OUTPUT ===
Dimensions : (1, 768)
5 premières valeurs : [-0.082  0.097  0.017  0.02  -0.045]
```

Sentence Bert
Charger le modèle

1. Importer la librairie spécifique (complémentaire de transformers)

```
from sentence_transformers import SentenceTransformer
```

2. Charger le modèle pré-entrainé

```
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
```

- Le modèle est récupéré automatiquement depuis le Hub Hugging Face : 🔗 https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
- Réutilisable offline après le premier téléchargement

Hugging Face

Sentence Bert
Obtenir les embeddings

```
# Importer Sentence-BERT
from sentence_transformers import SentenceTransformer
# Charger un modèle multilingue optimisé pour similarité
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
# Encoder une ou plusieurs phrases
sentences = [
      "Le chat dort sur le canapé.",
      "Un félin se repose sur le sofa."]
embeddings = model.encode(sentences)
print(embeddings.shape) # (nombre de phrases=2, dimensions=384)
print(embeddings) # matrice d'embeddings de phrase
```

`model.encode(liste_de_phrases)` → produit un tableau NumPy (nb_phrases × dim).
`print(embeddings.shape)` → dimensions de la matrice.
`embeddings[i]` → vecteur de la phrase *i*.

BERT/Sentence Bert
Manipuler les embeddings

```
# Similarité cosinus
from sklearn.metrics.pairwise import cosine_similarity
sim = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]

# Distance euclidienne
import numpy as np
dist = np.linalg.norm(embeddings[0] - embeddings[1])

# Réduction dimensionnelle
from sklearn.decomposition import PCA
pts = PCA(n_components=2).fit_transform(embeddings)

# Visualisation rapide
import matplotlib.pyplot as plt
plt.scatter(pts[:,0], pts[:,1])
```

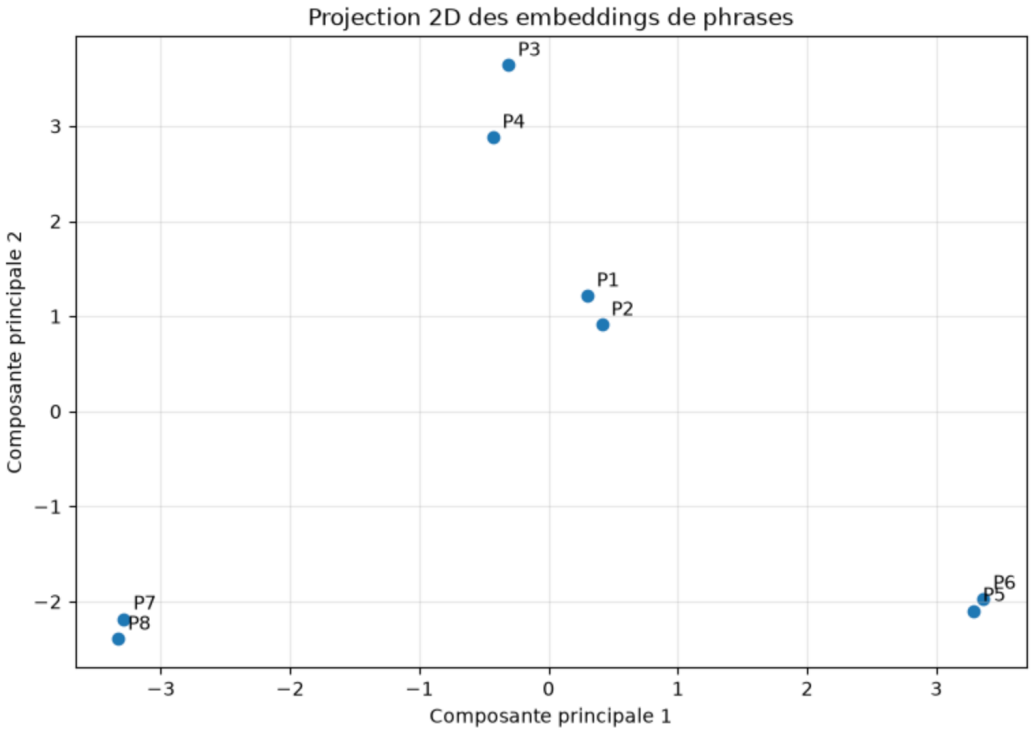
Sentence Bert : générer et comparer des embeddings de phrases

```python
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Charger un modèle multilingue de Sentence-BERT
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

# Deux phrases de sens proche
sentences = [
    "Le chat dort sur le canapé.",
    "Un félin se repose sur le sofa."
]

# Générer directement les embeddings de phrases
embeddings = model.encode(sentences)

print("=== EMBEDDINGS DE PHRASES ===")
print("Nombre de phrases :", len(sentences))
print("Dimensions :", embeddings.shape)  # (2, 384)

for i, phrase in enumerate(sentences):
    print(f"\nPhrase {i+1} : {phrase}")
    print("5 premières composantes :", embeddings[i, :5].round(3))

# Comparer les deux embeddings
sim = cosine_similarity([embeddings[0]], [embeddings[1]])[0, 0]
dist = np.linalg.norm(embeddings[0] - embeddings[1])

print("\n=== COMPARAISON ===")
print(f"Similarité cosinus : {sim:.3f}")
print(f"Distance euclidienne : {dist:.3f}")
```

```
=== EMBEDDINGS DE PHRASES ===
Nombre de phrases : 2
Dimensions : (2, 384)

Phrase 1 : Le chat dort sur le canapé.
5 premières composantes : [ 0.402 -0.287 -0.135  0.135  0.127]

Phrase 2 : Un félin se repose sur le sofa.
5 premières composantes : [ 0.213 -0.311 -0.041  0.032  0.195]

=== COMPARAISON ===
Similarité cosinus : 0.763
Distance euclidienne : 3.320
```

Sentence BERT : visualiser les embeddings de phrases

```python
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

sentences = [
    "Le chat dort sur le canapé.",
    "Un félin se repose sur le sofa.",
    "Le chien joue dans le jardin.",
    "Un animal court dans le parc.",
    "La batterie du téléphone est déchargée.",
    "Mon smartphone n'a plus d'énergie.",
    "Le train arrive à la gare.",
    "Un voyageur attend son train."
]

embeddings = model.encode(sentences)

pts = PCA(n_components=2).fit_transform(embeddings)

plt.figure(figsize=(9, 6))
plt.scatter(pts[:, 0], pts[:, 1])

for i, phrase in enumerate(sentences):
    plt.annotate(f"P{i+1}", (pts[i, 0], pts[i, 1]),
                 xytext=(5, 5), textcoords="offset points")

plt.xlabel("Composante principale 1")
plt.ylabel("Composante principale 2")
plt.title("Projection 2D des embeddings de phrases")
plt.grid(alpha=0.3)
plt.show()

for i, phrase in enumerate(sentences):
    print(f"P{i+1} : {phrase}")
```




```
P1 : Le chat dort sur le canapé.
P2 : Un félin se repose sur le sofa.
P3 : Le chien joue dans le jardin.
P4 : Un animal court dans le parc.
P5 : La batterie du téléphone est déchargée.
P6 : Mon smartphone n'a plus d'énergie.
P7 : Le train arrive à la gare.
P8 : Un voyageur attend son train.
```

A vous de jouer : TP2-SentenceEmb.pynb

CamemBERT (phrases) vs Sentence-BERT : comprendre et comparer les embeddings de phrases

Récupérez le notebook TP2-SentenceEmb.pynb sur Teams

- Partie A — CamemBERT (embeddings de tokens → embeddings de phrase)
  - Afficher les tokens et la matrice des embeddings de tokens initiale (entrée dans le modèle)
  - Visualiser la matrice d'embeddings de position (initiale)
  - Afficher la matrice d'embeddings de tokens (dernière couche)
  - Comparer les matrices initiales (Tokens+Positions) et la matrice finale
  - Evaluer l'évolution des embeddings : comparaison des vecteurs initiaux et finaux (similarité cosinus)
  - Visualiser les embeddings de tokens en 2D (PCA)
  - Générer et visualiser des embeddings de phrase avec 3 méthodes
  - Calculer la similarité Cosinus pour deux paires de phrases

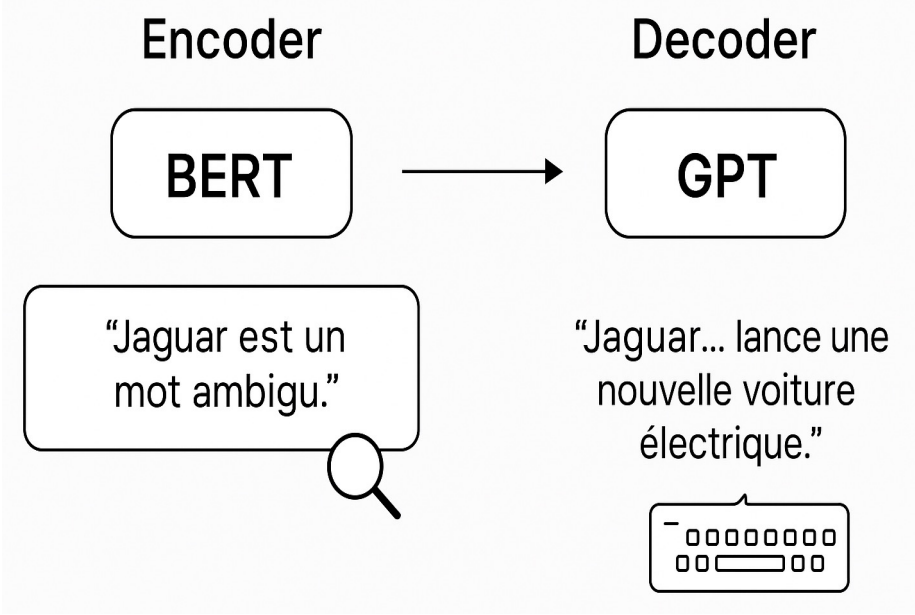
A vous de jouer : TP2-SentenceEmb.pynb

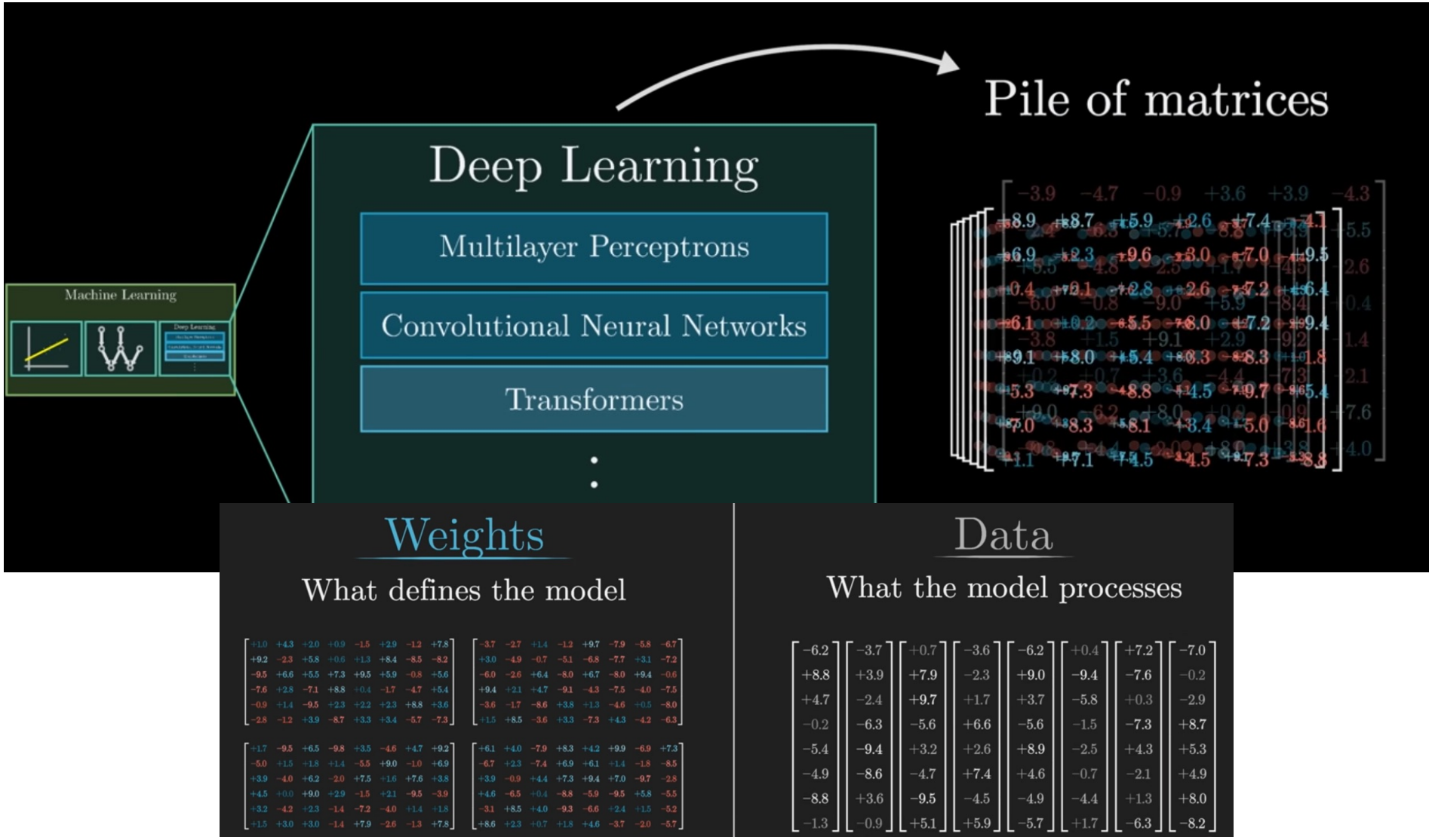
- Partie B — Sentence-BERT (embeddings de phrase optimisés)
  - Encoder les mêmes phrases et comparer les similarités cosinus par paires entre SentenceBert et Camembert
  - Mini-recherche
    - analyser un code non commenté.
    - identifier les indicateurs utilisés pour évaluer la qualité d'un système de recherche sémantique

Hugging Face

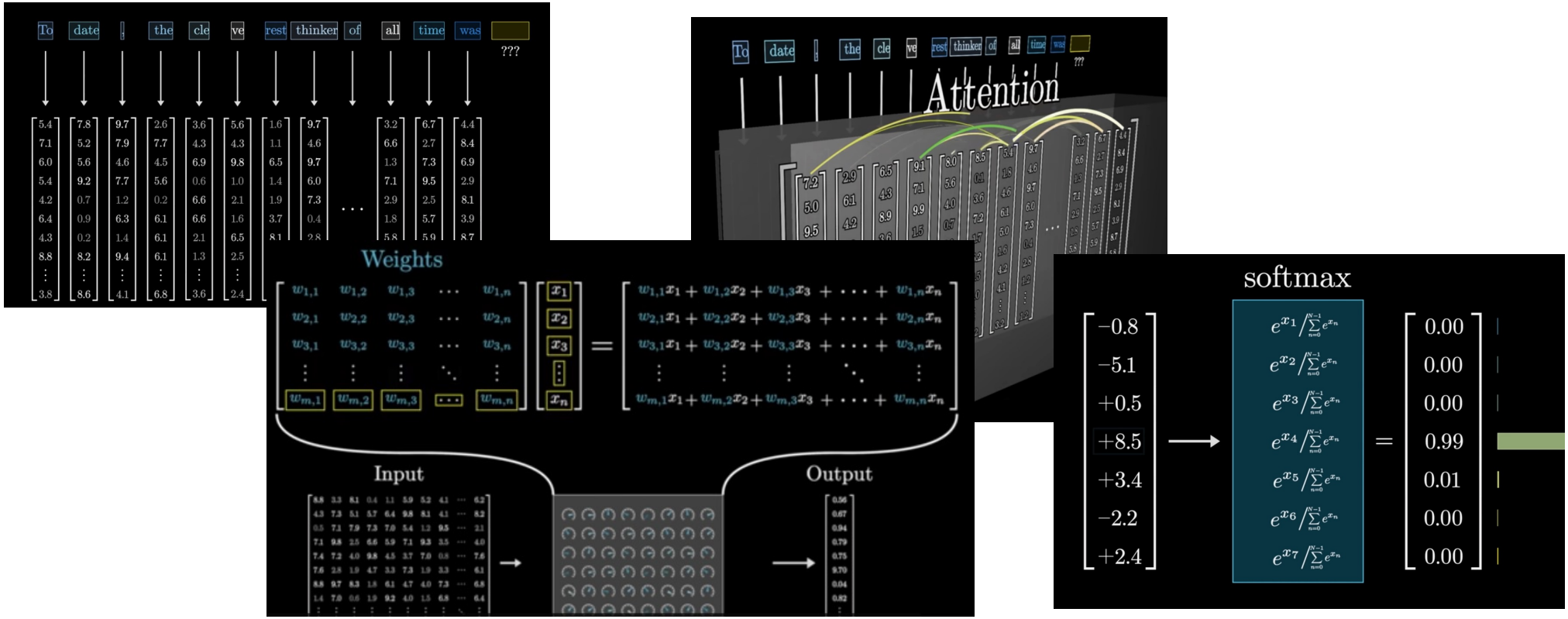
De BERT -> Sentence BERT aux LLM

- Les LLM sont construits sur les mêmes bases (Transformers)
- Ils génèrent aussi des embeddings internes exploitables pour chaque token
- Des vocations différentes :
  - BERT = modèle d'encodage
    (vocation= comprendre le langage)
  - Sentence BERT = modèle d'embeddings
    (vocation= produire des embeddings de phrase)
  - GPT & LLaMA = modèles génératifs
    (vocation= générer du texte)



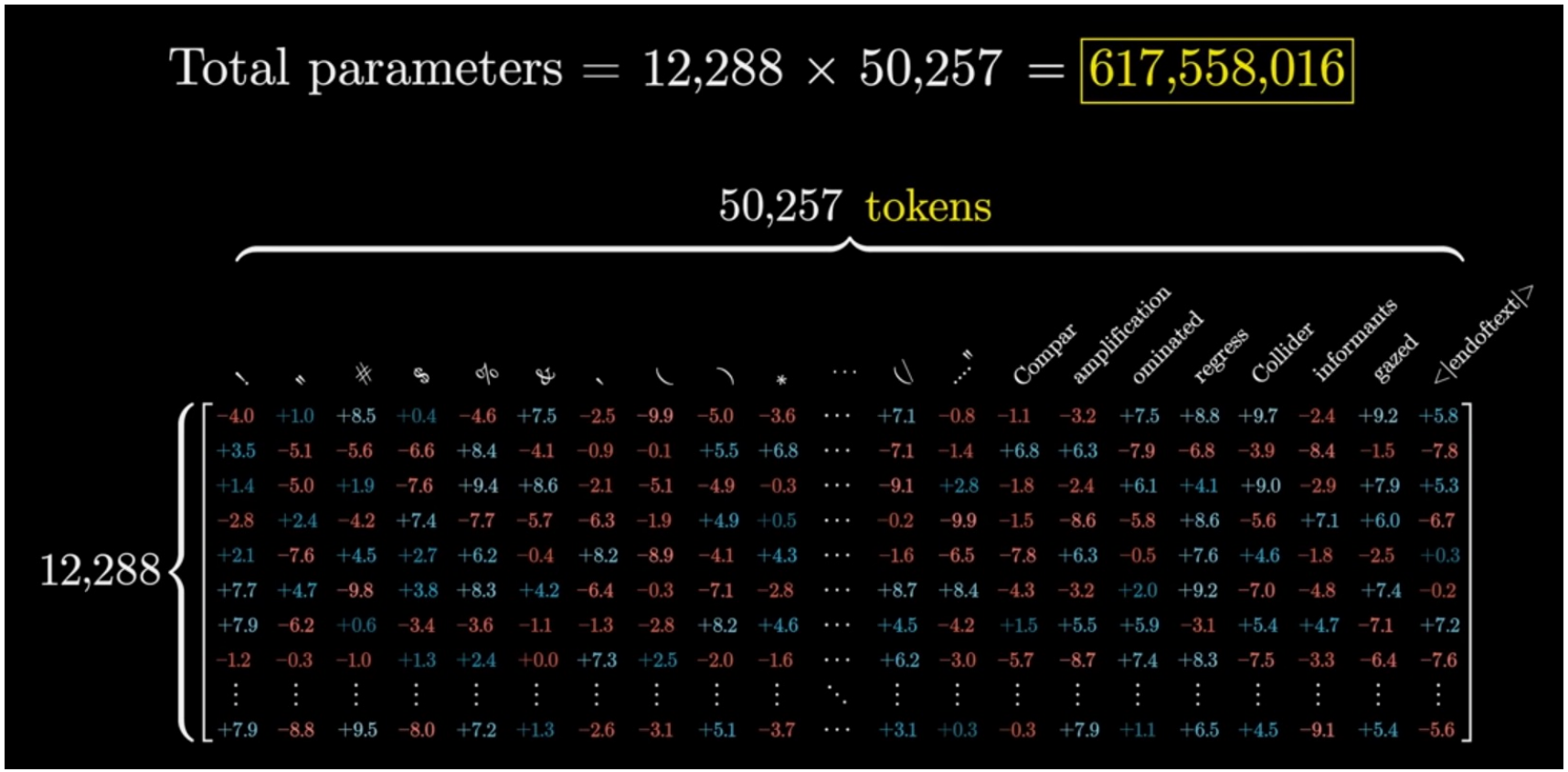

Machine Learning
Deep Learning
Multilayer Perceptrons
Convolutional Neural Networks
Transformers
Pile of matrices
Weights
What defines the model
Data
What the model processes

Etapes d'entrainement des transformers



video sur les transformers: https://www.youtube.com/watch?v=wjZofJX0v4M

W_E matrice d'embeddings de GPT3 (Davinci)



Recherche de similarité sémantique : Limites des modèles d'embeddings

🔍 Problème 1 : **Scalabilité**

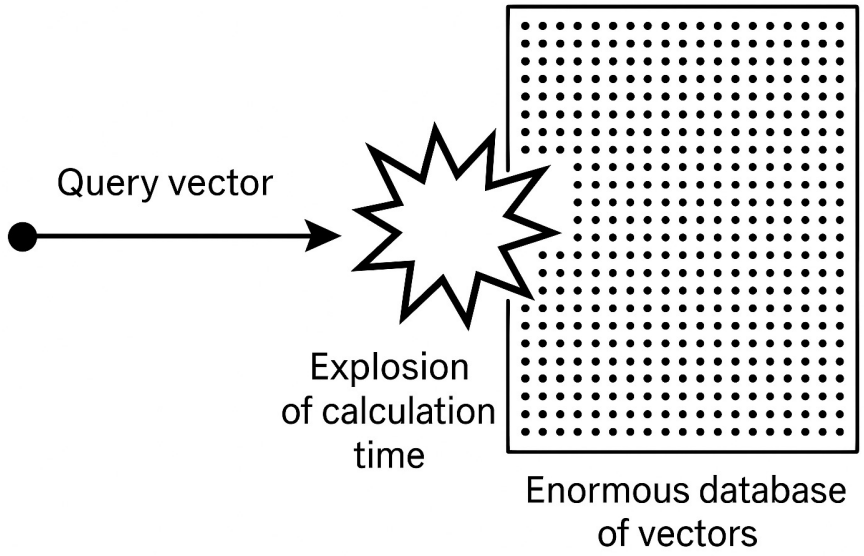
→ comparer un vecteur à des millions d'autres devient trop lent

⚡ Problème 2 : **Efficacité mémoire**

→ stockage de vecteurs haute dimension

🧭 Problème 3 : **Recherche avancée**

→ besoin de fonctions plus riches qu'un simple "plus proche voisin" (filtrage, métadonnées, mises à jour temps réel)



Nécessité d'outils supplémentaires

Pour en savoir plus

- Des vidéos particulièrement claires :
  - chaine YouTube **3Blue1Brown**
    - **cours Neural Networks**
      - https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- Des explications détaillées sur le fonctionnement d'un transformer (BERT)
  - https://lesdieuxducode.com/blog/2019/4/bert--le-transformer-model-qui-sentraine-et-qui-represente